

Introduction to Data Science -- Complete Topic Pages

Contents

0.1) Example Data Science Project	19
Outline	19
Overall philosophy	20
Learning objectives	20
Module structure	21
Running project repository	24
Data sources	24
0.2) R, RStudio, and Visual Studio Code Guide	25
Outline	25
Food Systems and Data Science Certificate	25
1. Software Overview	26
2. Installing R	26
3. Installing RStudio	27
4. Installing Visual Studio Code	28
5. Recommended VS Code Extensions	29
6. Installing Course Packages	29
7. Using RStudio	29
8. Working with Projects	30
9. Using Visual Studio Code	30
10. Troubleshooting	30
11. Best Practices	31
Summary	31
1.1) Why use Git and GitHub?	32
Outline	32
Learning objectives	32
A common problem	32
Git and GitHub are different tools	33

Why Git matters in data science	33
A simple mental model	34
What makes a useful commit?	34
What Git should not track	35
How this connects to the maize yield project	35
Check your understanding	35
Videos	36
Further reading	36
1.2) Version control and collaboration concepts	37
Learning objectives	37
Git records project history	37
GitHub supports shared work	37
Four states of a tracked change	38
Branches and synchronization	38
Track deliberately	39
Relationship to the other pages	39
Key message	39
1.3) Set up Git and GitHub	40
Outline	40
Learning objectives	40
1. Install Git	40
2. Create a GitHub account	41
3. Configure your Git identity	42
4. Check for an existing SSH key	42
5. Generate an SSH key	42
6. Add the key to the SSH agent	43
7. Add the public key to GitHub	43
8. Test the connection	44
Videos	44
Final checklist	44
Common problems	45

1.4) Create your maize yield repository	46
Outline	46
Goal	46
Before you start	47
1. Create an empty repository on GitHub	47
2. Clone the public course repository	47
3. Inspect the existing remote	48
4. Rename the course remote	48
5. Add your repository as origin	48
6. Push to your repository	49
7. Verify the setup	49
Videos	50
Your normal workflow	50
Receive later course updates	50
Troubleshooting	51
Completion checklist	52
 1.5) A collaborative Git workflow	 53
Outline	53
Learning objectives	53
The setting	54
The basic cycle	54
Before making a change	54
Scenario 1: Simple commit	55
Scenario 2: Non-conflicting diverging snapshots	56
Scenario 3: Conflicting diverging snapshots	58
If you are not ready to resolve a merge	60
Comparison of the scenarios	61
Habits that reduce problems	61
Videos	61
Practice exercise	62
Command summary	62
Further reading	63

1.6) A reproducible AI-assisted research workflow	64
Outline	64
Learning objectives	64
Place in the session	64
What reproducibility means here	65
AI assists; researchers remain responsible	66
The review–verify–commit loop	67
Prompting for reviewable work	68
Further detail	69
1.7) AI-assisted research workflow reference	70
Outline	70
The evidence to preserve	72
1. Begin from a known repository state	73
2. Define one bounded task	74
3. Choose a safe working context	75
4. Give the agent sufficient context and boundaries	76
5. Inspect every change	76
6. Verify behavior and research meaning	78
7. Record material decisions	79
8. Commit one reviewed outcome	80
9. Integrate and publish deliberately	81
Handling data, secrets, and external services	82
Research integrity and disclosure	83
What cannot be reproduced exactly	84
Failure and recovery	85
Application to the maize-yield project	86
Practice exercise	87
Completion checklist	88
Check your understanding	89
Further resources	89
2.1) Why use reproducible environments?	91
Learning objectives	91
The problem: “It works on my machine”	91
What is a computational environment?	91
Isolation, portability, and reproducibility	91

Two complementary tools	92
Why this matters in food-systems data science	92
What these tools do not guarantee	92
A practical reproducibility record	93
Check your understanding	93
Videos	93
Further reading	93
2.2) Reproducible-environment concepts	94
Learning objectives	94
What is a computational environment?	94
renv manages the R package layer	94
Docker records a broader execution context	95
renv and Docker are complementary	95
Reproducible environment versus reproducible analysis	96
Continue with the applications	96
Key message	96
2.3) Set up and use renv	97
Outline	97
Learning objectives	97
What renv manages	97
This repository is already initialized	98
1. Install R	98
2. Open the project correctly	98
3. Restore the recorded environment	99
4. Verify the environment	99
The everyday renv workflow	100
snapshot() and restore() point in opposite directions	101
Updating packages	101
Initialize renv in a different project	102
How this project uses renv	102
Troubleshooting	103
Completion checklist	104
Videos	104
Further reading	104

2.4) Apply renv to the example project	105
Learning objectives	105
Before you begin	105
Inspect the environment record	105
Restore the package library	106
Where restored packages live	106
Troubleshooting a failed restore	107
Run the project in the restored environment	107
Change dependencies deliberately	107
Verification checklist	108
Handoff to Docker	108
2.5) Install and set up Docker	109
Outline	109
Learning objectives	109
Docker concepts	110
Choose an installation method	111
Windows installation	111
macOS installation	111
Linux installation	112
Verify the complete setup	112
Verify Docker Compose	113
Understand the maize project image	114
Resource and security considerations	114
Troubleshooting	115
Completion checklist	116
Videos	116
Further reading	116
2.6) Apply Docker to the example project	117
Learning objectives	117
Before you begin	117
Inspect the build recipe	117
Build the image	117
When a build fails	118
Run the pipeline	118
Choosing what to mount	119

Stopping and reusing containers	119
Validate the run	119
Verification checklist	119
Key message	119
3.1) Why learn SSH and the Linux command line?	120
Learning objectives	120
From a laptop to other computers	120
Linux, terminal, and shell are related but different	120
Why the command line matters in data science	120
What SSH provides	121
Two SSH uses in this course	122
Public-key authentication	122
Command-line care	122
How this connects to the maize yield project	123
Check your understanding	123
Videos	123
Further reading	123
3.2) Remote-computing concepts	124
Learning objectives	124
Computing contexts	124
Related terms	124
SSH connections	125
Filesystems and paths	125
Processes, streams, and pipes	125
Sessions and disconnection	126
Data transfer and execution are separate	126
Continue with the applications	126
Key message	126
3.3) Use SSH with GitHub and remote servers	127
Learning objectives	127
Two connections with different outcomes	127
Part 1: Review SSH access to GitHub	127
Part 2: Connect to a remote computing server	128
Optional: alias and file transfer	129
Keys, jobs, and disconnection	130

Troubleshooting	130
Check your work	130
Videos	131
Further reading	131
3.4) Use Linux inside the maize yield container	132
Learning objectives	132
Before you begin	132
Method 1: Create and enter a container directly	132
Establish your context	133
Navigate through the project	133
Inspect text and data, and combine commands	133
Inspect storage and processes	133
Run project operations	134
A brief Vim exercise	134
Persistence: image, container, and mounts	135
Method 2: Enter an already running container	135
Troubleshooting	135
Check your work	136
Videos	136
Further reading	136
4.1) Why manage data?	137
Outline	137
Learning objectives	137
The session learning path	137
Reproducible does not automatically mean correct	138
Data are not self-explanatory	138
The data lifecycle	139
What data management contributes	139
Why this matters in food-systems research	140
What data management cannot guarantee	140
How this connects to the maize-yield project	141
Check your understanding	141
Further resources	141
Continue to Concepts	142

4.2) Understand data-management concepts	143
Outline	143
Learning objectives	143
Place in the session	144
Observations, variables, and values	144
Dataset grain	145
Identifiers, labels, and keys	145
Common data structures	145
Metadata	146
Data dictionaries	146
Provenance	147
Data quality and fitness for purpose	147
Project organization	148
Licensing, privacy, and FAIR principles	149
From concepts to application	149
Check your understanding	150
Further resources	150
4.3) Documentation tools for data management	151
Outline	151
Learning objectives	151
Why use plain-text documentation?	151
Choose a format by information structure	152
How the formats work together	154
SHA-256 checksums for file identity	155
Continue to the reference page	156
4.4) Manage the maize-yield data	157
Outline	157
Learning objectives	157
Place in the session	158
Scenario and deliverable	158
Before you begin	159
1. Preserve and identify the raw input	159
2. Establish the dataset grain	159
3. Create a data dictionary	160
4. Record provenance	161

5. Organize and govern the project artifacts	161
6. Validate structure	162
7. Validate meaning	163
8. Report rather than silently repair	164
Troubleshooting	164
Completion checklist	165
Reflect on the application	165
Further resources	165
4.5) Documentation tools for data management — reference	167
Outline	167
Markdown for explanations and guidance	168
YAML for structured records and configuration	171
CSV for rectangular documentation	174
A practical checksum workflow	176
Validating documentation and configuration	179
Application to the maize-yield project	182
Common mistakes	183
Completion checklist	185
Check your understanding	186
Further resources	186
5.1) Why integrate data?	187
Outline	187
Learning objectives	187
Place in the session	187
Food-systems questions need several sources	188
Acquisition supports integration	188
Integration is more than joining tables	189
Common integration dimensions	189
A reproducible integration workflow	190
What can go wrong	190
How this connects to the maize-yield project	191
Check your understanding	191
Further resources	192
Continue to Concepts	192

5.2) Understand data-integration concepts	193
Outline	193
Learning objectives	193
Place in the session	194
Data management, acquisition, integration, and preparation	194
Begin with a data requirement	195
Evaluate a source	195
Access methods	195
Tabular and semi-structured formats	196
Spatial and temporal data	197
Grain, keys, and join relationships	197
Identifiers and crosswalks	198
Alignment across sources	198
Join choice and integration audits	198
Lineage	199
Plan for changing services	199
Credentials and responsible access	199
Source and acquisition record	199
Check your understanding	200
Further resources	200
Continue to Application	201
 5.3) Integrate maize-yield and precipitation data	 202
Outline	202
Learning objectives	202
Place in the session	203
Scenario and deliverable	203
1. Identify the information gap	203
2. Understand and validate each source	204
3. Define the integration contract	205
4. Align identifiers, space, and time	205
5. Integrate and audit the sources	206
6. Document and interpret the result	207
Extending the integration	207
Troubleshooting	208
Completion checklist	208
Reflect on the application	209
Further resources	209

6.1) Why prepare data?	210
Outline	210
Learning objectives	210
Place in the session	210
Managed and integrated data are not automatically analysis-ready	211
Preparation decisions are analytical decisions	211
Preparation happens around integration	211
Preparation should create new artifacts	212
Common preparation decisions	212
What can go wrong	212
How this connects to the maize-yield project	213
Check your understanding	213
Further resources	213
Continue to Concepts	214
 6.2) Understand data-preparation concepts	 215
Outline	215
Learning objectives	215
Place in the session	215
Preparation and related activities	216
Begin with an analysis contract	216
Observations, variables, and tidy structure	217
Select and filter	217
Rename, parse, and recode	218
Reshape data	218
Convert units	219
Missing values and structural absence	219
Duplicates and repeated observations	219
Outliers and errors	219
Derived variables and transformations	220
Data types and valid domains	220
Preparation audits	220
Lineage and documentation	221
Preparation and data leakage	221
Idempotence and deterministic outputs	221
Check your understanding	222
Further resources	222
Continue to Application	222

6.3) Prepare the maize country-year data	223
Outline	223
Learning objectives	223
Place in the session	223
Scenario and deliverables	224
Before you begin	224
1. State the preparation contract	225
2. Inspect the managed input	226
3. Validate mappings before reshaping	226
4. Create the country-year maize panel	227
5. Verify unit conversion and derived variables	228
6. Integrate the prepared panel with CHIRPS	230
7. Audit the prepared artifacts	230
8. Document preparation and lineage	231
Preparation before predictive modeling	232
Troubleshooting	232
Completion checklist	233
Reflect on the application	233
Further resources	233
 7.1) Why visualize data?	 234
Outline	234
Learning objectives	234
Place in the session	234
Prepared data do not reveal patterns by themselves	235
A graphic is an analytical choice	235
Exploration and communication have different purposes	235
Visualization supports quality review	236
What can go wrong	236
How this connects to the maize-yield project	237
Initial activity	237
Check your understanding	238
Further resources	238
Continue to Concepts	238

7.2) Understand data-visualization concepts	239
Outline	239
Learning objectives	239
Place in the session	240
Use a question-to-graphic workflow	240
Know the observation grain	240
Match questions and variable types	241
Use a grammar of graphics	241
Choose visual encodings deliberately	242
Understand marks and geometric objects	242
Treat scales as part of the interpretation	242
Make grouping and aggregation explicit	243
Address overplotting	243
Use facets for repeated comparisons	244
Design colour and text accessibly	244
Separate exploratory and explanatory graphics	245
Export reproducible figures	245
Interpret without overclaiming	245
Check your understanding	246
Further resources	246
Continue to Application	247
 7.3) Visualize maize yield and precipitation	 248
Outline	248
Learning objectives	248
Place in the session	248
Scenario and deliverables	249
Before you begin	249
1. State visual questions and plot contracts	250
2. Inspect the analytical artifacts	251
3. Examine the yield distribution	252
4. Compare yield trends across countries	253
5. Examine growing-season precipitation	253
6. Explore yield and precipitation together	254
7. Refine one communication graphic	255
8. Export and review the figure artifacts	256
Independent extension	256

Troubleshooting	257
Completion checklist	257
Reflect on the application	257
Further resources	258
8.1) Why describe data numerically?	259
Outline	259
Learning objectives	259
Place in the session	259
Visual patterns need numerical descriptions	260
A single average is not a description	260
Descriptions depend on scope	260
Data-generating processes can change	261
Why stationarity matters before modeling	261
What can go wrong	261
How this connects to the maize-yield project	261
Initial activity	262
Check your understanding	262
Further resources	262
Continue to Concepts	262
8.2) Understand descriptive-data-analysis concepts	263
Outline	263
Learning objectives	263
Place in the session	264
Start with a descriptive contract	264
Define population, sample, and observation grain	264
Report coverage and missingness	265
Describe location	265
Describe dispersion	265
Describe distribution shape	266
Compare groups and periods deliberately	266
Quantify association	266
Separate pooled and within-group association	267
Account for ordered observations	267
Understand stationarity	267
Diagnose change descriptively	268

Treat formal tests as optional evidence	268
Connect description to modeling decisions	269
Check your understanding	269
Further resources	269
Continue to Application	270
8.3) Describe maize yield and precipitation	271
Outline	271
Learning objectives	271
Place in the session	271
Scenario and deliverables	272
Before you begin	272
1. Define descriptive contracts	274
2. Inspect coverage and denominators	274
3. Describe maize yield by country	275
4. Compare meaningful periods	275
5. Describe precipitation	276
6. Quantify yield-precipitation association	277
7. Assess evidence about stability	278
8. Write the modeling handoff	278
Independent extension	279
Troubleshooting	280
Completion checklist	280
Reflect on the application	280
Further resources	281
9.1) Why explanatory modeling requires causal reasoning	282
Outline	282
Learning objectives	282
Place in the session	282
Description is not explanation	283
A regression coefficient does not create causality	283
Causal questions compare potential outcomes	283
Research design comes before estimation	284
The precipitation intervention is difficult to define	284
What can go wrong	285
How this connects to the maize-yield project	285

Initial activity	285
Check your understanding	286
Further resources	286
Continue to Concepts	286
9.2) Understand explanatory-modeling concepts	287
Outline	287
Learning objectives	287
Place in the session	287
Separate question, estimand, identification, and estimation	288
Use potential outcomes to define causal effects	288
Make causal structure explicit	289
Distinguish variable roles	289
Evaluate identification assumptions	290
Understand linear regression	290
Interpret coefficients conditionally	291
Represent countries and time	291
Check functional form and interactions	291
Interpret uncertainty carefully	292
Use diagnostics for their proper purpose	292
Compare specifications as sensitivity evidence	292
Bound the causal conclusion	293
Check your understanding	293
Further resources	293
Continue to Application	293
9.3) Conduct a causal analysis of maize yield	294
Outline	294
Learning objectives	294
Place in the session	294
Scenario and deliverables	295
Before you begin	295
1. Define the causal question and estimand	296
2. Draw the causal diagram	296
3. Assess identification before estimation	297
4. Inspect exposure support	297
5. Fit the planned regression sequence	299

6. Interpret estimates and uncertainty	300
7. Diagnose models and test specification sensitivity	300
8. Write a bounded causal conclusion	301
Independent extension	302
Troubleshooting	302
Completion checklist	303
Reflect on the application	303
Further resources	303
10.1) Why predictive modeling requires unseen-data evaluation	304
Outline	304
Learning objectives	304
Place in the session	304
Prediction concerns observations not used for fitting	305
Explanation and prediction are different objectives	305
Intended use determines meaningful evaluation	305
Time makes maize-yield prediction difficult	305
Simple baselines are scientifically useful	306
What can go wrong	306
How this connects to the maize-yield project	306
Initial activity	307
Check your understanding	307
Further resources	307
Continue to Concepts	307
10.2) Understand predictive-modeling concepts	308
Outline	308
Learning objectives	308
Place in the session	308
Begin with a prediction contract	309
Separate training, validation, and testing	309
Choose a split that represents use	310
Prevent leakage	310
Evaluate the complete workflow	311
Use baselines to define useful performance	311
Understand prediction errors, MAE, and RMSE	312
Evaluate errors beyond one average	312

Recognize overfitting and distribution shift	313
Handle transformed outcomes carefully	313
Define reproducible prediction artifacts	313
Bound interpretation and use	314
Check your understanding	314
Further resources	314
Continue to Application	315
10.3) Evaluate predictive models for maize yield	316
Outline	316
Learning objectives	316
Place in the session	316
Scenario and deliverables	317
Before you begin	317
1. Write the prediction contract	318
2. Audit the temporal split	318
3. Inspect the candidate baselines	319
4. Run the predictive workflow	320
5. Reconstruct and verify the metrics	320
6. Diagnose errors by country and year	321
7. Compare evidence with intended use	322
8. Write a bounded predictive conclusion	323
Independent extension	323
Troubleshooting	323
Completion checklist	324
Reflect on the application	324
Further resources	324

0.1) Example Data Science Project

Outline

- Outline
- Overall philosophy
- Learning objectives
- Module structure
 - Week 0 – Welcome
 - Week 1 – Data Science Project Environment
 - Week 2 – Data Management
 - Week 3 – Data Preparation & Visualization

- Week 4 – Data Analysis
 - Week 5 – Summary
 - Running project repository
 - Data sources
-

Overall philosophy

The module is built around **one continuous, reproducible data science project** rather than a collection of disconnected examples.

Running case study:

Understanding changes in maize yield in Southern Africa

This allows every topic to be introduced within the same workflow:

```
Research Question
    ↓
Data Acquisition
    ↓
Data Management
    ↓
Data Preparation
    ↓
Exploratory Data Analysis
    ↓
Visualization
    ↓
Modeling
    ↓
Evaluation
    ↓
Interpretation
    ↓
Communication & Reproducibility
```

The project should use real **FAOSTAT** data and, optionally, selected World Bank indicators.

Learning objectives

After completing the module, participants should be able to

- organize a reproducible data science project;
- understand common food-system datasets;
- manage and document data;
- clean and prepare data for analysis;
- create informative visualizations;
- fit and interpret simple statistical models;

- distinguish explanation from prediction;
 - communicate results in a reproducible report.
-

Module structure

Week 0 – Welcome

- Introduction
 - Expectations
 - Course organization
 - Overview of the complete workflow
-

Week 1 – Data Science Project Environment

Topics

- Project organization
- RStudio Project
- Git
- GitHub / GitLab
- Reproducible environments
- renv
- Containers (overview)
- Local vs remote computing
- SSH (conceptual)

Deliverable

- Students can run the supplied project and reproduce all analyses.

Suggested resources

- The Turing Way (Reproducibility)
-

Week 2 – Data Management

Topics

- What is data?
- Structured vs semi-structured data
- Cross-sectional
- Time series
- Panel

- Spatial data
- CSV
- Excel
- JSON
- Databases
- Metadata
- FAIR principles
- Data provenance

Running example

- Import FAOSTAT maize production data.

Students inspect

- variables
- units
- countries
- years
- missing values
- metadata

Deliverable

- A documented data dictionary.
-

Week 3 – Data Preparation & Visualization

Topics

Data preparation

- filtering
- selecting variables
- joins
- missing values
- duplicates
- reshaping
- derived variables

Visualization

- distributions
- time series
- comparisons
- scatterplots

Running example

Questions

- Which countries have the largest maize yields?
- How have yields changed over time?
- Are changes driven by production or harvested area?

Deliverable

- A reproducible Quarto/HTML report with visualizations.
-

Week 4 – Data Analysis

Topics

Goals of analysis

- explanation
- prediction

Model workflow

- train/test split
- generalization
- evaluation
- interpretation

Models

- linear regression
- multiple regression

Running example

Example descriptive model

```
lm(log(yield) ~ year + country)
```

Prediction exercise

- Training: 1990–2017
- Testing: 2018+

Students compare

- historical mean
- linear trend
- country model

Deliverable

- Interpret model coefficients and prediction performance.
-

Week 5 – Summary

Students submit a short reproducible report including

- research question
 - data
 - preparation
 - visualizations
 - model
 - interpretation
 - limitations
 - reproducibility statement
-

Running project repository

```
maize-yield-project/  
  README.md  
  data-raw/  
  data-processed/  
  scripts/  
  reports/  
  figures/  
  renv.lock  
  maize-yield-project.Rproj
```

Data sources

Primary

- FAOSTAT

Optional

- World Bank Open Data

Possible variables

- maize production
- harvested area
- yield
- fertilizer use
- agricultural land

0.2) R, RStudio, and Visual Studio Code Guide

Outline

- Outline
 - Food Systems and Data Science Certificate
 - 1. Software Overview
 - 2. Installing R
 - Windows
 - macOS
 - Linux
 - * Ubuntu / Debian
 - * Fedora
 - * Arch Linux
 - 3. Installing RStudio
 - Windows
 - macOS
 - Linux
 - 4. Installing Visual Studio Code
 - Windows
 - macOS
 - Linux
 - 5. Recommended VS Code Extensions
 - 6. Installing Course Packages
 - 7. Using RStudio
 - Running Code
 - Using the Terminal
 - 8. Working with Projects
 - 9. Using Visual Studio Code
 - 10. Troubleshooting
 - Problem 01
 - Problem 02
 - Problem 03
 - 11. Best Practices
 - Summary
-

Food Systems and Data Science Certificate

Scope

This guide explains how to install and use **R**, **RStudio**, and **Visual Studio Code (VS Code)**. Git installation, GitHub, and repository setup are covered in a separate guide.

1. Software Overview

The course primarily uses **R** for programming and data analysis.

We recommend using:

Software	Purpose
R	Programming language for statistics and data science
RStudio	Primary integrated development environment (IDE) for writing and running R code
Visual Studio Code	General-purpose editor for Quarto, Markdown, documentation, and project files

Most analyses will be completed in **RStudio**, while **VS Code** is useful for editing text files, browsing project folders, and working with Quarto documents.

2. Installing R

Always install **R** before **RStudio**.

Download R from:

<https://cran.r-project.org/>

Windows

1. Select **Download R for Windows**.
2. Download the latest installer.
3. Run the installer.
4. Accept the default installation options.

Verify the installation by opening **R** from the Start Menu.

macOS

1. Select **Download R for macOS**.
2. Download the installer appropriate for your version of macOS (Apple Silicon or Intel if applicable).
3. Open the downloaded package.
4. Follow the installation wizard.

Launch R from the Applications folder to verify the installation.

Linux

Most Linux distributions provide R through their package manager.

```
sudo apt update
sudo apt install r-base
```

Ubuntu / Debian

```
sudo dnf install R
```

Fedora

```
sudo pacman -S r
```

Arch Linux Verify:

```
R --version
```

3. Installing RStudio

Download the free **RStudio Desktop** from:

<https://posit.co/download/rstudio-desktop/>

Choose the installer for your operating system.

Windows

Run the installer and follow the default options.

macOS

Open the downloaded `.dmg` file and drag **RStudio** into the Applications folder.

Linux

Install the package provided by Posit or use your distribution's package manager if available. After installation, open RStudio. It should automatically detect your R installation.

4. Installing Visual Studio Code

Download VS Code from:

<https://code.visualstudio.com/>

Windows

Run the installer and accept the default options.

macOS

Download the macOS version and move VS Code into the Applications folder.

Linux

Install using your preferred package manager.

Example (Ubuntu/Debian):

```
sudo snap install code --classic
```

or use the package provided on the VS Code website.

5. Recommended VS Code Extensions

Install the following extensions from the Extensions panel.

Extension	Purpose
Markdown All in One	Markdown editing
GitLens	Git integration
YAML	YAML editing
EditorConfig	Consistent formatting

6. Installing Course Packages

Open **RStudio**.

Install **renv** (only once):

```
install.packages("renv")
```

Restore the project environment:

```
renv::restore()
```

Install additional packages only when instructed by the course.

7. Using RStudio

The RStudio interface consists of four main panes:

- **Source** – edit scripts and Quarto documents
- **Console** – execute R commands
- **Environment/History** – inspect objects
- **Files/Plots/Packages/Help** – manage files and view outputs

Running Code

Run the current line:

- Windows/Linux: **Ctrl + Enter**
- macOS: **Cmd + Enter**

Run an entire script:

Source → Run All

Using the Terminal

The Terminal tab provides access to your operating system shell without leaving RStudio.

8. Working with Projects

Always open the supplied `.Rproj` file.

Working inside an RStudio Project ensures:

- consistent working directories
- reproducible file paths
- easier collaboration

Do not move files outside the project directory.

9. Using Visual Studio Code

VS Code complements RStudio rather than replacing it.

Recommended uses include:

- editing Markdown
- editing Quarto documents
- browsing project files
- reading documentation
- comparing files
- searching across the project

Open the **project folder**, not individual files.

10. Troubleshooting

Problem 01

Problem: RStudio cannot find R

Reinstall R first, then restart RStudio.

Problem 02

Problem: Missing packages

```
renv::restore()
```

Problem 03

Problem: “Package is not available”

Check your internet connection and ensure you are using the latest version of R.

11. Best Practices

- Work inside the supplied RStudio Project.
 - Keep raw data unchanged.
 - Store generated files separately.
 - Use Quarto for reports.
 - Keep scripts organized.
 - Restart R occasionally to ensure your scripts run from a clean session.
 - Write clear comments and meaningful variable names.
-

Summary

For this course:

- **R** is the programming language used for analysis.
- **RStudio** is the primary environment for programming and data analysis.
- **VS Code** is a complementary editor for Markdown, Quarto, and project files.
- **renv** keeps package versions consistent across all students.

1.1) Why use Git and GitHub?

Outline

- Outline
 - Learning objectives
 - A common problem
 - Git and GitHub are different tools
 - Why Git matters in data science
 - A traceable history
 - Safe experimentation
 - Reproducibility
 - Collaboration
 - A simple mental model
 - What makes a useful commit?
 - What Git should not track
 - How this connects to the maize yield project
 - Check your understanding
 - Videos
 - Further reading
-

Learning objectives

After reading this guide, you should be able to:

- explain the difference between Git and GitHub;
 - describe how version control supports reproducible data science;
 - identify useful points at which to commit work; and
 - explain how GitHub supports collaboration.
-

A common problem

Without version control, a project folder can quickly contain files such as:

```
analysis.R
analysis-new.R
analysis-final.R
analysis-final-revised.R
analysis-final-revised-2.R
```

It is difficult to tell which file is authoritative, what changed, or why. Sharing files by email or chat creates additional copies and makes collaborative editing harder still.

Git addresses this problem by recording meaningful snapshots of a project over time. GitHub provides an online home for Git repositories and adds tools for sharing, reviewing, discussing, and integrating work.

Git and GitHub are different tools

Git is a distributed version control system. It runs on your computer and records the history of files in a repository. Most Git operations work without an internet connection.

GitHub is an online platform that hosts Git repositories. It supports collaboration through pull requests, issues, code review, project permissions, and other services.

You can use Git without GitHub, and GitHub can host repositories created with Git. In this course, we use them together.

Why Git matters in data science

A traceable history

A commit records a snapshot of the tracked files together with an author, timestamp, and message. A useful history helps answer:

- What changed?
- Who changed it?
- Why was it changed?
- When did a result or error first appear?

This is especially valuable when a small change in data preparation or model code changes the final result.

Safe experimentation

Git makes it easier to try an idea without losing known working code. Branches can isolate experimental work, while previous commits provide recovery points.

Git is not a replacement for good backups, but it is much safer and clearer than maintaining manually numbered copies of scripts.

Reproducibility

A reproducible analysis needs more than a final report — it also needs the code and documentation that explain how the result was produced, connected to a specific point in the project's history.

For example, a commit can record the exact versions of:

- a data-cleaning script;
- a model specification;
- an environment lockfile such as `renv.lock`; and
- a Quarto report.

Large or sensitive datasets and secrets generally require other storage solutions and should not be committed automatically.

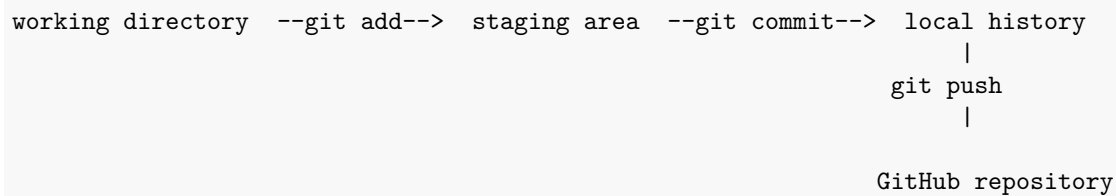
Collaboration

Git allows each collaborator to have a complete local copy of the repository and its history. Contributors can work independently and then integrate their changes.

GitHub adds a shared location where a team can:

- discuss tasks in issues;
- propose changes in pull requests;
- review code and documentation;
- see automated checks; and
- control who can read or modify a repository.

A simple mental model



- The **working directory** contains the files you are editing.
- The **staging area** contains the changes selected for the next commit.
- The **local history** is the sequence of commits stored by Git.
- A **remote repository** is another copy of the repository, commonly hosted on GitHub.

`git push` sends local commits to a remote. `git pull` retrieves and integrates remote changes. Saving a file does not automatically create a commit or upload anything to GitHub.

What makes a useful commit?

A useful commit is a small, coherent change that leaves the project in a reasonable state. Examples include:

- Add maize data preparation script
- Correct yield unit conversion
- Document how to restore the `renv` environment
- Add country-level yield figure

Messages such as `changes`, `stuff`, or `final` do not explain the purpose of a commit.

A practical workflow is:

```
git status
git diff
git add path/to/file
git diff --staged
git commit -m "Describe the completed change"
git push
```

Reviewing `git status`, the unstaged diff, and the staged diff reduces the risk of committing temporary files, data, credentials, or unrelated work.

What Git should not track

Not every project file belongs in version control. A `.gitignore` file commonly excludes:

- downloaded or reproducibly generated data;
- rendered reports and temporary figures;
- package libraries and caches;
- editor-specific state;
- logs; and
- `.env` files or other secrets.

Never commit passwords, access tokens, private SSH keys, or confidential data. Removing a secret in a later commit does not remove it from earlier history.

How this connects to the maize yield project

In this repository:

- analysis steps are separated into numbered scripts;
- `renv.lock` records the R package environment;
- the Quarto source records the report's code and narrative;
- `.gitignore` excludes reproducible or machine-specific artifacts; and
- Git history can connect changes in scripts to changes in results.

Git and GitHub are therefore part of the scientific workflow, not only tools for software developers.

Check your understanding

1. What is the difference between saving a file and committing it?
 2. What is the difference between committing and pushing?
 3. Why might a raw dataset be excluded from Git?
 4. What information should a commit message communicate?
 5. How could Git help identify when an analysis result changed?
-

Videos

The following videos provide an alternative introduction to the concepts in this guide:

- [Git and GitHub for Beginners: What are Git and version control?](#) — freeCodeCamp’s explanation of Git and version control.
- [GitHub for Beginners](#) — GitHub’s official beginner video series covering repositories, collaboration, and common workflows.

The freeCodeCamp video is a longer course; the link opens at the section most relevant here.

Further reading

- [About version control — Pro Git](#)
- [Getting started with Git — GitHub Docs](#)
- [Git cheat sheet](#)

1.2) Version control and collaboration concepts

Learning objectives

After reading this page, you should be able to:

- distinguish Git from GitHub;
 - explain working-directory, staging-area, commit, branch, and remote states;
 - distinguish saving, committing, pushing, and pulling;
 - explain why small, coherent commits support review and recovery; and
 - identify files that should not be placed in version control.
-

Git records project history

Git is a distributed version-control system. A Git repository contains the current project files and a history of recorded project states. Each collaborator normally has a complete local repository, including its history.

Git does not record every save automatically. The researcher decides which changes belong together and records them as a **commit**. A useful commit:

- represents one coherent change;
- contains only intended files;
- leaves the project in an understandable state; and
- has a message that explains the purpose of the change.

The commit history is therefore a sequence of meaningful project states, not a backup of every keystroke.

GitHub supports shared work

GitHub hosts Git repositories and adds collaboration services such as access control, issues, pull requests, reviews, and automation. Git and GitHub are related but not interchangeable:

Git	GitHub
Runs locally	Provides an online service
Records commits and branches	Hosts shared repositories
Works without a network for most operations	Requires network access for synchronization
Manages version history	Adds review and coordination tools

A local repository can exist without GitHub. GitHub cannot replace the local Git concepts needed to understand the history.

Four states of a tracked change

A typical change moves through four states:

working directory	→	staging area	→	local history	→	remote history
edit		git add		git commit		git push

- The **working directory** contains the files currently being edited.
- The **staging area** selects the exact content intended for the next commit.
- The **local history** contains commits already recorded on the computer.
- The **remote history** contains commits shared through a service such as GitHub.

`git status`, `git diff`, and `git diff --staged` make these states visible. Inspecting them is part of the method, not merely troubleshooting.

A minimal example

Consider a single change to `README.md`:

```
git status
# nothing to commit, working tree clean

echo "Document the maize-yield teaching dataset." >> README.md
git status
# Changes not staged for commit: README.md

git add README.md
git status
# Changes to be committed: README.md

git commit -m "Document the maize-yield teaching dataset"
git push origin main
```

Each command narrows the change from an edited file to a permanent, shared entry in the project history. Skipping `git add` before `git commit` records nothing; skipping `git push` after `git commit` keeps the change on the local computer only, invisible to collaborators who only look at GitHub.

Branches and synchronization

A branch is a movable name pointing to a line of commits. Branches allow work to develop without immediately changing another line of work. When histories diverge, Git must integrate them through a merge or rebase.

The central synchronization operations are:

- `git fetch`: obtain remote history without integrating it;
- `git pull`: obtain and integrate remote history; and

- `git push`: publish local commits to a remote repository.

A push can be rejected when the remote branch contains commits that the local branch does not yet contain. This protects shared history from being silently overwritten.

Two common ways to integrate diverging histories are a **merge**, which creates a new commit joining both histories, and a **rebase**, which replays local commits on top of the updated remote history. A merge preserves the exact order of events and is the default and safer choice for beginners; a rebase produces a straighter, easier-to-read history but rewrites commit identifiers. Avoid rebasing commits that have already been pushed and shared with collaborators — rewriting shared history forces everyone else to reconcile their own copy with the rewritten one.

Track deliberately

Version control is appropriate for source code, documentation, configuration, small metadata files, and environment lockfiles. It is usually inappropriate for:

- passwords, tokens, private keys, or other secrets;
- confidential or restricted data;
- large downloaded datasets that can be acquired reproducibly;
- generated caches and package libraries; and
- outputs that can be rebuilt reliably and need not be distributed in Git.

Tracking generated or reproducible artifacts inflates repository size, produces noisy diffs on every rebuild, and creates a false impression that the committed file — rather than the script that produces it — is the authoritative source.

`.gitignore` documents recurring exclusions, but it does not remove a file that has already been committed. Sensitive information requires prevention rather than reliance on later cleanup.

Relationship to the other pages

[Why use version control and collaboration?](#) introduces the problem. The [application page](#) applies these concepts in a collaborative Git workflow. The setup pages cover local tools and repository initialization.

Key message

Version control makes project changes inspectable, recoverable, and shareable. Its value depends on deliberate file selection, coherent commits, and a clear understanding of local and remote history.

1.3) Set up Git and GitHub

Outline

- Outline
 - Learning objectives
 - 1. Install Git
 - Windows
 - macOS
 - Ubuntu or Debian Linux
 - 2. Create a GitHub account
 - 3. Configure your Git identity
 - 4. Check for an existing SSH key
 - 5. Generate an SSH key
 - 6. Add the key to the SSH agent
 - 7. Add the public key to GitHub
 - 8. Test the connection
 - Videos
 - Final checklist
 - Common problems
 - Problem 01
 - Problem 02
 - Problem 03
 - Problem 04
-

Learning objectives

After completing this guide, you should be able to:

- confirm that Git is installed;
- create and secure a GitHub account;
- configure the identity attached to your commits;
- authenticate to GitHub with SSH; and
- test your setup.

The commands in this guide are entered in a terminal. On Windows, use Git Bash unless your instructor specifies another terminal.

1. Install Git

First, check whether Git is already installed using a command prompt on Windows, a terminal on macOS or Linux:

```
git --version
```

If the command prints a version number, continue to the next section.

Windows

Download and run [Git for Windows](#). The default installation choices are suitable for this course. Git for Windows includes Git Bash.

After installation, open **Git Bash** and run:

```
git --version
```

macOS

In Terminal, run:

```
git --version
```

If Git is missing, macOS may offer to install the Command Line Tools. Follow the prompt. Alternatively, use the installer from git-scm.com.

Ubuntu or Debian Linux

```
sudo apt update
sudo apt install git
git --version
```

For another Linux distribution, follow its package manager's instructions or the [official Git installation guide](#).

2. Create a GitHub account

1. Go to github.com.
2. Select **Sign up** and create a free personal account.
3. Choose a professional username that you are comfortable sharing.
4. Use a unique, strong password.
5. Verify your email address.
6. Enable two-factor authentication and store the recovery codes safely.

Your GitHub username will be used in repository addresses later. It does not have to match your computer username.

GitHub's current account instructions are available in [Creating an account on GitHub](#).

3. Configure your Git identity

Git records a name and email address in every commit. Set them globally for your user account on this computer:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use an email address associated with your GitHub account if you want GitHub to link your commits to your profile. If you prefer not to expose your address, GitHub provides a **noreply** commit email under **Settings** → **Emails**.

Set **main** as the initial branch name for new repositories:

```
git config --global init.defaultBranch main
```

Review the configuration:

```
git config --global --list
```

Check that **user.name**, **user.email**, and **init.defaultbranch** have the intended values. Do not copy another student's identity.

For more detail, see [Set up Git — GitHub Docs](#).

4. Check for an existing SSH key

SSH allows Git to authenticate securely to GitHub without entering your GitHub password for every push.

List any existing public keys:

```
ls -al ~/.ssh
```

Look for a matching pair such as:

```
id_ed25519
id_ed25519.pub
```

The file ending in **.pub** is the **public key**. The file without **.pub** is the **private key**. Never share, upload, email, or commit the private key.

If an existing key is already used for GitHub, ask your instructor before replacing it. Otherwise, continue with the next section.

5. Generate an SSH key

Replace the example email with the address associated with your GitHub account:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

When prompted:

1. Press Enter to accept the default file location, unless that would overwrite a key you need.
2. Enter a strong passphrase.
3. Enter the passphrase again.

On an older system that does not support Ed25519, consult GitHub's [SSH key instructions](#).

6. Add the key to the SSH agent

Start the agent:

```
eval "$(ssh-agent -s)"
```

On Linux or Git Bash, add the default Ed25519 key:

```
ssh-add ~/.ssh/id_ed25519
```

On macOS, use:

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Enter the key's passphrase if requested.

7. Add the public key to GitHub

Display the public key:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the complete line beginning with `ssh-ed25519`. Sharing this public key is safe; do not display or copy the file without `.pub`.

On GitHub:

1. Open your profile menu and select **Settings**.
 2. Select **SSH and GPG keys**.
 3. Select **New SSH key**.
 4. Give the key a descriptive title, such as `Personal laptop`.
 5. Keep the key type as **Authentication Key**.
 6. Paste the public key and select **Add SSH key**.
-

8. Test the connection

Run:

```
ssh -T git@github.com
```

On the first connection, SSH may ask whether you trust the host. Compare the displayed fingerprint with GitHub's published fingerprints before entering **yes**.

A successful response includes your GitHub username and explains that GitHub does not provide shell access. The command may still return exit status 1; that is expected because this test authenticates you but does not open an interactive shell.

See [Testing your SSH connection — GitHub Docs](#) for expected output and troubleshooting.

Videos

Use these videos as demonstrations alongside the written steps. Interfaces can change after a video is recorded, so follow the current written instructions when a button or menu looks different.

- [Create a GitHub account](#) — freeCodeCamp course, starting at the GitHub sign-up demonstration.
- [Install Git](#) — the same course, starting at its Git installation section.
- [Create and add an SSH key to GitHub](#) — an official GitHub for Beginners walkthrough.
- [Configure SSH and push to GitHub](#) — freeCodeCamp course, starting at its SSH section.

Pause after each step and compare the result in your terminal with the checkpoints in this guide. Never display your private SSH key while following a recording.

Final checklist

Run these commands:

```
git --version
git config user.name
git config user.email
ssh -T git@github.com
```

You are ready for the repository setup exercise when:

- Git reports a version;
 - the configured name and email are yours; and
 - GitHub's SSH response contains your GitHub username.
-

Common problems

Problem 01

Problem: `git: command not found`

Git is not installed or the terminal has not detected the new installation. Install Git, close the terminal, open it again, and retry.

Problem 02

Problem: Permission denied (publickey)

Confirm that:

- the public key was added to the correct GitHub account;
 - the private key was added to the SSH agent; and
 - `ssh-add -l` lists the expected key.
-

Problem 03

Problem: Commits appear under the wrong identity

Check:

```
git config --show-origin --get-regexp "^user\\."
```

Correct the global name or email with the commands in section 3. Repository- specific configuration can override the global configuration.

Problem 04

Problem: SSH warns that the host key has changed

Do not bypass the warning. Stop and ask your instructor or system administrator to verify the connection.

1.4) Create your maize yield repository

Outline

- Outline
 - Goal
 - Before you start
 - 1. Create an empty repository on GitHub
 - 2. Clone the public course repository
 - 3. Inspect the existing remote
 - 4. Rename the course remote
 - 5. Add your repository as **origin**
 - 6. Push to your repository
 - 7. Verify the setup
 - Videos
 - Your normal workflow
 - Receive later course updates
 - Troubleshooting
 - Problem 01
 - Problem 02
 - Problem 03
 - Problem 04
 - Problem 05
 - Completion checklist
-

Goal

In this exercise, you will:

1. create an empty **maize-yield-project** repository in your GitHub account;
2. clone the public course repository;
3. keep the course repository as a remote named **upstream**;
4. add your repository as the remote named **origin**; and
5. push the project history to your repository.

At the end, the relationship will be:

```
course repository
git@github.com:mmoessler/maize-yield-project.git

    fetch updates

local maize-yield-project

    push your work

your repository
git@github.com:YOUR-USERNAME/maize-yield-project.git
```

Before you start

Complete the [version-control setup page](#) first. Confirm that:

```
git --version
ssh -T git@github.com
```

Replace **YOUR-USERNAME** in this guide with your actual GitHub username. Do not type the angle brackets sometimes used around placeholders.

Choose a parent directory in which to store course projects. Do not run the clone command from inside another copy of **maize-yield-project**.

1. Create an empty repository on GitHub

1. Sign in to [GitHub](#).
2. Select the + menu in the upper-right corner.
3. Select **New repository**.
4. Choose your personal account as the owner.
5. Enter **maize-yield-project** as the repository name.
6. Choose the visibility required by your course. If no visibility was specified, choose **Private** for course-work or **Public** if you intend to share it openly.
7. Do **not** add a README, **.gitignore**, or license.
8. Select **Create repository**.

The repository must be empty because the course project already has files and Git history. Initializing both repositories separately can create unrelated histories and unnecessary merge problems.

Keep the new repository's Quick Setup page open. Its SSH address should have this form:

```
git@github.com:YOUR-USERNAME/maize-yield-project.git
```

GitHub also documents this empty-repository approach in [Adding locally hosted code to GitHub](#).

2. Clone the public course repository

In a terminal, move to the parent directory where you keep projects. For example:

```
cd ~/projects
```

The directory must already exist. You may use a different location.

Clone the course repository:

```
git clone git@github.com:mmoessler/maize-yield-project.git
```

Enter the new project directory:

```
cd maize-yield-project
```

Check its status:

```
git status
```

You should be on a branch named `main` with no uncommitted changes.

3. Inspect the existing remote

A clone automatically calls its source remote `origin`. Confirm this:

```
git remote -v
```

You should initially see the course address for both fetch and push:

```
origin  git@github.com:mmoessler/maize-yield-project.git (fetch)
origin  git@github.com:mmoessler/maize-yield-project.git (push)
```

Do not push student work to the course repository.

4. Rename the course remote

Rename the existing remote from `origin` to `upstream`:

```
git remote rename origin upstream
```

`upstream` is a conventional name for the repository from which your copy originated. It will allow you to fetch later course updates without confusing them with your own repository.

Verify the change:

```
git remote -v
```

The course address should now be listed as `upstream`.

5. Add your repository as `origin`

Replace `YOUR-USERNAME` with your GitHub username:


```
git remote add origin git@github.com:YOUR-USERNAME/maize-yield-project.git
```

Inspect both remotes:

```
git remote -v
```

The result should follow this pattern:

```
origin    git@github.com:YOUR-USERNAME/maize-yield-project.git (fetch)
origin    git@github.com:YOUR-USERNAME/maize-yield-project.git (push)
upstream  git@github.com:mmoessler/maize-yield-project.git (fetch)
upstream  git@github.com:mmoessler/maize-yield-project.git (push)
```

Read each address carefully. **origin** must contain your username; **upstream** must contain **mmoessler**.

6. Push to your repository

Push the existing **main** branch and set it to track **origin/main**:

```
git push -u origin main
```

The **-u** option records the tracking relationship. After this first push, you can normally use **git push** and **git pull** without repeating the remote and branch names.

Refresh your repository page on GitHub. The project files and commit history should now be visible.

7. Verify the setup

Run:

```
git status
git branch -vv
git remote -v
```

Check that:

- the working tree is clean;
- **main** tracks **origin/main**;
- **origin** points to your GitHub account; and
- **upstream** points to the public course repository.

You can inspect one remote in more detail with:

```
git remote show origin
```

Videos

No general video uses the course-specific repository addresses in this guide. Follow the written commands above for the exercise, and use these videos to reinforce the underlying operations:

- [Clone a GitHub repository](#) — freeCodeCamp course, starting at its cloning demonstration.
- [Push a local repository to GitHub using SSH](#) — covers SSH authentication followed by the first push.
- [GitHub for Beginners](#) — GitHub's official series on repositories and GitHub workflows.

The cloning demonstration uses an editor interface, whereas this exercise uses terminal commands. Both approaches perform the same Git operation. The course-specific remote configuration remains:

```
origin    git@github.com:YOUR-USERNAME/maize-yield-project.git
upstream  git@github.com:mmoessler/maize-yield-project.git
```

Your normal workflow

After editing a file:

```
git status
git diff
git add path/to/changed-file
git diff --staged
git commit -m "Describe the completed change"
git push
```

Stage specific files rather than automatically staging everything. Always review changes before committing, especially when the project contains data, environment files, or credentials.

Receive later course updates

Only do this when instructed, particularly if you have changed the same files as the course repository.

Fetch the upstream history:

```
git fetch upstream
```

Inspect the incoming commits:

```
git log --oneline main..upstream/main
```

Integrating upstream changes may require a merge or rebase and may produce conflicts. Your instructor will specify the appropriate method for the course exercise. Fetching alone does not change your working files.

Troubleshooting

Problem 01

Problem: Repository not found

Check the spelling and capitalization of your username and repository name. Also confirm that you are authenticated as the GitHub account that owns the repository.

Problem 02

Problem: Permission denied (publickey)

Return to the [version-control setup page](#) and test:

```
ssh -T git@github.com
```

Problem 03

Problem: remote origin already exists

Inspect the current configuration before changing anything:

```
git remote -v
```

If `origin` already points to your repository, no change is needed. If you missed the rename step and `origin` still points to the course repository, run:

```
git remote rename origin upstream
git remote add origin git@github.com:YOUR-USERNAME/maize-yield-project.git
```

Problem 04

Problem: The destination directory already exists

Do not delete it blindly. It may contain work. Use `pwd`, `ls`, and `git status` to determine what it contains, then ask your instructor if you are unsure.

Problem 05

Problem: The GitHub repository contains an initial README

The safest beginner solution is usually to delete and recreate the new GitHub repository as an empty repository, provided it contains no work you need. Do not force-push unless your instructor explicitly asks you to.

Completion checklist

- My GitHub repository is named `maize-yield-project`.
- I cloned the course repository using SSH.
- `origin` points to my repository.
- `upstream` points to the course repository.
- My local `main` branch tracks `origin/main`.
- The files and commit history appear in my GitHub repository.
- I understand that commits stay local until I push them.

1.5) A collaborative Git workflow

Outline

- Outline
 - Learning objectives
 - The setting
 - The basic cycle
 - Before making a change
 - Scenario 1: Simple commit
 - Starting point
 - Collaborator 1
 - Scenario 2: Non-conflicting diverging snapshots
 - Collaborator 1
 - Collaborator 2
 - Integrate the remote commit
 - What to notice
 - Scenario 3: Conflicting diverging snapshots
 - Collaborator 1
 - Collaborator 2
 - Resolve the conflict
 - If you are not ready to resolve a merge
 - Comparison of the scenarios
 - Habits that reduce problems
 - Videos
 - Scenario 1: Add, commit, and push
 - Scenario 2: Diverging histories and merging
 - Scenario 3: Merge conflicts
 - Practice exercise
 - Command summary
 - Further reading
-

Learning objectives

After completing this guide, you should be able to:

- follow the pull–edit–stage–commit–push workflow;
 - explain why a push can be rejected;
 - integrate non-conflicting work from another collaborator;
 - recognize and resolve a merge conflict; and
 - distinguish local commits from commits on GitHub.
-

The setting

This guide considers two collaborators working on the same branch of the same GitHub repository:

- **Collaborator 1:** you;
- **Collaborator 2:** someone else; and
- **Remote:** the shared repository on GitHub, named `origin`.

The examples use the `main` branch to keep the exercise small. In a larger project, collaborators would usually work on separate branches and integrate them through pull requests.

The basic cycle

```
GitHub          your computer
  git pull
    working directory
      edit
        changed files
          git add
            staging area
              git commit
                local history
                  git push
```

The commands have different purposes:

- `git pull` obtains and integrates remote commits.
- Editing changes files only in your working directory.
- `git add` selects content for the next commit.
- `git commit` creates a snapshot in your local repository.
- `git push` sends local commits to GitHub.

Saving, adding, committing, and pushing are separate actions.

Before making a change

Move into the repository and inspect its state:

```
cd maize-yield-project
git status
git branch --show-current
git remote -v
```

Confirm that:

- you are in the intended repository;
- the working tree does not contain unexpected changes;
- the current branch is `main`; and
- `origin` points to the shared repository.

Then retrieve the latest shared work:

```
git pull --no-rebase origin main
```

`--no-rebase` tells Git to integrate diverging histories with a merge, making the behavior explicit regardless of the student's global Git configuration.

Pulling before you edit reduces the chance of divergence, but it cannot prevent another collaborator from pushing while you are working.

Scenario 1: Simple commit

Only Collaborator 1 changes the repository.

Starting point

Both the local and remote repositories contain commit A:

```
local main:  A
remote main: A
```

Collaborator 1

First, pull the current state:

```
git pull --no-rebase origin main
```

Edit a file. For example, add a sentence to `README.md`. Then inspect the change:

```
git status
git diff
```

Stage only the intended file:

```
git add README.md
```

Review exactly what will be committed:

```
git diff --staged
```

Create the local commit:

```
git commit -m "Clarify the project description"
```

The histories are now:

```
local main:  A B
remote main: A
```

Push the new commit:

```
git push origin main
```

After the push:

```
local main:  A B
remote main: A B
```

Verify the result:

```
git status
git log --oneline --decorate -5
```

`git status` should report that the branch is up to date with `origin/main` and that the working tree is clean.

Scenario 2: Non-conflicting diverging snapshots

Both collaborators pull commit A. Collaborator 1 changes `file-1.md`, while Collaborator 2 changes `file-2.md`.

Because both people start from the same commit, their local histories diverge:

```

      B collaborator 1 changes file-1.md
     /
starting A
     \
      C collaborator 2 changes file-2.md
```

The edits are non-conflicting because they affect different files.

Collaborator 1


```
git pull --no-rebase origin main
# Edit file-1.md
git diff
git add file-1.md
git diff --staged
git commit -m "Update file 1"
git push origin main
```

The push succeeds, so GitHub now contains:

A B

Collaborator 2

Collaborator 2 had already pulled A, edited `file-2.md`, and committed:

```
# Edit file-2.md
git diff
git add file-2.md
git diff --staged
git commit -m "Update file 2"
```

Their local history contains A C, but GitHub contains A B. If Collaborator 2 now runs:

```
git push origin main
```

Git rejects the push because it would discard commit B from the remote branch. The rejection commonly includes the term `non-fast-forward`.

This rejection protects the shared history. It does not mean that Collaborator 2's commit has been lost.

Integrate the remote commit

Collaborator 2 pulls the remote history:

```
git pull --no-rebase origin main
```

Because B and C change different files, Git can normally merge them automatically. Git creates a merge commit M:

```

      B   M
     /   /
    A   C
```

Both changes are now present locally. Collaborator 2 should inspect and, where appropriate, test the combined project:

```
git status
git log --oneline --graph --decorate -5
```

Finally, push the integrated history:

```
git push origin main
```

GitHub now contains B, C, and merge commit M.

What to notice

- Different files can still produce diverging Git histories.
 - A push rejection is expected when another commit reached GitHub first.
 - Non-conflicting divergence can usually be merged automatically.
 - The collaborator who integrates the histories should inspect the combined result before pushing.
-

Scenario 3: Conflicting diverging snapshots

Both collaborators pull commit A, then both change the same part of `file-1.md`.

```

      B  collaborator 1 changes the same lines
      /
starting A
      \
      C  collaborator 2 changes the same lines differently
```

Git cannot decide which content should be retained.

Collaborator 1

```
git pull --no-rebase origin main
# Edit file-1.md
git diff
git add file-1.md
git diff --staged
git commit -m "Revise the introduction"
git push origin main
```

The push succeeds. GitHub now contains A B.

Collaborator 2

Collaborator 2, still working from A, makes and commits a different edit:

```
# Edit the same part of file-1.md
git diff
git add file-1.md
git diff --staged
git commit -m "Rewrite the introduction"
git push origin main
```

The push is rejected because GitHub already contains commit B.

Collaborator 2 retrieves and attempts to merge that commit:

```
git pull --no-rebase origin main
```

Git stops the merge and reports a conflict. `git status` identifies the affected files:

```
git status
```

A conflicted file contains markers similar to:

```
<<<<<< HEAD
The introduction written by Collaborator 2.
=====
The introduction written by Collaborator 1.
>>>>>> origin/main
```

The sections mean:

- <<<<<< HEAD to ===== is the content from the current local branch;
- ===== to >>>>>> origin/main is the incoming remote content.

Do not simply choose one section without understanding both changes; discuss the intended result with the collaborator if needed.

Resolve the conflict

Open `file-1.md` in an editor. Replace the complete marked section with the agreed final content and remove all three marker lines.

For example:

```
The revised introduction combining the relevant ideas from both collaborators.
```

Confirm that no conflict markers remain:

```
git diff
```

Search the repository if necessary:

```
git grep -n -e '<<<<<<' -e '=====' -e '>>>>>>'
```

An empty result is expected after all markers have been removed.

Stage the resolved file and check the state:

```
git add file-1.md
git status
git diff --staged
```

Complete the merge:

```
git commit -m "Merge introduction changes"
```

Inspect or test the combined work, then push:

```
git log --oneline --graph --decorate -5
git push origin main
```

The resulting history has the same shape as the non-conflicting case:

```
      B   M
     /     \
    A       C
```

The difference is that a person had to determine the contents of merge commit M.

If you are not ready to resolve a merge

While a conflicted merge is still in progress, you can return to the state before the pull:

```
git merge --abort
```

This is useful when you need help or want to discuss the changes first. It does not delete the local commit you made before beginning the merge.

Do not use destructive commands such as `git reset --hard` or force-push as a shortcut. They can discard work or overwrite shared history.

Comparison of the scenarios

Scenario	Remote changed while you worked?	Same content changed?	Expected result
Simple commit	No	No	Push succeeds
Non-conflicting divergence	Yes	No	Push rejected; pull merges automatically; push again
Conflicting divergence	Yes	Yes	Push rejected; pull reports conflict; resolve, commit, and push

Habits that reduce problems

1. Pull before beginning a unit of work.
2. Communicate which files or sections each person is changing.
3. Keep changes focused and commits small.
4. Check `git status` frequently.
5. Review `git diff` before staging.
6. Review `git diff --staged` before committing.
7. Write commit messages that describe the completed change.
8. Push completed work regularly.
9. Never commit secrets, private data, or generated files that belong in `.gitignore`.
10. In larger collaborations, use separate branches and pull requests.

Videos

Use these videos to reinforce the workflow after reading the corresponding scenario:

Scenario 1: Add, commit, and push

- [Git and GitHub for Beginners: add, commit, and push](#) — freeCodeCamp, beginning with its demonstration of staging, committing, and pushing changes.
- [GitHub for Beginners](#) — GitHub's official beginner series, including its overview of commonly used Git commands.

Pause before each command and predict which state it changes: working directory, staging area, local history, or remote history.

Scenario 2: Diverging histories and merging

- [Git and GitHub for Poets: branches](#) — The Coding Train’s visual introduction to branches and histories that develop from a shared starting point.
- [GitHub for Beginners: answers to common questions](#) — GitHub’s official discussion includes merging versus rebasing and synchronizing work with an upstream repository.

The branch video demonstrates divergence through separate branches. Scenario 2 creates divergence through separate clones of a shared branch, but the important history shape and the need to integrate both commits are equivalent.

Scenario 3: Merge conflicts

- [GitHub for Beginners](#) — watch the official episode about merging a pull request, which creates a conflict, displays the conflict markers, resolves the file locally, commits the resolution, and pushes it.
- [GitHub for Beginners: answers to common questions](#) — includes a shorter official explanation of why conflicts occur and how a person chooses the final content.

GitHub’s demonstration uses two pull-request branches, whereas Scenario 3 uses two collaborators on `main` — in both cases, Git reports a conflict because different commits changed the same lines. Use the commands and safety checks in this guide for the exercise.

Practice exercise

Practise these cases in a temporary repository before using important project work. Two collaborators can use separate clones, or one learner can create two clones in different directories to simulate two computers.

For each scenario:

1. Record the output of `git log --oneline --graph --all`.
 2. Predict whether the next push will succeed.
 3. Explain why Git can or cannot merge automatically.
 4. Confirm that the final files contain both intended changes.
 5. Confirm that both collaborators can pull the final history.
-

Command summary

```
# Inspect
git status
git diff
git diff --staged
git log --oneline --graph --decorate -5
```

```
# Synchronize before editing
git pull --no-rebase origin main

# Record a change
git add path/to/file
git commit -m "Describe the completed change"

# Share a change
git push origin main

# During a conflict
git status
git add path/to/resolved-file
git commit -m "Merge conflicting changes"
git push origin main

# Cancel an unfinished merge
git merge --abort
```

Further reading

- [Git basics — GitHub Docs](#)
- [About remote repositories — GitHub Docs](#)
- [Dealing with non-fast-forward errors — GitHub Docs](#)
- [Resolving a merge conflict using the command line — GitHub Docs](#)

1.6) A reproducible AI-assisted research workflow

Outline

- Outline
 - Learning objectives
 - Place in the session
 - What reproducibility means here
 - Reproducibility is not automatic replay
 - Git records accepted states, not truth
 - AI assists; researchers remain responsible
 - Human review is an activity, not a button
 - The review–verify–commit loop
 - Prompting for reviewable work
 - Give a concrete outcome
 - Separate diagnosis from modification
 - State invariants
 - Ask for evidence
 - Further detail
-

Learning objectives

After completing this guide, you should be able to:

- explain how Git supports transparent AI-assisted research without making AI output automatically trustworthy;
- break work into bounded, reviewable tasks;
- establish permission, data, and file boundaries before using an agent;
- inspect and verify AI-generated changes before accepting them; and
- distinguish reproducible repository state from exact reproduction of an AI conversation.

For the full step-by-step procedure, checklists, and a worked example, see the [AI-assisted research workflow reference](#).

Place in the session

This page extends the Version Control session:

```
Why Git? → Setup → Repository setup → Collaborative workflow
                                   ↓
                                   Reproducible AI-assisted workflow
```

Review these pages first:

- [Why use Git and GitHub?](#)
- [Create your maize yield repository](#)
- [A collaborative Git workflow](#)

This guide assumes that you can inspect a repository, create a branch, review changes, stage selected files, commit, and push.

The goal is not to recommend one AI service. The workflow applies to coding agents, chat interfaces, editor assistants, and command-line tools. Their capabilities and interfaces differ, but the researcher still has to define, review, verify, and document the work.

What reproducibility means here

An AI-assisted research change is reproducible when another person can recover the accepted project state and understand how the result was produced and checked.

Useful evidence includes:

- the repository commit;
- input-data identity and provenance;
- scripts, configuration, and environment definitions;
- the accepted code and documentation changes;
- commands or procedures used for verification;
- important assumptions and human decisions; and
- relevant information about AI assistance when it materially affected the work.

Reproducibility is not automatic replay

An AI response can depend on:

- model and model version;
- system instructions and available tools;
- conversation context;
- repository state;
- external search or service results;
- random or nondeterministic generation;
- provider-side updates; and
- time-dependent information.

Even with the same prompt, another run may produce a different response. Therefore, the primary reproducible object is normally the **reviewed repository state and its evidence**, not the expectation that an agent will regenerate the same text byte for byte.

Git records accepted states, not truth

Git can show what changed and preserve a reviewed snapshot. It does not prove that:

- code is correct;
- an analysis is scientifically valid;
- data are representative;
- a citation exists;
- a model result is unbiased;
- a licence permits reuse; or
- the researcher understood the change.

Version control supports auditability. Testing, validation, domain knowledge, and human judgment establish whether a change should be accepted.

AI assists; researchers remain responsible

Treat an AI system as a fallible tool that can propose, explain, transform, and inspect material. Do not treat it as an accountable researcher or an authoritative source.

AI output can contain:

- syntactically correct but logically wrong code;
- invented functions, packages, papers, quotations, or URLs;
- hidden changes outside the requested scope;
- inappropriate statistical assumptions;
- insecure commands or credential handling;
- outdated information;
- code copied or closely derived from material with unclear licensing; and
- confident explanations unsupported by evidence.

The human researcher remains responsible for:

- defining the question;
- deciding which data and methods are appropriate;
- controlling access to files, services, and credentials;
- checking facts and references against authoritative sources;
- reviewing every accepted change;
- running and interpreting verification;
- deciding what to commit and publish; and
- reporting methods and limitations honestly.

Human review is an activity, not a button

Selecting “accept,” “apply,” or “approve” in an interface is not sufficient review. Review means reading the diff, understanding the logic, checking the scientific meaning, running suitable verification, and resolving uncertainty before accepting the change.

The review–verify–commit loop

Use this loop for each bounded change:

```
Define one outcome
  ↓
Inspect repository and data boundaries
  ↓
Create or choose a branch
  ↓
Give the agent context, scope, and constraints
  ↓
Review the complete diff
  ↓
Verify software behavior and research meaning
  ↓
Revise or reject if necessary
  ↓
Stage selected files and review again
  ↓
Commit the reviewed outcome
```

An AI interaction does not automatically deserve a commit. A commit should represent one coherent, accepted project outcome. The loop expands into nine concrete steps:

1. **Begin from a known repository state.** Check `git status`, the current branch, remotes, and the starting commit before asking an agent to act.
2. **Define one bounded task**, describing an observable outcome, in-scope and out-of-scope files, and required verification — not a vague instruction such as “improve the project.”
3. **Choose a safe working context**, normally a descriptively named branch. A branch isolates history; it is not a security boundary for an agent’s filesystem or network access.
4. **Give the agent sufficient context and boundaries:** the objective, relevant paths, conventions, invariants, and which actions require confirmation.
5. **Inspect every change yourself** — `git status --short` and `git diff` — including untracked files and history-sensitive files such as lockfiles, `.gitignore`, and credentials.
6. **Verify behavior and research meaning** at the software, data, and scientific level; passing tests do not guarantee a valid method.
7. **Record material decisions** that affect research interpretation, such as why a source or exclusion was chosen, and verify any AI-suggested citation against its authoritative source.
8. **Commit one reviewed outcome**, staging only the intended paths and writing a message that explains the completed change, not that it was AI-assisted.
9. **Integrate and publish deliberately** through the project’s normal review mechanism, confirming checks, documentation, and sensitive-material screening before merging.

See the [reference page](#) for the full detail, command examples, and checklists behind each step, plus guidance on handling data and secrets, research-integrity disclosure, failure recovery, and a worked maize-yield example.

Prompting for reviewable work

Give a concrete outcome

Weak:

Improve the analysis.

Better:

Explain why the model evaluation may leak future information. Do not modify files. Cite the exact code paths involved and propose two fixes.

Better implementation request:

Implement the approved time-based split in `scripts/model-maize-yield.R`. Preserve the existing output schema, add a test for the split boundary, and update the report. Do not change package versions.

Separate diagnosis from modification

When the cause is uncertain:

Inspect and explain the failure first. Do not edit files yet.

After reviewing the diagnosis:

Implement the selected fix and run the targeted checks.

This prevents an early guess from becoming an unreviewed code change.

State invariants

Examples:

- raw data must remain unchanged;
 - public function signatures must remain compatible;
 - the output key must remain unique;
 - the train/test boundary must remain 2017/2018;
 - no new dependency may be added; and
 - no network access is permitted.
-

Ask for evidence

Useful request:

```
After implementation, report changed files, verification commands, results,  
remaining limitations, and any assumptions you could not verify.
```

Then independently inspect that evidence.

Further detail

The [AI-assisted research workflow reference](#) covers each of the nine steps in full, including command examples, permission boundaries, data and secrets handling, research-integrity and disclosure requirements, what cannot be reproduced exactly, failure and recovery patterns, a worked maize-yield example, a practice exercise, completion checklists, and further resources.

1.7) AI-assisted research workflow reference

This page provides the full procedural detail, checklists, and worked example for the [reproducible AI-assisted research workflow](#). Read that page first for the concepts, the condensed review–verify–commit loop, and prompting guidance.

Outline

- Outline
- The evidence to preserve
 - Do not turn Git into a chat archive
 - Suggested AI-use record
- 1. Begin from a known repository state
 - Synchronize deliberately
 - Record the starting point when it matters
- 2. Define one bounded task
 - Task brief template
 - Why bounded tasks improve reproducibility
- 3. Choose a safe working context
 - Branch naming examples
 - Separate exploratory and accepted work
- 4. Give the agent sufficient context and boundaries
 - Permission boundaries
 - Repository instructions
- 5. Inspect every change
 - Review untracked files
 - Review history-sensitive files carefully
 - Ask the agent to explain, then verify independently
- 6. Verify behavior and research meaning
 - Software verification
 - Data verification
 - Scientific verification
 - Record what was not verified
- 7. Record material decisions
 - Avoid unsupported AI-generated citations
- 8. Commit one reviewed outcome
 - Commit content
 - Commit message
 - A commit is not a certificate of correctness
- 9. Integrate and publish deliberately
 - Before merging

- Tag research milestones
 - Handling data, secrets, and external services
 - Classify information before sharing it
 - Never include secrets in prompts or tracked files
 - Review agent tool use
 - Research integrity and disclosure
 - Verify facts and sources
 - Preserve intellectual contribution accurately
 - Follow applicable disclosure policy
 - Keep methods and writing distinct
 - What cannot be reproduced exactly
 - Model and service changes
 - Nondeterminism
 - Hidden context
 - External state
 - Practical response
 - Failure and recovery
 - The agent changed unrelated files
 - The proposed approach is wrong
 - Tests pass but results changed unexpectedly
 - A citation cannot be verified
 - A sensitive value was exposed
 - The session context is lost
 - Application to the maize-yield project
 - Scenario
 - Starting checks
 - Invariants
 - Review
 - Verification
 - Commit
 - Practice exercise
 - Reflection
 - Completion checklist
 - Before AI assistance
 - During AI assistance
 - Before committing
 - Before publishing or merging
 - Check your understanding
 - Further resources
 - Version control
 - Responsible AI and research practice
 - Reproducibility and documentation
-

The evidence to preserve

Not every AI interaction needs to be committed. Preserve information in proportion to its importance for understanding or reproducing the research.

Evidence	Usually preserve?	Suitable location
Accepted source-code change	Yes	Git commit
Test and validation code	Yes	Repository
Environment definitions	Yes	Lockfile, container, or configuration
Data provenance and checksums	Yes	Metadata/provenance records
Research decision affected by AI	Yes	Commit body, issue, decision record, or methods note
Verification commands and results	Yes when material	Test configuration, report, log summary, or commit body
Tool/model/interface and date	When relevant to disclosure or reproduction	Methods note or AI-use record
Full conversation transcript	Sometimes	Approved research record outside Git when needed
Routine brainstorming	Usually no	Temporary notes, if useful
Secrets or confidential prompts	Never in Git	Approved secure system only
Unaccepted AI output	Usually no	Discard or retain only when needed for an audit

Do not turn Git into a chat archive

Committing every prompt and response can:

- expose sensitive data;
- preserve copyrighted or restricted material unnecessarily;
- overwhelm the project history;
- obscure the decisions that actually mattered; and
- create records that cannot be shared with the repository.

For a methodologically important interaction, record a concise, structured summary unless a complete transcript is required by the study protocol, institution, funder, journal, or audit process.

Suggested AI-use record

For a material contribution, a short record might contain:

```
## AI assistance record
```

- ```
- Date: 2026-08-06
- Tool/interface: coding agent in the project workspace
- Task: add validation for the teaching-data candidate key
```



- Repository starting commit: abc1234
- Human-supplied constraints: preserve raw data; modify validation only
- Accepted contribution: proposed duplicate-key check and documentation
- Human verification: reviewed diff; ran validation; inspected duplicate count
- Human decision: accepted check; rewrote interpretation paragraph
- Resulting commit: def5678
- Limitations: exact model version and generation cannot be replayed locally

Do not claim that this record makes the model response deterministic. It makes the accepted research decision easier to audit.

---

## 1. Begin from a known repository state

Move to the repository root and inspect it:

```
pwd
git status
git branch --show-current
git remote -v
git log --oneline --decorate -5
```

Confirm:

- this is the intended repository;
- the current branch is understood;
- existing changes belong to you or are otherwise accounted for;
- remotes point to the expected locations; and
- the starting commit is identifiable.

Do not ask an agent to “clean up everything” in a working tree that already contains unexplained changes. The agent may overwrite, combine, or misclassify someone else’s work.

---

### Synchronize deliberately

If the project workflow requires an up-to-date base and the working tree is clean:

```
git pull --no-rebase origin main
```

Use the actual branch and remote defined by the project. Inspect fetched or merged changes before continuing.

---

### Record the starting point when it matters

```
git rev-parse HEAD
```

The commit ID is useful when an AI task spans several revisions or when a methods record needs to identify the exact input state.

---

## 2. Define one bounded task

A useful task describes an observable outcome.

Weak request:

Improve the project.

More reviewable request:

Add a validation check that stops when the teaching dataset contains a duplicated `area + item + element + year + unit` key. Update the validation report to explain the check. Do not change the raw data or analysis model.

---

### Task brief template

Outcome:  
Files or components in scope:  
Files or components out of scope:  
Starting evidence:  
Scientific assumptions that must not change:  
Required verification:  
Expected documentation:  
Actions requiring approval:

Example:

Outcome: Document and test the fixed teaching-sample checksum.  
In scope: metadata/provenance.yml, scripts/validate-data.R.  
Out of scope: raw data, modeling scripts, package versions.  
Starting evidence: checksum supplied with the reviewed snapshot.  
Scientific assumptions: validation must not modify the input.  
Required verification: matching checksum passes; modified copy fails.  
Expected documentation: explain identity versus data quality.  
Actions requiring approval: network access and dependency installation.

---

## Why bounded tasks improve reproducibility

They make it easier to:

- detect unrelated edits;
  - define verification before implementation;
  - understand a diff;
  - reject an unsuitable approach;
  - create a coherent commit; and
  - explain the decision later.
- 

## 3. Choose a safe working context

Use a branch for a feature, experiment, or uncertain change:

```
git switch -c feature/validate-teaching-data
```

A branch isolates history; it does not create a security boundary. An agent with filesystem or network access may still affect files, services, or data outside the branch.

---

## Branch naming examples

```
feature/add-data-validation
docs/explain-provenance
experiment/compare-yield-models
bugfix/correct-unit-conversion
```

Choose a name that describes the purpose rather than the tool:

|                             |           |
|-----------------------------|-----------|
| feature/add-data-validation | preferred |
| ai-work                     | too vague |

---

## Separate exploratory and accepted work

Exploration can include failed approaches. The accepted branch history should still make clear which change was reviewed and why. Depending on project policy, retain experimental commits, squash them during review, or summarize the exploration in an issue or decision record.

Do not rewrite shared history without coordinating with collaborators.

---

## 4. Give the agent sufficient context and boundaries

An agent needs enough context to follow the project, but it should not receive unnecessary sensitive information or authority.

Provide:

- the research or software objective;
  - relevant repository paths;
  - coding and documentation conventions;
  - data grain, units, and key assumptions;
  - tests or expected behavior;
  - files that must remain unchanged;
  - whether network access is permitted;
  - whether installing dependencies is permitted; and
  - which actions require confirmation.
- 

### Permission boundaries

Distinguish among:

| Action                  | Example                                         | Typical treatment                             |
|-------------------------|-------------------------------------------------|-----------------------------------------------|
| Read-only inspection    | Read files, show status, parse configuration    | Usually low risk within project scope         |
| Reversible project edit | Modify one script on a branch                   | Review diff before acceptance                 |
| Environment mutation    | Install packages, update lockfiles              | Require explicit scope and review             |
| External state change   | Push, open issue, send message, start cloud job | Require clear authorization                   |
| Destructive action      | Delete data, rewrite history, overwrite release | Require explicit approval and verified target |

Do not broaden authority merely because the agent proposes a convenient next step.

---

### Repository instructions

If a project has contributor instructions, style guides, tests, or data policies, point the agent to them. A tool cannot follow a convention it has not seen.

---

## 5. Inspect every change

After the agent works, inspect the repository yourself:

```
git status --short
git diff
```

Check:

- Are only intended files changed?
  - Is every new file expected?
  - Were data, credentials, generated outputs, or lockfiles added accidentally?
  - Does the implementation match the task rather than merely compile?
  - Are units, identifiers, joins, missing values, and boundary cases handled?
  - Were comments and documentation updated?
  - Did the change weaken validation or silently suppress errors?
  - Is the implementation unnecessarily complex?
- 

### Review untracked files

`git diff` does not display the content of untracked files. Use `git status` to find them and open each intended file before staging it.

---

### Review history-sensitive files carefully

Pay particular attention to:

- environment lockfiles;
  - dependency manifests;
  - `.gitignore`;
  - CI/CD configuration;
  - database migrations;
  - data snapshots;
  - submodule pointers;
  - credentials and `.env` files; and
  - generated reports presented as research results.
- 

### Ask the agent to explain, then verify independently

Useful questions include:

- Which assumptions does this implementation make?
- Which files changed and why?
- What failure cases remain?
- Which verification was run?
- What was not tested?

An explanation helps review but is not evidence by itself. Compare it with the diff, commands, and results.

---

## 6. Verify behavior and research meaning

Verification should be proportional to the risk of the change.

---

### Software verification

Possible checks include:

- parse or syntax checks;
  - unit tests;
  - integration tests;
  - schema and configuration validation;
  - a clean render or build;
  - reproducibility from an empty output directory;
  - comparison with a known result; and
  - inspection of logs and warnings.
- 

### Data verification

Check relevant properties such as:

- input identity and provenance;
  - row counts before and after transformation;
  - grain and candidate-key uniqueness;
  - units and conversions;
  - missingness;
  - allowed classifications and flags;
  - unmatched and multiplied join keys;
  - temporal and geographic coverage; and
  - unexpected changes in summaries or distributions.
- 

### Scientific verification

Tests can pass while the method is inappropriate. Ask:

- Does the method answer the stated research question?
  - Are causal, predictive, and descriptive claims distinguished?
  - Are assumptions defensible?
  - Is leakage introduced between training and evaluation data?
  - Are uncertainty and limitations reported?
  - Are output interpretations supported by the data and model?
  - Did AI-generated text overstate the evidence?
-

## Record what was not verified

Examples:

```
Verified: scripts parse; unit tests pass; sample workflow completes.
Not verified: full provider download; Docker build; cross-platform behavior.
```

Stating a verification boundary is more transparent than implying complete confidence.

---

## 7. Record material decisions

Code shows what the computer executes. It may not explain why a source, threshold, model, join, or exclusion was chosen.

Record decisions that affect research interpretation, for example:

- why one provider was chosen;
- why a country or year was excluded;
- why a particular unit conversion is valid;
- why a model is descriptive rather than causal;
- why a warning was accepted;
- why generated code was revised or rejected; and
- what AI assistance contributed to a material method.

Suitable locations include:

- README or implementation documentation;
  - data dictionary or provenance record;
  - analysis report;
  - issue or pull request;
  - architecture or decision record;
  - commit body; and
  - methods or AI-use statement.
- 

## Avoid unsupported AI-generated citations

For every paper, dataset, quotation, version, licence, and URL suggested by an AI system:

1. locate the authoritative source;
2. confirm title, authors, date, identifier, and content;
3. read enough of the source to ensure it supports the claim;
4. cite the source rather than the AI response; and
5. state uncertainty when verification is incomplete.

Do not include a citation merely because it looks plausible.

---

## 8. Commit one reviewed outcome

Stage selected paths rather than every visible change:

```
git add scripts/validate-data.R reports/data-validation.qmd
```

Review the staged snapshot:

```
git diff --staged
git status --short
```

Then commit:

```
git commit -m "Validate the teaching-data candidate key"
```

---

### Commit content

A strong commit contains one coherent outcome and, when applicable:

- implementation;
- tests or validation;
- documentation;
- configuration needed to run it; and
- intentional updates to generated release artifacts.

Avoid mixing an AI-assisted feature with unrelated formatting, dependency updates, personal files, or previous uncommitted work.

---

### Commit message

The title should describe the completed outcome:

```
Validate the teaching-data candidate key
```

Use the body to explain motivation, important decisions, and verification:

```
Validate the teaching-data candidate key
```

- ```
- stop when area, item, element, year, and unit are duplicated
- preserve duplicate evidence rather than deleting rows automatically
- document the check in the validation report
```

```
Verified with the fixed teaching snapshot and a modified duplicate fixture.
```

The message does not need to say “AI generated.” Disclosure belongs in the appropriate research or project record. The commit message should explain the project change.

A commit is not a certificate of correctness

The commit records the accepted state. Preserve test evidence and review processes appropriate to the project's risks.

9. Integrate and publish deliberately

Push the working branch when it is ready to share:

```
git push -u origin feature/validate-teaching-data
```

Use the project's review mechanism, such as a pull or merge request. A reviewer should be able to see:

- the problem and intended outcome;
 - the complete diff;
 - verification evidence;
 - scientific or data assumptions;
 - known limitations; and
 - any material AI assistance required by project policy.
-

Before merging

Confirm:

- automated checks pass;
 - human review is complete;
 - conflicts are resolved intentionally;
 - documentation and environment files agree with the implementation;
 - no sensitive material is present;
 - generated results were recreated from the reviewed code; and
 - the target branch is correct.
-

Tag research milestones

After review and integration, a tag can identify a milestone:

```
git tag -a analysis-v1 -m "Reviewed analysis used for the first report"  
git push origin analysis-v1
```

A useful research release also identifies data, environment, configuration, and outputs; the tag alone cannot recreate ignored external data.

Handling data, secrets, and external services

Classify information before sharing it

Do not send data to an AI service merely because the interface accepts an upload. Determine:

- whether the data contain personal, confidential, proprietary, embargoed, or location-sensitive information;
- which contractual, ethical, institutional, and legal conditions apply;
- whether the service retains or uses submitted content;
- where processing occurs;
- whether the approved account and settings are being used; and
- whether a de-identified, synthetic, aggregate, or local alternative is sufficient.

If the conditions are unclear, do not provide the data.

Never include secrets in prompts or tracked files

Secrets include:

- passwords;
- API tokens;
- private SSH keys;
- cloud credentials;
- database connection strings;
- private endpoints; and
- confidential participant identifiers.

Use approved secret-management mechanisms. If a secret appears in a prompt, log, diff, or commit, stop, revoke or rotate it, and follow the project's incident process. Deleting the current line does not remove copies from Git history or an external service.

Review agent tool use

An agent may be able to:

- read files outside the intended task;
- access the network;
- install or execute software;
- modify Git state;
- call external APIs; or
- send messages and create remote resources.

Grant only the capabilities needed for the bounded task. Review external and destructive actions before authorizing them.

Research integrity and disclosure

Verify facts and sources

AI systems are not bibliographic databases or authoritative documentation. Verify claims with primary sources and record the sources actually consulted.

Preserve intellectual contribution accurately

AI systems do not take responsibility for research. Human authors and contributors remain accountable for the work. Describe human contributions using the study's authorship policy; a contributor-role taxonomy can help record conceptualization, data curation, formal analysis, software, validation, visualization, and writing.

Follow applicable disclosure policy

Institutions, journals, funders, courses, and collaborators may require different forms of AI-use disclosure. Determine the policy before submission.

A useful disclosure can identify:

- tool or service;
- relevant date or version when available;
- purpose of use;
- material affected;
- human review and verification; and
- known reproducibility limitations.

Do not make a generic statement that implies every generated claim was checked if it was not.

Keep methods and writing distinct

Assistance with grammar is different from assistance that changes:

- the research question;
- source selection;
- data exclusions;
- transformation logic;
- statistical methods;
- interpretation; or
- reported conclusions.

The more methodologically important the contribution, the stronger the need for explicit records, verification, and disclosure.

What cannot be reproduced exactly

Model and service changes

A provider may update a model or interface without making the old version available. Tool behavior, safety rules, search results, and context handling may change.

Nondeterminism

The same apparent request can produce different results. A saved prompt does not guarantee replay.

Hidden context

System instructions, retrieved context, account settings, tool outputs, or conversation truncation may not be visible or exportable.

External state

An agent may use websites, package registries, APIs, or databases that later change.

Practical response

Preserve what the project controls:

- accepted code and documentation;
- Git commit and branch;
- input identities and source records;
- environment lockfiles or images;
- configuration and parameters;
- verification code and results;
- concise decision records; and
- tool/model metadata that is actually available and relevant.

State what cannot be reconstructed. Reproducibility is strengthened by honest boundaries, not by claiming exact replay when it is impossible.

Failure and recovery

The agent changed unrelated files

Inspect first:

```
git status --short
git diff
```

Do not discard changes until you know whether they belong to you or another collaborator. Ask the agent to separate or revert only its own changes when the ownership is clear. Avoid destructive repository-wide resets.

The proposed approach is wrong

Reject it. An AI response is a proposal, not sunk cost. Preserve a short decision note only if the failed approach is scientifically or operationally important.

Tests pass but results changed unexpectedly

Stop and compare:

- input checksums and versions;
- row counts and keys;
- environment changes;
- model parameters and random seeds;
- warnings and logs;
- numerical summaries; and
- the exact diff.

Passing tests may not cover the relevant scientific behavior.

A citation cannot be verified

Remove it or replace it with a verified source. Do not invent missing bibliographic details.

A sensitive value was exposed

Stop using the value, rotate or revoke the credential if applicable, notify the responsible person, and follow institutional incident procedures. Do not rely on deleting the local file alone.

The session context is lost

Use the repository as the recovery point:

```
git status
git log --oneline --decorate -10
git show --stat HEAD
```

Read the README, task record, relevant diff, and latest verification evidence. A concise session summary can help, but the committed repository remains the authoritative accepted state.

Application to the maize-yield project

Scenario

Ask an AI coding agent to add one data-management check without changing the raw teaching snapshot.

Suitable task:

Review `metadata/data-dictionary.csv` against the columns in `data-raw/faostat-maize-yield-sample.csv`. Add a validation failure when the input contains an undocumented column or the dictionary describes a column absent from the input. Do not edit either CSV. Update the validation report and explain the limitation of column-name agreement.

Starting checks

```
cd maize-yield-project
git status
git branch --show-current
git log --oneline -5
```

Create a branch:

```
git switch -c feature/validate-dictionary-coverage
```

Invariants

- The fixed teaching CSV must remain byte-identical.
 - The dictionary must not be rewritten automatically.
 - A mismatch must stop or clearly fail validation.
 - Matching names do not prove correct definitions or units.
 - No package dependency should be added unless justified and approved.
-

Review

```
git status --short
git diff
sha256sum data-raw/faostat-maize-yield-sample.csv
```

Confirm that the source checksum still matches the recorded provenance.

Verification

At minimum:

1. parse the changed R and Quarto files;
2. run validation with the fixed snapshot and dictionary;
3. test a temporary dictionary missing one input variable;
4. test a temporary dictionary containing one extra variable;
5. confirm the raw CSV did not change; and
6. inspect the report explanation.

Do not modify tracked source evidence to create a failure fixture. Use a temporary copy outside the tracked raw-data path or a test fixture designed for that purpose.

Commit

```
git add scripts/validate-data.R reports/data-validation.qmd
git diff --staged
git commit -m "Validate dictionary coverage of teaching data"
```

The commit should record the accepted validation behavior. If AI assistance was methodologically material, add the appropriate project or course disclosure.

Practice exercise

Choose one small improvement in your maize-yield repository:

- clarify one README instruction;
- add one validation assertion;
- add one test for a helper function;
- improve one error message; or
- document one analysis assumption.

Complete these steps:

1. Confirm a clean or understood starting state.
 2. Create a descriptive branch.
 3. Write a bounded task brief with invariants and verification.
 4. Ask an AI assistant to inspect or implement the task.
 5. Review every changed and untracked file.
 6. Run suitable software, data, and scientific checks.
 7. Record one material decision and one limitation.
 8. Stage only the intended files.
 9. Review the staged diff.
 10. Commit with an outcome-focused message.
 11. Write a short AI-use record if the assistance materially affected the research or if course policy requires it.
-

Reflection

Answer:

- Which part of the AI output did you reject or revise?
 - Which verification provided the strongest evidence?
 - What remained unverified?
 - Could another learner recover the accepted state without the chat transcript?
 - Did the interaction expose any information that should not have been shared?
-

Completion checklist

Before AI assistance

- ☐ The repository, branch, status, remote, and starting commit are known.
 - ☐ Existing changes are understood and preserved.
 - ☐ The task has one bounded outcome.
 - ☐ In-scope and out-of-scope paths are stated.
 - ☐ Scientific and data invariants are explicit.
 - ☐ Required verification is defined before implementation.
 - ☐ Sensitive data and credential constraints are understood.
 - ☐ External and destructive actions require explicit authorization.
-

During AI assistance

- ☐ The agent receives only the context and permissions needed.
 - ☐ Diagnosis is separated from modification when the cause is uncertain.
 - ☐ Assumptions and unverified claims are made visible.
 - ☐ Authoritative sources are used for facts, APIs, methods, and citations.
 - ☐ No secret or restricted information is placed in prompts or files.
-

Before committing

- ☐ `git status` and the complete unstaged diff were reviewed.
 - ☐ Every new file was opened and inspected.
 - ☐ Unrelated changes are excluded.
 - ☐ Software checks pass at an appropriate level.
 - ☐ Data grain, keys, units, missingness, and coverage remain valid.
 - ☐ Scientific assumptions and interpretations were reviewed by a human.
 - ☐ Facts, references, quotations, and links were verified.
 - ☐ Verification boundaries and remaining limitations are recorded.
 - ☐ The staged diff contains exactly the intended outcome.
 - ☐ The commit message explains the completed change.
-

Before publishing or merging

- ☐ Review and automated checks are complete.
 - ☐ Environment, data provenance, and configuration are reproducible.
 - ☐ No credential, personal data, or restricted material is present.
 - ☐ AI-use disclosure follows course, institutional, funder, and journal policies.
 - ☐ Accepted results can be traced to a repository commit and input state.
 - ☐ Claims do not exceed the verified evidence.
-

Check your understanding

1. Why does saving a prompt not guarantee exact reproduction of an AI response?
 2. What should be the primary reproducible object in an AI-assisted coding workflow?
 3. Why should a commit correspond to a reviewed outcome rather than every AI interaction?
 4. Name three kinds of evidence that Git does not provide by itself.
 5. What should you inspect in addition to `git diff` before staging?
 6. Give an example of a software test passing while the research method remains invalid.
 7. Which information about an AI interaction should be preserved when it materially changes a research method?
 8. Why is a Git branch not a security boundary?
 9. What should you do when an AI system proposes a paper or quotation?
 10. How would you recover after losing the conversation context?
-

Further resources

Version control

- [Pro Git: Recording Changes to the Repository](#) explains the working tree, staging area, and commit cycle.
 - [Pro Git: Branches in a Nutshell](#) explains how branches isolate lines of development.
 - [The Turing Way: Version Control](#) places version control in a reproducible-research workflow.
-

Responsible AI and research practice

- [NIST AI Risk Management Framework](#) provides a voluntary framework for governing, mapping, measuring, and managing AI risks.
 - [NIST Generative AI Profile](#) addresses risks that are specific to or intensified by generative AI.
 - [CRediT contributor roles](#) provide a vocabulary for describing human contributions such as software, validation, data curation, formal analysis, and writing. CRediT does not determine authorship.
-

Reproducibility and documentation

- [The Turing Way: Overview of Reproducible Research](#) connects version control, computational environments, data, and research documentation.
- [Documentation tools for data management](#) explains how Markdown, YAML, CSV, and SHA-256 checksums support structured, reviewable project records.

2.1) Why use reproducible environments?

Learning objectives

After reading this guide, you should be able to:

- explain why code alone is not enough to reproduce an analysis;
 - describe dependencies and dependency versions;
 - distinguish an R project environment from a container; and
 - identify important aspects of reproducibility that environment tools do not solve.
-

The problem: “It works on my machine”

Imagine that one student runs the maize yield analysis successfully, but another student receives an error. Both students have the same scripts. The difference might be:

- a package is not installed;
- different package versions behave differently;
- the computers use different versions of R;
- a package needs a system library that is missing;
- the operating systems handle paths or fonts differently; or
- an external dataset or website has changed.

The scripts describe **what to do**, but an analysis also depends on the environment in which those scripts run.

What is a computational environment?

A computational environment includes the software and configuration required to execute a project: the language and its version, installed packages and their versions, operating-system libraries, external tools such as Quarto and Pandoc, environment variables, and the data and inputs the code reads. If important parts of this environment are unspecified, a collaborator may have to guess how to recreate it.

An R script that begins with `library(dplyr)` states that the project needs `dplyr`, but not which version. Packages change over time: functions can gain arguments, defaults can change, bugs can be fixed, and older behavior can be removed. Installing the newest available package versions months later may therefore produce different output or cause code to fail. Recording versions turns an implicit assumption into explicit project metadata.

Isolation, portability, and reproducibility

A useful environment aims to provide three properties:

- **Isolation:** changing packages for one project does not unexpectedly break another project.

- **Portability:** another person or computer can recreate the environment from shared instructions and metadata.
- **Reproducibility:** the recreated environment uses the intended dependency versions rather than whichever versions happen to be installed.

These properties overlap, but they are not identical. An isolated environment that exists only on one laptop is not yet portable. A portable description that always installs unpinned latest versions is not fully reproducible.

Two complementary tools

Two kinds of tools address these properties at different layers. [renv](#) gives an R project its own package library and records package versions in `renv.lock`, so the R package layer can be restored exactly. Docker packages a broader execution context — an operating system, system libraries, external tools, and the analysis code — into a portable, runnable image. Python and other language ecosystems have their own equivalents to `renv`, such as `venv`, Conda, or Poetry; the goal in every case is the same: keep project dependencies separate from other projects, and provide a machine-readable way to reconstruct them. The [concepts page](#) explains how `renv` and Docker divide this work and how they fit together.

Why this matters in food-systems data science

Food-systems analyses may inform research, policy, monitoring, or operational decisions. They often combine data from different organizations and must be revisited after a semester, reporting cycle, or data revision.

A documented environment helps:

- a classmate reproduce a result;
- an instructor diagnose a problem;
- a research team rerun an analysis after new data arrive;
- reviewers understand the software context;
- an automated service execute the same workflow; and
- future maintainers update dependencies deliberately.

Reproducibility also improves learning: instead of spending class time on unexplained package differences, students can work from a known starting point.

What these tools do not guarantee

`renv` and Docker improve computational reproducibility, but neither guarantees scientific validity or identical results in every situation. You must still manage source-data versions and provenance, random-number seeds, external web services and changing download URLs, secrets and credentials, hardware-specific calculations, locale and platform differences, undocumented manual steps, and statistical assumptions and data quality.

An environment can reproduce an incorrect analysis perfectly. Reproducibility makes work inspectable; it does not replace good methods.

A practical reproducibility record

For a small data science project, record and version source scripts and reports, a README with execution instructions, an R version and `renv.lock`, `.Rprofile` and the `renv/` project files, a Dockerfile and `.dockerignore`, data provenance and acquisition instructions, and checks that reveal whether execution succeeded. Do not commit package libraries, passwords, tokens, or confidential data.

Check your understanding

1. Why can the same script behave differently on two computers?
 2. What does `renv.lock` record, and why is it committed to Git?
 3. What additional layer does Docker control compared with `renv`?
 4. Name two reproducibility concerns that neither `renv` nor Docker solves by itself.
-

Videos

These videos provide complementary explanations of the motivation and tools:

- [renv: Project Environments for R](#) — Kevin Ushey from Posit introduces the “it worked before” problem, project-local libraries, isolation, portability, lockfiles, `snapshot()`, and `restore()`.
- [You should be using renv](#) — E. David Aja at `rstudio::conf` explains how `renv` supports diagnosis, collaboration, and moving R projects between environments.
- [Docker in 100 Seconds](#) — Fireship gives a concise visual explanation of containers, virtual machines, Dockerfiles, images, and running containers.
- [Docker Tutorial for Beginners](#) — TechWorld with Nana provides a longer introduction to the motivation for containers and demonstrates images, containers, commands, Dockerfiles, volumes, and Compose.

The Docker videos use web-application examples rather than an R analysis. Focus on the environment concepts: a Dockerfile describes an image, an image packages the environment, and a container is a running instance of that image.

Further reading

- [Introduction to renv](#)
- [Using renv with Docker](#)
- [Docker: What is an image?](#)
- [The Turing Way: Reproducible environments](#)

2.2) Reproducible-environment concepts

Learning objectives

After reading this page, you should be able to:

- define a computational environment and its main layers;
- explain the **renv** snapshot/restore workflow and what it records;
- distinguish a Docker image from a container and explain what a Dockerfile records;
- explain the complementary roles of **renv** and Docker;
- distinguish recording an environment from reproducing an analysis; and
- identify limitations that environment tools cannot remove.

What is a computational environment?

A computational result depends on more than source code. Relevant conditions can include:

```
project code and configuration
R packages and package versions
R and publishing-tool versions
system libraries and operating system
hardware and external services
```

Differences at any layer can change whether code runs or what it produces. A reproducible environment records enough of these conditions to recreate the intended execution context.

Environment isolation keeps one project's dependencies from unexpectedly changing another project. Portability makes the recorded context usable on a different computer. Neither property guarantees that the scientific method is valid.

Every language ecosystem provides its own tools for the package layer: R projects typically use **renv**, Python projects often use **venv**, Conda, or Poetry, and other languages have comparable managers. The mechanism differs, but the goal is the same — keep a project's dependencies separate from other projects and describe them in a machine-readable form. A language-level environment manager generally does not fully specify the host operating system, system libraries, drivers, or external command-line programs; that is the layer Docker addresses.

renv manages the R package layer

renv creates a project-specific R package library and records package versions and sources in **renv.lock**. The core workflow moves between the project library and the lockfile:

```
project code    dependency discovery

               project library

    snapshot          restore

               renv.lock
```

Two operations form the central workflow:

- `renv::snapshot()` records the packages used by the project; and
- `renv::restore()` installs the state described by the lockfile.

`renv::status()` compares the code, library, and lockfile and reports where they disagree. The lockfile belongs in version control. The project library normally does not: it is large and machine-specific, so it is reconstructed from the lockfile rather than shared directly. Lockfile changes should be reviewed because they alter the environment that collaborators and automated systems will restore.

`renv` does not record the complete operating system, every system library, hardware, credentials, or remote services.

Docker records a broader execution context

A Docker **image** is a packaged, layered description containing files, binaries, libraries, and configuration; a **container** is an isolated process started from an image:

```
Dockerfile  docker build  image  docker run  container
```

A Dockerfile is a recipe for building that image. It can record:

- a base operating-system image;
- system packages and command-line tools;
- R and other language runtimes;
- project files and dependency installation, for example restoring an `renv` library; and
- the default command and working directory.

Rebuilding an image and starting a new container are different actions: the image is a versioned description, and a container is one run of it. Bind mounts can connect selected host directories to a container, allowing data or results to persist after the container stops. Containers are lighter than full virtual machines because they share the host kernel, but they still provide a controlled user-space environment. Docker improves portability, but tags such as `latest`, changing external repositories, hardware differences, and network services can still introduce variation.

`renv` and Docker are complementary

Concern	<code>renv</code>	Docker
Project-specific R packages	Primary responsibility	Can install from the lockfile
R package sources and versions	Recorded in <code>renv.lock</code>	Inherits the restored package state
Operating-system packages	Not managed	Recorded in the Dockerfile
Runtime isolation	R package library	Process and filesystem context
Typical artifact	<code>renv.lock</code>	<code>Dockerfile</code> and image

Using both tools avoids asking either one to describe a layer it was not designed to manage. The lockfile and Dockerfile should be versioned together with the analysis code: in the maize yield project, `renv.lock` records R 4.3.3 and the R package dependencies, the `Dockerfile` starts from `rocker/verse:4.3.3` and restores those packages during the build, and the resulting image runs `scripts/run-all.R`. Git stores both recipes; `renv` and Docker reconstruct the environments they describe.

Reproducible environment versus reproducible analysis

Recreating an environment is necessary for many analyses but is not sufficient. The result also depends on:

- available and correctly interpreted data;
- deterministic or recorded random processes;
- complete workflow instructions;
- accessible external services;
- appropriate analytical assumptions; and
- responsible interpretation.

An environment can reproduce a coding error or a scientifically invalid analysis exactly. Reproducibility makes work inspectable; it does not certify correctness.

Continue with the applications

Use [the `renv` application](#) to restore and verify the project-local R environment. Then use [the Docker application](#) to build and run the broader execution environment.

Key message

Record dependencies at the layer where they arise: `renv` describes the R package state, while Docker describes a broader system and execution context.

2.3) Set up and use `renv`

Outline

- Outline
 - Learning objectives
 - What `renv` manages
 - This repository is already initialized
 - 1. Install R
 - 2. Open the project correctly
 - 3. Restore the recorded environment
 - 4. Verify the environment
 - The everyday `renv` workflow
 - Use the existing environment
 - Add a package
 - Record the new environment
 - Receive an environment update
 - `snapshot()` and `restore()` point in opposite directions
 - Updating packages
 - Initialize `renv` in a different project
 - How this project uses `renv`
 - Troubleshooting
 - Problem 01
 - Problem 02
 - Problem 03
 - Problem 04
 - Problem 05
 - Completion checklist
 - Videos
 - Further reading
-

Learning objectives

After completing this guide, you should be able to:

- explain the files that make an `renv` project portable;
 - restore the maize yield project's R package library;
 - check whether the library and lockfile are synchronized;
 - add or update a package deliberately; and
 - share an environment change safely through Git.
-

What `renv` manages

`renv` creates a private R package library for each project. This allows two projects on the same computer to use different package versions.

The principal files are:

Path	Purpose	Commit to Git?
<code>.Rprofile</code>	Activates renv when R starts in the project	Yes
<code>renv.lock</code>	Records R and package versions and package sources	Yes
<code>renv/activate.R</code>	Bootstraps and activates renv	Yes
<code>renv/settings.json</code>	Stores project-level renv settings	Yes
<code>renv/library/</code>	Contains installed packages for this machine	No

The project library is excluded because it is large and platform-specific. Another student recreates it from the tracked metadata.

This repository is already initialized

Do **not** run `renv::init()` for the maize yield repository. It already contains the required metadata and a lockfile pinned to R 4.3.3.

For a fresh clone, your task is to **restore** that recorded environment:

```
tracked renv.lock  renv::restore()  local project library
```

`renv::init()` is discussed later for use in a new project.

1. Install R

Install R 4.3.3 if the course environment requires an exact match. Check the active version:

```
R --version
```

From inside R, check:

```
R.version.string
```

renv records the R version but does not install or switch R itself. A different R version may still restore successfully, but it no longer recreates the environment as closely and package binaries may differ.

2. Open the project correctly

Start from the repository root—the directory containing `renv.lock`:

```
cd maize-yield-project
pwd
```

You can then:

- open `maize-yield-project.Rproj` in RStudio;
- start R by running R; or
- execute an R script with `Rscript`.

When R starts in this directory, `.Rprofile` sources `renv/activate.R`. On a fresh machine, the activation script can bootstrap the version of `renv` required by the project.

Check that `renv` is active:

```
renv::project()
.libPaths()
```

The first library path should refer to this project's `renv/library` directory.

3. Restore the recorded environment

The repository provides a setup script:

```
Rscript scripts/00-setup.R
```

The script:

1. confirms that `renv.lock` exists;
2. installs `renv` if it is unavailable;
3. runs `renv::restore()` without an interactive prompt; and
4. checks `renv::status()`.

The first restore downloads and installs many packages and can take time. It requires internet access and may require compilers or system libraries if binary packages are unavailable.

Alternatively, from an R session in the project:

```
renv::restore()
renv::status()
```

Read the proposed package actions before accepting an interactive restore.

4. Verify the environment

Run:

```
renv::status()
```

A synchronized project has agreement among:

- packages used by the project;
- packages installed in the project library; and
- packages recorded in `renv.lock`.

You can inspect selected versions:

```
R.version.string  
packageVersion("renv")  
packageVersion("dplyr")
```

Do not compare only the package count. Exact versions and sources matter.

The everyday `renv` workflow

Use the existing environment

Normally, start R in the project and work as usual:

```
library(dplyr)  
library(ggplot2)
```

Because `renv` is active, R loads packages from the project library.

Add a package

Install into the active project library:

```
renv::install("package-name")
```

You can also use `install.packages("package-name")` while `renv` is active. `renv::install()` makes the project context explicit.

Use the package in project code and verify that the analysis still works.

Record the new environment

Check what changed:

```
renv::status()
renv::snapshot()
renv::status()
```

`renv::snapshot()` updates `renv.lock` to describe the packages used by the project. Review the proposed changes rather than accepting a surprising lockfile update.

Outside R, inspect and commit the metadata:

```
git status
git diff -- renv.lock
git add renv.lock
git commit -m "Record package-name dependency"
git push
```

If `renv` intentionally changed another tracked metadata file, review and stage that file too.

Receive an environment update

After pulling a collaborator's changed lockfile:

```
git pull --no-rebase origin main
Rscript -e 'renv::restore(prompt = FALSE)'
Rscript -e 'renv::status()'
```

Restoring changes the local package library. It should not rewrite the lockfile.

`snapshot()` and `restore()` point in opposite directions

This distinction is essential:

project library	snapshot	renv.lock
project library	restore	renv.lock

- Use `snapshot()` after intentionally changing packages and testing the code.
- Use `restore()` after cloning the project or receiving a changed lockfile.

Running `snapshot()` when you meant `restore()` can record unintended local versions. Always inspect `renv::status()` first.

Updating packages

Package updates should be deliberate:

```
renv::update("package-name")
```

Then:

1. run the complete analysis and relevant tests;
2. inspect warnings and outputs;
3. run `renv::snapshot()`;
4. review the `renv.lock` diff; and
5. commit the code and lockfile changes together when appropriate.

Do not update every package immediately before a deadline merely because newer versions exist. A lockfile is useful precisely because updates become explicit project changes.

Initialize `renv` in a different project

For a new R project that does not already use `renv`:

```
install.packages("renv", repos = "https://cloud.r-project.org")
renv::init()
```

After initialization:

```
renv::status()
renv::snapshot()
```

Commit `.Rprofile`, `renv.lock`, `renv/activate.R`, and `renv/settings.json`. Do not commit `renv/library/`.

This initialization procedure is **not** required for the maize yield project.

How this project uses `renv`

The project has an implicit snapshot setting. `renv` discovers dependencies in the project files and records the packages required by those dependencies.

The Docker build also restores from the same lockfile:

```
COPY renv.lock renv.lock
COPY .Rprofile .Rprofile
COPY renv/activate.R renv/activate.R
RUN Rscript -e 'renv::restore(prompt = FALSE)'
```

This allows local execution and container execution to share the same R package specification.

Troubleshooting

Problem 01

Problem: The project library is not synchronized

Start with:

```
renv::status()
```

Read whether packages are missing, installed at different versions, or used in code but absent from the lockfile. Do not choose `snapshot()` or `restore()` until you understand which state is intended.

Problem 02

Problem: A package fails to compile

The host may lack a compiler or system library. Read the complete installation message. On Linux, `renv::sysreqs()` can help report system requirements:

```
renv::sysreqs()
```

Ask the instructor before installing system packages on a managed computer. Docker may provide a more consistent alternative.

Problem 03

Problem: R is loading packages from the wrong library

Confirm that R was started from the project root:

```
getwd()
renv::project()
.libPaths()
```

Close R, navigate to the repository root, and start a new session.

Problem 04

Problem: Restore cannot download a package

Check internet access, proxy settings, and the package source recorded in `renv.lock`. A lockfile records where a package should come from, but it cannot guarantee that the source remains available forever.

Problem 05

Problem: You see unexpected lockfile changes

Do not commit them automatically. Inspect:

```
git diff -- renv.lock
```

If the changes are not intentional, use `renv::restore()` to return the project library to the recorded state. Ask for help before discarding work.

Completion checklist

- I started R from the repository root.
 - `.libPaths()` shows a project library.
 - `renv::status()` reports a synchronized environment.
 - I can explain the difference between `snapshot()` and `restore()`.
 - I know which `renv` files belong in Git.
 - I do not commit `renv/library/`.
-

Videos

- [renv: Project Environments for R](#) — a 20-minute Posit introduction by `renv` maintainer Kevin Ushey. It covers project-local libraries, the lockfile, the global cache, `snapshot()`, and `restore()`.
- [You should be using renv](#) — a practical Posit conference talk by E. David Aja about diagnosing environments, collaborating, and moving projects between computers.

Both presentations initialize demonstration projects. Remember that the maize yield repository is already initialized: use `renv::restore()` to recreate its recorded library rather than running `renv::init()` again.

While watching, identify the direction of each operation:

```
project library  snapshot  renv.lock
project library  restore   renv.lock
```

Further reading

- [renv workflow](#)
- [Introduction to renv](#)
- [renv function reference](#)
- [Using renv with Docker](#)

2.4) Apply `renv` to the example project

Learning objectives

After completing this application, you should be able to:

- restore the R package environment from `renv.lock`;
 - inspect whether the library and lockfile agree;
 - run project code using the restored library;
 - interpret common `renv::restore()` and `renv::status()` messages; and
 - explain when a lockfile should be updated.
-

Before you begin

Work from the root of the standalone example repository. Confirm the location and inspect its state:

```
pwd
git status
R --version
```

Do not begin by changing the lockfile. First reproduce the environment that the project already records. Restoring before editing anything establishes a known baseline: if something fails later, you can tell whether the cause is your change or an environment that was never correctly restored in the first place.

Inspect the environment record

The repository contains:

```
renv.lock
renv/
.Rprofile
```

`renv.lock` is the machine-readable package record: for every recorded package it lists a name, version, and source (for example CRAN or GitHub). The `renv/` directory contains project infrastructure such as the local package cache and the bootstrap script, while `.Rprofile` activates `renv` automatically whenever R starts in the project directory — this is why opening the project in RStudio or starting R from its root already uses the project library rather than your personal one.

Inspect the lockfile without editing it:

```
git log --oneline -- renv.lock
git status --short renv.lock
```

Restore the package library

Start R from the repository root and run:

```
renv::restore()
```

`renv::restore()` reads `renv.lock`, compares it with the packages already present in the project library, and installs, removes, or leaves packages unchanged so that the library matches the lockfile exactly. The first restore may download packages and therefore requires network access. Review prompts before accepting them. When transfer must be minimized, preserve the local `renv` cache and avoid deleting a working project library merely to demonstrate restoration — a warm cache lets `renv` reuse already-downloaded package builds instead of fetching them again.

Check the result:

```
renv::status()  
sessionInfo()
```

`renv::status()` explains whether the project library, lockfile, and declared dependencies agree. A message such as “No issues found” means the three are consistent. If it instead reports packages that are recorded in the lockfile but missing from the library, run `renv::restore()` again; if it reports packages that are used in the code but not recorded in the lockfile, do not snapshot them yet — first decide, as below, whether adding that dependency is intentional.

A typical clean report looks like:

```
No issues found -- the project is in a consistent state.
```

A report that needs attention instead names specific packages, for example:

```
The following package(s) are recorded in the lockfile but are not  
installed:
```

```
  dplyr [1.1.4]
```

```
Use `renv::restore()` to install the packages recorded in the lockfile.
```

Read the message before acting on it. “Recorded in the lockfile but not installed” calls for `renv::restore()`; “used in the project but not recorded in the lockfile” calls for a decision about whether to add that dependency, not an automatic snapshot.

Where restored packages live

`renv` does not download a fresh copy of every package for every project. Installed packages are stored once in a shared, version-specific cache outside the project (by default under a per-user cache directory), and the project library links to that cache. Restoring a package that another `renv` project on the same machine already restored at the same version is therefore usually fast, because `renv` reuses the cached build instead of reinstalling it. Deleting a project’s `renv/library` directory does not delete the shared cache, and clearing the shared cache does not remove anything from `renv.lock`: the lockfile is the durable record, while the cache and the project library are both disposable and can be rebuilt from it.

Troubleshooting a failed restore

A restore can fail for reasons unrelated to the project code itself:

- **Missing system libraries.** Some R packages depend on system-level libraries when compiling from source. The error message usually names the missing library; install it with the operating system's package manager rather than editing the lockfile.
- **Network or mirror problems.** A transient failure reaching a package repository can often be resolved by retrying `renv::restore()`.
- **R version mismatch.** `renv.lock` records the R version the project was built with. A materially different local R version can make some packages fail to install; report the mismatch rather than silently switching to a different lockfile.

Investigate the actual error before changing the lockfile — a failed restore is a signal to fix the environment, not a reason to regenerate the recorded package versions.

Run the project in the restored environment

Exit R and run a lightweight project command, or execute the documented pipeline when its data requirements are available:

```
Rscript scripts/run-all.R
```

The command should use the project library activated by **renv**. Record the command, input state, and any external-data requirements when reporting the result.

Change dependencies deliberately

`renv::snapshot()` updates `renv.lock` to match the current project library, so it should only run after a change you intend to keep. When the project genuinely requires a new or updated package:

1. create a focused Git branch or commit context;
2. install the intended package;
3. run the relevant checks or analysis;
4. inspect `renv::status()`;
5. call `renv::snapshot()` only when the new state is intentional; and
6. review the `renv.lock` diff before committing it.

```
renv::snapshot()
```

Do not snapshot unrelated packages simply because they happen to be installed — `renv::status()` will list them, and a snapshot records everything currently in the library, not only the package you meant to add.

Verification checklist

- ☐ R starts with the project `renv` environment active.
 - ☐ `renv::status()` reports the expected state.
 - ☐ The project command uses the restored package library.
 - ☐ No unintended lockfile changes were created.
 - ☐ Any intentional lockfile change was reviewed and tested.
-

Handoff to Docker

The restored package layer does not describe every system dependency. Continue with [the Docker application](#) to reproduce the broader execution context.

2.5) Install and set up Docker

Outline

- Outline
 - Learning objectives
 - Docker concepts
 - Image
 - Container
 - Dockerfile
 - Bind mount
 - Docker Compose
 - Choose an installation method
 - Windows installation
 - macOS installation
 - Linux installation
 - Verify the complete setup
 - Check versions
 - Check the client–engine connection
 - Run a test container
 - Inspect Docker state
 - Verify Docker Compose
 - Understand the maize project image
 - Resource and security considerations
 - Troubleshooting
 - Problem 01
 - Problem 02
 - Problem 03
 - Problem 04
 - Problem 05
 - Problem 06
 - Completion checklist
 - Videos
 - Concepts and first commands
 - Docker Compose
 - Further reading
-

Learning objectives

After completing this guide, you should be able to:

- distinguish an image, container, Dockerfile, bind mount, and Compose file;
 - install Docker and Docker Compose for your operating system;
 - verify that the Docker engine is running;
 - run and remove a test container; and
 - identify common installation and permission problems.
-

Docker concepts

Image

An **image** is an immutable, layered package containing the files, binaries, libraries, and configuration needed to run an application.

Container

A **container** is an isolated process created from an image. Starting two containers from one image creates two separate runtime instances.

Dockerfile

A **Dockerfile** is a text recipe for building an image:

```
Dockerfile  docker build  image  docker run  container
```

Bind mount

A **bind mount** exposes a host path inside a container. It is useful when inputs or generated outputs must remain on the host after the container exits.

Docker Compose

Docker Compose describes one or more related containers and their settings in a YAML file, normally `compose.yaml`. It can record images, build instructions, commands, environment variables, networks, and mounts.

Modern Compose uses:

```
docker compose
```

The older hyphenated `docker-compose` standalone program is considered legacy.

Choose an installation method

For Windows and macOS, use Docker Desktop. It includes Docker Engine, the Docker command-line interface, Docker Build, and Docker Compose.

For Linux, follow one of these paths:

- Docker Desktop, if appropriate for the course computer; or
- Docker Engine plus the Docker Compose plugin from Docker's official package repository.

Do not install unofficial packages or copy installation commands from an old blog post. Docker's repository and requirements change over time.

Docker Desktop has licensing terms for commercial use in larger organizations. Students should review the current terms if installing it on an employer-owned computer.

Windows installation

Docker Desktop normally uses the WSL 2 backend for Linux containers.

1. Confirm that the computer meets Docker's current Windows and virtualization requirements.
2. Install or update WSL 2 if required.
3. Download Docker Desktop from the official Docker website.
4. Run the installer and use the WSL 2 backend.
5. Start Docker Desktop and wait until the engine reports that it is running.
6. Use Linux containers for this R project.

Follow the current [Docker Desktop for Windows instructions](#).

Open PowerShell or another terminal and verify:

```
docker version
docker compose version
```

If the client is installed but the server section is missing, Docker Desktop is probably not running.

macOS installation

1. Determine whether the Mac uses Apple silicon or an Intel processor.
2. Download the matching Docker Desktop installer.
3. Move Docker to Applications as instructed.
4. Start Docker Desktop and complete its initial configuration.
5. Wait until the Docker engine is running.

Follow the current [Docker Desktop for Mac instructions](#).

In Terminal, verify:

```
docker version
docker compose version
```

Linux installation

Installation commands differ by distribution. Use Docker's official page for your distribution from the [Docker Engine installation overview](#).

For Ubuntu, follow the official [Install Docker Engine on Ubuntu](#) guide. The maintained repository installation includes packages such as Docker Engine, the CLI, Buildx, and the Compose plugin.

Start and verify the service according to the distribution's instructions:

```
sudo systemctl status docker
docker version
docker compose version
```

Some installations require `sudo` for Docker commands. Adding a user to the `docker` group allows root-level control through the Docker daemon; treat that membership as a security-sensitive administrative decision. Follow Docker's [Linux post-installation guidance](#) and the policy of the course computer.

If Docker Engine is already installed but Compose is missing, install the [Docker Compose plugin](#) from Docker's repository.

Verify the complete setup

Check versions

```
docker --version
docker compose version
```

Both commands should print versions.

Check the client–engine connection

```
docker version
```

This should show both **Client** and **Server** information.

Run a test container


```
docker run --rm hello-world
```

Docker downloads the image if necessary, starts a container, prints a success message, and removes the stopped container because of `--rm`.

The first download requires internet access.

Inspect Docker state

```
docker image ls
docker container ls
docker container ls --all
```

The `hello-world` image may remain cached. The test container should not remain because it was run with `--rm`.

Verify Docker Compose

This repository currently has a `Dockerfile` but no `compose.yaml`. The maize analysis therefore uses `docker build` and `docker run`; Compose is installed for later workflows that coordinate services or record longer run configurations.

To see the Compose help:

```
docker compose --help
```

A minimal Compose file has this general structure:

```
services:
  analysis:
    build: .
    command: ["Rscript", "scripts/run-all.R"]
```

Do not add this example to the maize repository as part of the installation exercise. A real Compose configuration must also decide how data and output directories are mounted.

When a project provides `compose.yaml`, common commands are:

```
docker compose config
docker compose build
docker compose run --rm analysis
docker compose up
docker compose down
```

- `config` validates and renders the configuration.
- `build` creates images for services with a `build` definition.

- `run --rm` performs a one-off service command.
- `up` creates and starts the defined services.
- `down` stops and removes resources created by `up`.

Do not run `docker compose down --volumes` unless you understand that it also removes named volumes and their data.

Understand the maize project image

The repository's Dockerfile begins with:

```
FROM rocker/verse:4.3.3
```

The versioned Rocker image provides R 4.3.3 and publishing tools. The Dockerfile then:

1. selects `/work` as the working directory;
2. copies the `renv` lockfile and activation files;
3. restores the R packages;
4. copies the project files;
5. checks `renv` status; and
6. defines `Rscript scripts/run-all.R` as the default command.

The `.dockerignore` prevents Git state, editor files, and the host's machine-specific `renv` library from entering the build context.

Resource and security considerations

Docker images can use substantial disk space. Inspect usage with:

```
docker system df
```

Containers are isolated processes, not an absolute security boundary:

- use trusted base images;
- review Dockerfiles before building them;
- do not mount sensitive host directories unnecessarily;
- do not pass secrets in image layers or commit them to Git; and
- do not expose the Docker daemon to untrusted users.

Avoid broad cleanup commands during coursework. They may delete images, containers, caches, networks, or volumes needed by other projects.

Troubleshooting

Problem 01

Problem: `docker: command not found`

Docker is not installed or its CLI directory is not on `PATH`. Complete the platform installation, open a new terminal, and retry.

Problem 02

Problem: Cannot connect to the Docker daemon

On Windows or macOS, start Docker Desktop. On Linux, check the Docker service and whether your user has the required permission.

Problem 03

Problem: Virtualization or WSL error

On Windows, confirm that hardware virtualization and WSL 2 meet Docker's current requirements. Institution-managed machines may require administrator support.

Problem 04

Problem `docker compose` is not recognized

Docker Compose is missing or outdated. Docker Desktop includes it. On Linux, install the Compose plugin from Docker's official repository.

Problem 05

Problem: A bind mount is empty or inaccessible

Check that the host path exists and that Docker Desktop is permitted to share it. On Linux, inspect ownership and permissions. A bind mount hides files that were already present at the target path inside the container.

Problem 06

Problem: Build runs out of disk space

Inspect usage with `docker system df`. Ask the instructor before deleting shared images, caches, or volumes.

Completion checklist

- `docker version` shows a client and server.
 - `docker compose version` prints a version.
 - `docker run --rm hello-world` succeeds.
 - I can explain the difference between an image and a container.
 - I know that this repository currently does not contain a Compose file.
 - I understand that mounted host directories are writable from the container.
-

Videos

Concepts and first commands

- [Docker in 100 Seconds](#) — a short visual overview of containers, images, Dockerfiles, and how Docker differs from a virtual machine.
- [Docker Tutorial for Beginners](#) — TechWorld with Nana’s complete beginner course, including installation, basic commands, debugging, Dockerfiles, volumes, and containerized development.

Use Docker’s current written installation instructions in this guide even when a video’s installer screens differ. Operating-system requirements and Docker Desktop interfaces change more quickly than the underlying concepts.

Docker Compose

- [Ultimate Docker Compose Tutorial](#) — TechWorld with Nana explains why Compose is useful, translates `docker run` settings into YAML, and demonstrates `up`, `down`, services, variables, and multi-container applications.

The Compose demonstration uses multiple application services. This maize repository currently has no `compose.yaml`; watch it to learn the tool, but use the project’s documented `docker build` and `docker run` commands until a Compose configuration is provided.

Further reading

- [Get started with Docker](#)
- [Docker Desktop](#)
- [Install Docker Compose](#)
- [Docker CLI cheat sheet](#)

2.6) Apply Docker to the example project

Learning objectives

After completing this application, you should be able to:

- explain what the project Dockerfile records;
 - build the example-project image and interpret common build failures;
 - distinguish image contents from bind-mounted host files;
 - run the pipeline in a container; and
 - decide when an image must be rebuilt.
-

Before you begin

Complete the [Docker setup page](#) and work from the example repository root.

```
docker --version
git status
```

Building may download a base image and packages. Reuse an existing image when the Dockerfile, lockfile, and files copied into the image have not changed.

Inspect the build recipe

Read the Dockerfile before building it:

```
sed -n '1,240p' Dockerfile
```

Identify:

- the base image;
- installed system dependencies;
- the working directory;
- files copied into the image;
- how the R package environment is restored; and
- the default command.

The Docker build context determines which project files can become part of the image. `.dockerignore` should exclude unnecessary data, results, caches, and secrets — anything listed there is never sent to the Docker build process at all, which keeps the image smaller and prevents accidentally baking local data or credentials into a shareable image.

Build the image

```
docker build -t maize-yield-project .
```

Docker reuses cached build layers when their instructions and inputs have not changed, and reruns a layer — along with every layer after it — as soon as one of its inputs changes. This is why a well-ordered Dockerfile installs rarely-changing system dependencies before it copies frequently-edited analysis code: editing a script should not force system packages to reinstall. Rebuild the image after changing the Dockerfile, `renv.lock`, or source files copied during the build. A change only in a bind-mounted host directory does not necessarily require rebuilding.

Inspect the image:

```
docker image ls maize-yield-project
```

When a build fails

A failed build is usually one of a few kinds of problem:

- **A layer instruction fails**, for example a system package name that does not exist for the base image's distribution. The build output identifies the failing instruction; fix the Dockerfile rather than working around the failure inside a running container.
- **`renv::restore()` fails inside the build**. The same causes as a local restore failure apply (missing system library, network problem, R version mismatch); the fix belongs in the Dockerfile or the lockfile, not in a container patched after the fact.
- **The build context is too large or includes unwanted files**. Check `.dockerignore` first.

Run the pipeline

Persist data and outputs by mounting explicit host directories:

```
docker run --rm -it \  
-v "$(pwd)/data:/work/data" \  
-v "$(pwd)/results:/work/results" \  
-v "$(pwd)/figures:/work/figures" \  
-v "$(pwd)/reports:/work/reports" \  
maize-yield-project bash
```

Inside the container, confirm the context and execute the documented workflow:

```
pwd  
Rscript scripts/run-all.R
```

The image supplies the code and software environment; anything COPY-ed into it during the build is fixed at build time. The `-v` mounts instead connect selected host directories to paths inside the running container, so data can be read and results can be written back to the host. Changes made elsewhere inside a container — anywhere not covered by a mount — are discarded when a `--rm` container exits.

Choosing what to mount

Mount only the host directories the workflow actually needs to read or write; avoid bind-mounting the whole repository over the image's working directory, which would silently replace the code baked into the image with whatever happens to be on the host and defeat the point of building a versioned image in the first place. The example mounts above cover **data**, **results**, **figures**, and **reports** because those are the directories the pipeline reads from or writes to — the code itself is expected to come from the image, not from a mount.

Stopping and reusing containers

`--rm` removes the container as soon as it exits, which is appropriate for a one-off pipeline run. Omitting `--rm` keeps the container around after it stops, which can be useful while debugging: `docker ps -a` lists stopped containers, `docker start -ai <container>` resumes one, and `docker rm <container>` removes it once it is no longer needed. A stopped container still occupies disk space until it is removed.

Validate the run

After leaving the container, inspect the mounted output directories on the host:

```
git status --short
find results figures reports -maxdepth 2 -type f | sort
```

Confirm that:

- expected outputs exist;
 - output ownership and permissions are usable on the host;
 - no secrets or unintended data entered the image; and
 - the command and image-building inputs are documented.
-

Verification checklist

- ☐ The Dockerfile and `.dockerignore` were inspected.
 - ☐ The image builds successfully.
 - ☐ The container starts in the expected working directory.
 - ☐ The project pipeline runs using the image environment.
 - ☐ Required outputs persist through explicit mounts.
 - ☐ You can explain whether the next source change requires rebuilding.
-

Key message

An image is a versioned execution environment; a container is one run of that image. Rebuild when image inputs change, and use deliberate mounts for data and outputs that must persist.

3.1) Why learn SSH and the Linux command line?

Learning objectives

After reading this guide, you should be able to:

- explain why command-line skills are useful in data science;
 - distinguish a terminal, a shell, and Linux;
 - explain what SSH provides;
 - distinguish SSH access to GitHub from an SSH session on a remote server; and
 - identify basic security practices for remote work.
-

From a laptop to other computers

An analysis may begin on a laptop but later run somewhere else:

- in a Docker container;
- on a university server or computing cluster;
- on a cloud virtual machine; or
- as an automated task without a graphical desktop.

These systems are commonly controlled through text commands. A small Linux vocabulary keeps the workflow usable across systems, while SSH reaches a remote system securely over a network.

Linux, terminal, and shell are related but different

The terms are often used interchangeably, but they describe different things:

- **Linux** is an operating-system kernel and, informally, a family of operating systems built around it.
- A **terminal** is the window or application through which you enter text commands.
- A **shell** is the program that reads and runs those commands. Bash is a common shell and is the one used in this course's container examples.
- The **command line** is the text-based way of interacting with the shell.

A macOS terminal also provides a Unix-like command line; on Windows, students can use Git Bash or Windows Subsystem for Linux, depending on the course setup.

Why the command line matters in data science

The same instructions can be repeated

A graphical action is hard to describe precisely: “click this button, select that file.” A command records the operation directly:


```
Rscript scripts/run-all.R
```

Commands can be copied into documentation, reviewed, and repeated — though reproducibility also depends on documented inputs and software environment.

Small tools can be combined

Linux commands each do one focused task; pipes connect them:

```
find scripts -type f | sort
```

Here `find` lists files and `sort` orders them — composability that is useful for inspecting files, filtering logs, and automating routine work.

Remote systems often have no desktop

Servers and containers are commonly smaller and easier to automate without a graphical interface. The command line becomes the shared interface between your laptop, a container, and a remote system.

It helps you see the computing context

Commands such as `pwd`, `whoami`, and `hostname` answer three important questions:

```
Where am I?    Which user am I?    Which computer am I using?
```

These questions prevent many mistakes when several terminals, containers, or remote servers are open at the same time.

What SSH provides

SSH, **Secure Shell**, is a protocol and tool family for encrypted communication between computers. It authenticates with a password or cryptographic key, opens or runs a command-line session on a remote server, transfers files with `scp` or `sftp`, and carries Git traffic between a local repository and GitHub.

Encryption protects data in transit; authentication establishes which account is connecting; authorization still determines what that account is allowed to do.

Two SSH uses in this course

GitHub repository access

An SSH Git remote can look like:

```
git@github.com:mmoessler/maize-yield-project.git
```

Git uses SSH to authenticate and exchange repository data; GitHub does **not** give a general-purpose interactive shell. The account name here is normally the literal `git` — GitHub identifies you from the registered public key.

Remote computing

A server login commonly looks like:

```
ssh student@login.example.edu
```

After authentication, this opens a shell on another computer; commands entered there run using its files, software, memory, and CPU. The real hostname and username must come from the instructor or system administrator.

Public-key authentication

An SSH key pair contains a **private key**, which stays secret on your computer, and a **public key**, which may be registered with GitHub or placed on a server.

The private key proves your identity. Never send it to another person, commit it to Git, or copy it into a course container; protect it with a strong passphrase when practical.

On the first connection, SSH may ask whether you trust a host key, which identifies the server, not you. Verify its fingerprint through an official, independent source before accepting it. If SSH later warns that a host key changed, stop and contact the administrator rather than bypassing the warning.

Command-line care

Text commands are exact and can change many files quickly. Before pressing Enter:

- confirm whether the prompt is local, inside a container, or on a remote server;
- use `pwd`, `hostname`, and `git status` to establish context;
- read unfamiliar commands and options instead of pasting them blindly;
- treat `sudo`, recursive deletion, and output redirection with particular care;
- do not expose passwords, tokens, private keys, or confidential data; and
- log out with `exit` when remote work is complete.

Linux expertise is not memorizing commands — it is understanding paths, permissions, input/output, and knowing how to consult `--help`, manual pages, and trustworthy documentation.

How this connects to the maize yield project

The project combines these ideas:

```
local computer
  Git over SSH           GitHub repository
  Docker                 Linux container at /work
  SSH login (when provided)  remote Linux server
```

The Git repository records the project, the Docker image supplies a controlled Linux and R environment, and SSH moves Git data to GitHub or reaches a larger remote computer. The same core command-line habits apply in each setting.

Check your understanding

1. What is the difference between a terminal and a shell?
 2. Why are text commands useful for reproducible work?
 3. How does SSH use differ between GitHub and a remote computing server?
 4. Which part of an SSH key pair may be shared?
 5. What should you do before accepting an unfamiliar host fingerprint?
-

Videos

- [Bash/Terminal Crash Course for Beginners](#) — Traversy Media; navigation, files, pipes, and common shell commands.
- [SSH Crash Course](#) — Traversy Media; SSH keys, remote connections, and related tools.
- [SSH key authentication for GitHub](#) — GitHub; configuring an SSH key for GitHub.

Use the application guides in this repository for commands and safety boundaries specific to the maize yield project.

Further reading

- [The Linux command line for beginners](#) — Ubuntu
- [The Unix Shell](#) — Software Carpentry
- [Secure Shell introduction](#) — Ubuntu
- [About SSH](#) — GitHub Docs

3.2) Remote-computing concepts

Learning objectives

After reading this page, you should be able to:

- distinguish Linux, a terminal, a shell, and SSH;
- explain local and remote computing contexts;
- distinguish authentication from authorization;
- reason about paths, processes, pipes, and permissions; and
- identify safe practices for remote work.

Computing contexts

The same project may be used on several computers:

```
learner laptop → local container → remote server → shared infrastructure
```

Each context can have a different hostname, user account, filesystem, software environment, and resource policy. Before running a consequential command, ask:

```
Which computer? Which user? Which directory?
```

The commands `hostname`, `whoami`, and `pwd` answer these questions.

Context	Reached over a network	Shares the laptop's filesystem	Shares the laptop's software
Local shell	No	Yes	Yes
Local container	No	Only bind-mounted paths	No — its own image
Remote server (SSH)	Yes	No	No — the server's own environment

A command that works in one context can fail, silently use different data, or run against a different software version in another. Confirming the context before acting is part of the method, not an optional courtesy.

Related terms

- **Linux** is an operating-system kernel and commonly refers to operating systems built around it.
- A **terminal** is an interface for entering and viewing text commands.
- A **shell** interprets commands; Bash is a common shell.
- The **command line** is the text-based interaction with a shell.
- **SSH** is a protocol and tool family for encrypted communication with remote services.

- A **session** is one running shell together with the connection or terminal it is attached to. A remote session ends when that connection closes, even if the underlying command is still conceptually “your work.”

Opening a terminal does not necessarily create a remote session. Running a shell in a local container also differs from running a shell on a remote server.

SSH connections

An SSH connection normally identifies:

- a client computer;
- a remote host and network address;
- a remote user account;
- an authentication method; and
- the service and host key being trusted.

Authentication establishes who is connecting. Authorization determines what that account may do after connecting. A successful SSH login does not imply permission to read every file or use every shared resource.

Public-key authentication uses a private key retained by the user and a public key registered with the service. Private keys must not be shared or committed. The server host key protects against connecting silently to the wrong host and should be verified when first encountered or unexpectedly changed.

Filesystems and paths

An absolute path begins at `/`; a relative path begins at the shell’s current working directory. The same textual path can identify different files on a laptop, in a container, and on a server.

Useful inspection commands include:

```
pwd
ls -la
find . -maxdepth 2 -type f | sort
```

File permissions specify what the owner, group, and other users may read, write, or execute. On shared infrastructure, project directories and sensitive data should follow the institution’s access policy rather than permissive defaults.

Processes, streams, and pipes

A process is a running program. Shell commands communicate through standard input, standard output, and standard error. A pipe sends one command’s standard output to the next command:

```
find scripts -type f | sort
```

Redirection and pipes are powerful because they can overwrite files or pass unexpected data onward. Inspect a command and its current directory before running it, especially with elevated permissions or recursive operations.

Long-running remote work should use the infrastructure’s recommended job scheduler or session-management tool. An ordinary SSH connection ending may also end processes attached to that session.

Sessions and disconnection

A foreground process started in an SSH session is normally a child of that session’s shell. When the network connection closes — deliberately or because of a dropped network, a closed laptop lid, or a timeout — the shell and its foreground children are commonly terminated as well. This is not a bug in SSH; it follows from how the process is attached to the session that launched it.

Two consequences follow. First, a long analysis should run under a mechanism that deliberately detaches it from the login session, such as a job scheduler or a persistent terminal multiplexer, rather than in the foreground of an ordinary connection. Second, “the job is still running” is a claim that should be verified on the remote system itself — for example by listing processes or checking the scheduler’s queue — rather than inferred from the fact that the laptop is still open.

Data transfer and execution are separate

SSH can support remote shells and secure file-transfer tools, but transferring code does not reproduce its environment. A remote analysis still needs:

- versioned project files;
- recorded software dependencies;
- available and permitted data;
- appropriate compute resources; and
- documented execution commands.

Use Git for versioned source collaboration and approved transfer tools for data that do not belong in Git.

Continue with the applications

Use [the SSH application](#) to establish and inspect a remote connection. Use [the Linux application](#) to navigate, inspect, and combine command-line tools safely.

Key message

Remote computing becomes manageable when the computer, identity, directory, permissions, and software environment are made explicit before commands run.

3.3) Use SSH with GitHub and remote servers

Learning objectives

After completing this guide, you should be able to:

- inspect an SSH setup without exposing private keys, and test GitHub authentication;
- confirm which Git remote a repository uses, and how GitHub SSH differs from a server login;
- open and close an SSH session on an authorized remote server; and
- recognize common SSH security warnings and connection errors.

Two connections with different outcomes

This guide uses SSH in two ways:

Git operations	SSH	github.com	repository data, no general shell
ssh command	SSH	remote server	interactive remote shell

Both use encrypted connections and may share the same local key pair, but they connect to different services and grant different capabilities.

Part 1: Review SSH access to GitHub

The complete initial key setup is in the [version-control setup page](#). The following steps review and test that setup.

1. Inspect names, not private-key contents

```
ls -al ~/.ssh
```

Common files:

id_ed25519	private key - never share or display its contents
id_ed25519.pub	public key - this is the key registered with GitHub
known_hosts	identities of hosts accepted previously
config	optional per-host client settings

A student may have differently named keys; do not create a replacement merely because the names differ. If an SSH agent is in use, `ssh-add -l` lists the keys it holds — **The agent has no identities** means no key is loaded there, not that no key exists on disk.

2. Test GitHub authentication

```
ssh -T git@github.com
```

On the first connection, SSH may show a host fingerprint; compare it with [GitHub's published SSH key fingerprints](#) before accepting it.

A successful test identifies your GitHub username and explains that GitHub does not provide shell access. The test can still return exit status 1 — for this special GitHub command, the authentication message is the result that matters. The literal username in the command is `git`, not your GitHub username; GitHub determines your account from the public key used during authentication.

3. Inspect and test this repository's Git remote

From the repository root:

```
git remote -v
git remote get-url origin
git ls-remote origin
```

An SSH URL has the form `git@github.com:OWNER/REPOSITORY.git` — for the public maize yield repository, `git@github.com:mnoessler/maize-yield-project.git`. If the repository setup exercise changed `origin` to your own GitHub repository, the owner should instead be your GitHub username. Remote names are local labels, not guarantees; read the URLs before pulling or pushing.

`git ls-remote` reads the remote references without creating a commit or push. Authentication to GitHub can succeed even when your account lacks permission to write to a particular repository.

Part 2: Connect to a remote computing server

Only connect to systems for which you have authorization. Obtain the server hostname, username, port (if not 22), VPN requirement, authentication method, and official host-key fingerprint from the instructor or administrator; the examples below use placeholders.

1. Open the connection

```
ssh USER@HOST
```

For example, a course might provide a command resembling `ssh student@login.example.edu`. If the administrator specifies another port:


```
ssh -p PORT USER@HOST
```

On the first connection, verify the displayed host fingerprint through a trusted channel before typing **yes** — the accepted key is recorded in `~/.ssh/known_hosts`. Enter a password only at the SSH password or key-passphrase prompt; the terminal normally displays no characters while you type it.

2. Confirm where you are

After login:

```
hostname
whoami
pwd
date
uptime
df -h
```

These identify the remote computer, account, directory, time, load, and storage. Do not assume the remote server has the same files, R version, or packages as your laptop or the Docker image. Follow the server's policies for storage locations, software modules, data, and jobs — on a shared cluster, substantial work commonly belongs in a batch scheduler rather than on the login node.

3. Run a command and end the session

SSH can run one command without an interactive session (`ssh USER@HOST hostname`, output in the local terminal). Leave an interactive session with **exit** or **Ctrl-D**, and confirm the prompt has returned to the local computer.

Optional: alias and file transfer

An entry in `~/.ssh/config` can store non-secret connection details for reuse:

```
Host course-server
  HostName HOST
  User USER
  Port PORT
  IdentityFile ~/.ssh/id_ed25519
```

Omit **Port** for port 22, use the real key path from setup, connect with `ssh course-server`, and protect the files with `chmod 700 ~/.ssh` and `chmod 600 ~/.ssh/config`. Never put a password or private-key contents in the configuration file.

`scp` copies files over SSH — confirm source and destination carefully:

```
scp LOCAL_FILE USER@HOST:~/
scp USER@HOST:~/REMOTE_FILE .
```

The colon separates the remote host from its path; without it, `scp` may treat both arguments as local. For large datasets, use the institution’s recommended transfer service instead of assuming `scp` is suitable.

Keys, jobs, and disconnection

Keep personal credentials on the host computer; do not copy a private key into the maize yield image or commit one to Git — cloning and version control belong on the host.

A foreground process may stop when its SSH session disconnects, so do not assume a closed laptop leaves an analysis running. Use the mechanism approved for the server — a job scheduler such as Slurm, a persistent terminal tool such as `tmux` if allowed, or a managed service — and avoid long or resource-heavy work on a shared login node unless its policy explicitly permits it.

Troubleshooting

Problem	Response
Permission denied (publickey)	Run <code>ssh-add -l</code> and <code>ssh -v USER@HOST</code> for diagnostics, then confirm username and key registration with the administrator. For GitHub, see the SSH troubleshooting guide .
Could not resolve hostname	Check for a typo and confirm whether a VPN is required.
Connection timed out or refused	Server, port, network, VPN, or firewall may be unavailable — repeated key creation will not fix it.
REMOTE HOST IDENTIFICATION HAS CHANGED	Stop. Contact the administrator and verify the new fingerprint before changing <code>known_hosts</code> .
GitHub authentication works but <code>git push</code> fails	Authentication and authorization are separate — check <code>git remote -v</code> and confirm write access.

Check your work

- I can explain why `ssh -T git@github.com` does not open a shell.
- I can identify the owner and repository in an SSH Git remote.
- I know which SSH key file must remain private, and verify a new host fingerprint through an official source.
- I use `hostname`, `whoami`, and `pwd` after a remote login, and know long work may need a scheduler or persistent session tool.

Videos

- [SSH key authentication for GitHub](#) — GitHub; generating, registering, and testing a key.
 - [SSH Crash Course](#) — Traversy Media; remote login, keys, configuration, and file transfer.
-

Further reading

- [Connecting to GitHub with SSH](#) — GitHub Docs
- [OpenSSH server documentation](#) — Ubuntu
- [ssh manual page](#) — OpenBSD

3.4) Use Linux inside the maize yield container

Learning objectives

After completing this guide, you should be able to:

- start and enter the project's Docker container, and tell the host from the container;
 - navigate, inspect, and combine project files with basic Linux commands and pipes;
 - understand which container changes persist; and
 - make and save a small practice edit with Vim.
-

Before you begin

Open a terminal in the repository root, check the context, and build the image if needed (or rebuild after changing the Dockerfile or `renv.lock`):

```
pwd
git status
docker version
docker build -t maize-yield-project .
```

The image uses Linux and R 4.3.3 from `rocker/verse`. Its working directory is `/work`, and its default command runs `scripts/run-all.R`.

Method 1: Create and enter a container directly

Prepare the host directories that will receive data and results, then create a container and enter its Bash shell:

```
mkdir -p data-raw data-processed figures reports

docker run --rm -it \
  -v "$(pwd)/data-raw:/work/data-raw" \
  -v "$(pwd)/data-processed:/work/data-processed" \
  -v "$(pwd)/figures:/work/figures" \
  -v "$(pwd)/reports:/work/reports" \
  maize-yield-project \
  bash
```

`--rm` removes the container after it stops, `-it` allocates an interactive terminal, `-v host:container` binds-mounts a host directory, and `bash` replaces the default analysis command with an interactive shell.

Establish your context

Run these commands inside the container:

```
pwd
whoami
hostname
cat /etc/os-release
R --version
```

`pwd` should report `/work`; `hostname` shows the container's generated identifier. Compare with the same commands on the host, and when in doubt ask: which prompt, computer, directory, user?

Navigate through the project

List and move between directories, then find files without changing them:

```
ls -la
cd scripts && pwd && ls
cd .. && cd reports && cd -
find . -maxdepth 2 -type f
find scripts -type f -name "*.R"
```

`/work/scripts` is an **absolute path**; `scripts` is a **relative path** starting from the current directory. `.` is the current directory, `..` the parent, and `cd -` returns to the previous one. Linux paths are case-sensitive: `README.md` and `readme.md` are different names.

Inspect text and data, and combine commands

```
cat .Rprofile                                # display a short file
head -n 10 scripts/run-all.R                 # first lines
tail -n 10 scripts/run-all.R                 # last lines
less README.md                                # scrollable viewer - /word to search, q to quit
wc -l scripts/*.R                             # count lines
grep -R -n "renv" docs                         # search text
head -n 5 data-processed/maize-yield-panel.csv # peek at processed data
```

A pipe, `|`, sends the output of the command on its left to the input of the command on its right — useful for combining the tools above, for example `find scripts -type f | sort` or `grep -R -n "maize" scripts | head`. This does not create a file; redirection (`>` overwrites, `>>` appends) does. Do not practise redirection on tracked project files — an accidental `>` can replace their contents.

Inspect storage and processes

```
df -h          # filesystem capacity
du -sh /work   # project directory size
ps             # processes in the current shell
```

Many commands describe their options with `--help` (e.g. `grep --help`); manual pages may not be installed in a minimal container.

Run project operations

Check the restored R environment, inspect the pipeline before running it, and run the complete analysis only when ready for the FAOSTAT download with the output directories mounted:

```
Rscript -e 'renv::status()'
less scripts/run-all.R
Rscript scripts/run-all.R
```

The first stage downloads a large FAOSTAT archive. See the [renv application](#) and [Docker application](#) for the local and container workflows.

A brief Vim exercise

Vim is a modal command-line text editor. Check availability with `vim --version` (try `vi --version` otherwise); do not install software in the container for this exercise. Use a disposable file, `vim /tmp/vim-practice.txt`, so the exercise cannot alter the repository. Vim starts in **Normal mode**: press `i` for **Insert mode**, type `Maize yield analysis`, press `Esc`, then type `:w` and Enter to save and `:q` and Enter to quit.

Essential commands:

Keys	Action
<code>i</code>	enter Insert mode
<code>Esc</code>	return to Normal mode
<code>:w</code>	save
<code>:q</code>	quit when no unsaved changes remain
<code>:wq</code>	save and quit
<code>:q!</code>	quit and discard unsaved changes
<code>u</code>	undo
<code>dd</code>	delete the current line
<code>/word</code>	search for <code>word</code>
<code>n</code>	move to the next search result

`cat /tmp/vim-practice.txt` reads it back; it disappears with this `--rm` container.

Persistence: image, container, and mounts

The project source was copied into the image at build time; edits to that copy exist only in the container and disappear with `--rm`. The bind-mounted directories (`data-raw`, `data-processed`, `figures`, `reports`) differ — changes there affect the host immediately, since a container is isolation, not a backup. Edit source on the host, record it with Git, and rebuild the image.

Method 2: Enter an already running container

`docker exec` opens another process in an existing container — start one named in the background, then enter it:

```
docker run --rm -dit \
  --name maize-shell \
  -v "$(pwd)/data-raw:/work/data-raw" \
  -v "$(pwd)/data-processed:/work/data-processed" \
  -v "$(pwd)/figures:/work/figures" \
  -v "$(pwd)/reports:/work/reports" \
  maize-yield-project \
  bash

docker ps
docker exec -it maize-shell bash
```

Type `exit` to leave the added shell — the container keeps running. Stop it from the host with `docker stop maize-shell`; because it was created with `--rm`, Docker removes it after it stops. (To leave the *direct* container from Method 1, run `exit` at its shell prompt — this stops and removes it, but not the image or the mounted host files.)

Troubleshooting

Problem	Response
<code>docker: command not found</code>	Install Docker per the Docker setup page , then open a new terminal.
Unable to find image 'maize-yield-project:latest'	Run <code>docker build -t maize-yield-project .</code> from the repository root.
A bind mount is empty or unexpected	Run <code>pwd</code> on the host before <code>docker run</code> — <code>\$(pwd)</code> must be the repository root.
<code>vim: command not found</code>	Use <code>vi</code> if available, or do the exercise elsewhere; this does not block the analysis.
Permission errors in a mounted directory	Stop before using <code>sudo</code> . Record the command, path, and <code>ls -l</code> output, and ask the instructor.

Check your work

- I can identify whether a prompt is on the host or in a container.
 - I can navigate, inspect files, and combine commands with pipes.
 - I know which project directories are bind-mounted, and can save a disposable Vim file, leave, and stop the container cleanly.
-

Videos

- [Bash/Terminal Crash Course for Beginners](#) — Traversy Media; shell, files, navigation, search, and piping.
 - [Vim in 100 Seconds](#) — Fireship; a concise introduction to Vim's modal editing.
-

Further reading

- [The Linux command line for beginners](#) — Ubuntu
- [The Unix Shell lesson](#) — Software Carpentry
- [docker container exec reference](#) — Docker Docs

4.1) Why manage data?

Outline

- Outline
 - Learning objectives
 - The session learning path
 - Reproducible does not automatically mean correct
 - Data are not self-explanatory
 - The data lifecycle
 - What data management contributes
 - Meaning
 - Origin and history
 - Quality
 - Organization
 - Responsibility
 - Why this matters in food-systems research
 - What data management cannot guarantee
 - How this connects to the maize-yield project
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- explain why reproducible code is not sufficient for trustworthy research;
 - describe the stages of a simple data lifecycle;
 - distinguish data management from data preparation and analysis;
 - identify common risks in food-systems datasets; and
 - explain which project artifacts make data understandable and auditable.
-

The session learning path

The Data Management session follows three connected parts:

Motivation	Concepts	Application
Why manage data? →	How do we understand, describe, and assess a dataset? →	How do we document and validate the maize data?

This page provides the **Motivation**. It establishes the problem and the purpose of data management. The [Concepts page](#) develops the vocabulary and mental models. The [Application page](#) uses them to inspect, understand, document, organize, validate, and report on the supplied maize dataset.

The goal is to explain why each project artifact is needed and which risk it addresses, not merely to complete a sequence of commands.

Reproducible does not automatically mean correct

Imagine that the maize-yield workflow is completely reproducible:

- Git records every change to the scripts;
- `renv` restores the required R packages;
- Docker supplies the operating-system environment;
- the same command produces the same report.

The result can still be wrong or misleading if:

- a yield value is interpreted using the wrong unit;
- a blank or provider-specific missing code is read as zero;
- production and harvested-area records are confused;
- a country label refers to a different geographic unit than expected;
- an estimated value is treated as a direct observation; or
- the provider revises historical data after the first download.

Reproducibility answers, “Can the workflow be repeated?” Data management also asks, “What evidence entered the workflow, what does it mean, and is it suitable for this use?”

Data are not self-explanatory

A rectangular file may look simple:

Area	Year	Element	Unit	Value	Flag
Zambia	2020	Production	t	...	A
Zambia	2020	Area harvested	ha	...	E
Zambia	2020	Yield	kg/ha	...	I

The values cannot be interpreted safely without additional information:

- What does each element mean?
- Is `t` tonnes or another unit?
- What do the flags A, E, and I mean?
- Are the values measured, estimated, imputed, or calculated?
- Does 2020 refer to a calendar year, harvest year, or reporting period?
- Which geographic definition of Zambia is used?
- Which variables together identify a unique observation?

Metadata, code lists, methodological notes, and provenance are therefore part of the data evidence—not optional decoration.

The data lifecycle

Data move through connected stages:

```
Plan
↓
Acquire
↓
Describe and organize
↓
Validate
↓
Prepare and analyze
↓
Share
↓
Preserve or dispose
```

The lifecycle is not strictly linear: validation may reveal that the wrong source was acquired, analysis may expose an incomplete data dictionary, or a new release may require repeating acquisition and checks.

Decisions made early constrain later work. An unrecorded source, unit, license, or grain can leave later analysts unable to interpret or share the results responsibly.

What data management contributes

Good data management makes several aspects of a project explicit.

Meaning

- the observational unit and dataset grain;
- variable definitions, units, classifications, and flags;
- the difference between identifiers, labels, and measurements.

Origin and history

- provider and dataset;
- version or release;
- access date and retrieval method;
- transformations that produced derived artifacts.

Quality

- expected structure and coverage;
- validation rules and results;
- known anomalies, limitations, and unresolved questions.

Organization

- which files are raw inputs;
- which files are derived;
- which scripts create each output;
- which data should or should not be stored in Git.

Responsibility

- license and citation requirements;
 - whether redistribution is permitted;
 - privacy, confidentiality, security, and access constraints;
 - how long data should be retained.
-

Why this matters in food-systems research

Food-systems analyses often combine observations from different institutions and purposes. Common challenges include:

- changing country or administrative boundaries;
- different commodity and land-use classifications;
- annual, seasonal, monthly, and daily reporting periods;
- estimated, imputed, modeled, and directly observed values;
- measurements at farm, household, market, district, country, or grid-cell level;
- multiple unit conventions;
- incomplete coverage of informal activities or marginalized populations;
- revisions to historical series.

A dataset can be internally tidy and technically valid while still being unsuitable for a particular research question. Fitness for purpose requires scientific judgment and knowledge of how the values were produced.

What data management cannot guarantee

Data management can improve transparency and reduce avoidable errors. It cannot guarantee:

- that the provider measured the intended concept accurately;
- that the sample represents the target population;
- that missing groups or activities are unimportant;
- that a documented assumption is scientifically valid;
- that a FAIR dataset is open or unrestricted;
- that a reproducible result supports a causal conclusion.

Documentation makes limitations visible. It does not remove them.

How this connects to the maize-yield project

The Data Management session produces a documented package around a supplied FAOSTAT extract:

```
maize-yield-project/  
  data-raw/  
    faostat-maize.csv  
  metadata/  
    data-dictionary.csv  
    provenance.yml  
  scripts/  
    validate-data.R  
  reports/  
    data-validation.html
```

The goal is not yet to clean or model the data. The goal is to make it possible for another person to determine:

- what one row represents;
 - what each variable, unit, code, and flag means;
 - where the data came from;
 - which validation checks passed or failed;
 - which concerns remain; and
 - how the data may be used and shared.
-

Check your understanding

1. A report can be reproduced exactly. Name three reasons why its results could still be misleading.
 2. Why are provider metadata part of the evidence rather than optional background reading?
 3. Which information belongs in a provenance record?
 4. Why does “raw” describe a role rather than a guarantee of quality?
 5. Give one example of a technically valid food-systems dataset that may not be fit for a particular research question.
 6. Which questions should be answered before data are committed to Git or shared publicly?
-

Further resources

Use these resources to deepen the motivation and place this session in the wider research-data-management landscape:

- [Research Data Management — The Turing Way](#) connects storage, documentation, metadata, sharing, and FAIR practice to reproducible research.
- [Overview of Reproducible Research — The Turing Way](#) places data planning, processing, and reuse within the wider research lifecycle.
- [Research Data Management — UK Data Service](#) is a practical learning hub covering the data lifecycle, rights, storage, and sharing.

- [Data management checklist — UK Data Service](#) offers planning questions on documentation, access, and preservation.
 - Wilkinson, M. D. et al. (2016), [The FAIR Guiding Principles for scientific data management and stewardship](#), introduces the Findable, Accessible, Interoperable, and Reusable principles.
-

Continue to Concepts

Continue with [Understand data-management concepts](#). It introduces dataset grain, keys, structures, meta-data, provenance, quality, project organization, and responsible use—the concepts required by the application.

4.2) Understand data-management concepts

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Observations, variables, and values
 - Dataset grain
 - Identifiers, labels, and keys
 - Common data structures
 - Cross-sectional data
 - Time-series data
 - Panel data
 - Hierarchical data
 - Spatial vector data
 - Raster or gridded data
 - Metadata
 - Data dictionaries
 - Provenance
 - Data quality and fitness for purpose
 - Project organization
 - Licensing, privacy, and FAIR principles
 - From concepts to application
 - Check your understanding
 - Further resources
 - Research data and the lifecycle
 - Data structure and organization
 - Metadata and documentation
 - FAIR principles
-

Learning objectives

After completing this page, you should be able to:

- state the observational unit and grain of a dataset;
 - distinguish identifiers, labels, measures, units, and flags;
 - propose and test a candidate key;
 - recognize common tabular, temporal, hierarchical, and spatial structures;
 - distinguish source metadata, a project data dictionary, and provenance;
 - organize data artifacts by their role in a workflow; and
 - identify basic conditions for responsible storage and reuse.
-

Place in the session

This is the **Concepts** part of the Data Management session:

```
Motivation → Concepts → Application
              ↑
            this page
```

The [Motivation page](#) explains why reproducible computation is not sufficient when data are misunderstood. This page provides the vocabulary and decisions needed to complete [the maize data-management application](#).

Use the concepts as questions to ask of a dataset, not only as definitions to memorize:

- What does one row represent?
- Which variables identify it?
- Which metadata give its values meaning?
- How can its structure and meaning be checked?
- Under which conditions may it be stored, used, or shared?

Observations, variables, and values

For a rectangular dataset:

- an **observation** records information about one unit in one defined context;
- a **variable** records one property across observations;
- a **value** is the recorded content for one variable in one observation.

These definitions depend on the intended structure. In a country–year panel, an observation may be a country in a year. In a more detailed FAOSTAT table, the observation may be an area–year–item–element combination.

Variables can play different roles:

Role	Example	Purpose
Identifier	Area Code	Identifies an entity
Label	Area	Displays a readable name
Time	Year	Locates an observation in time
Classification	Item Code, Element Code	Identifies a defined category
Measure	Value	Records a quantity
Unit	Unit	Defines the measurement scale
Quality information	Flag	Qualifies how a value was obtained

Do not assume that a numeric column is a measure. Codes may be stored as numbers but should not be added or averaged.

Dataset grain

The **grain** states what one row represents. Write it as a sentence:

One row represents one element for one item in one area and year, expressed in one unit.

Grain is important because it determines:

- which variables should identify a row uniquely;
- which comparisons are meaningful;
- whether a join will preserve or multiply observations;
- whether an aggregation changes the scientific meaning.

Before using a dataset, answer:

1. What real-world or statistical unit does one row represent?
 2. At what temporal resolution is it observed?
 3. At what geographic resolution is it observed?
 4. Which classifications distinguish otherwise similar rows?
 5. Can more than one record exist for the same apparent unit?
-

Identifiers, labels, and keys

An **identifier** is intended to distinguish an entity. A **label** is intended for people to read.

Identifiers are usually safer for matching because labels can differ in spelling, punctuation, language, abbreviation, capitalization, or naming convention over time.

A **candidate key** is a set of variables expected to identify each row uniquely. A **composite key** uses more than one variable.

For a FAOSTAT extract, a candidate key could include:

```
Area Code + Year + Item Code + Element Code + Unit
```

The exact key must be inferred from documentation and tested against the data. Finding duplicates does not automatically mean that one row should be deleted. The proposed key may be incomplete, or the source may intentionally publish multiple statuses, methods, or revisions.

Common data structures

Cross-sectional data

Many units observed for one period, such as maize yield for several countries in 2022.

Time-series data

One unit observed through time, such as annual maize yield for Zambia from 1990 to 2022.

Panel data

Many units observed through time, such as annual maize yield for several countries.

Hierarchical data

Units are nested, such as farms within districts and districts within countries. Records within the same group may not be independent.

Spatial vector data

Locations are represented by points, lines, or polygons. Examples include markets, roads, and administrative boundaries.

Raster or gridded data

Space is divided into cells. Climate and satellite products often use rasters. Resolution, extent, and coordinate reference system are essential metadata.

These structures can coexist. A panel can be spatial, and sensor observations can form several time series at point locations.

Metadata

Metadata are data about data. Useful source metadata may describe concepts and definitions, collection or estimation methods, population and geographic coverage, temporal coverage and frequency, classifications and code lists, units and conversion rules, missing-value conventions, quality flags and revisions, and license and citation requirements.

Metadata should be read before interpreting values. A familiar column name does not guarantee a familiar definition.

Keep a provider's metadata or a stable reference to it when permitted. External pages and code lists may change.

Data dictionaries

A project data dictionary documents the variables actually used by the project. It complements rather than replaces provider metadata.

A suggested structure is:

```
variable,label,definition,type,unit,role,allowed_values,missing_values,source
Area_Code,Area code,FAOSTAT area identifier,character,,identifier,...,FAOSTAT
Year,Year,Reporting year,integer,year,time,1990-2022,,FAOSTAT
Value,Value,Recorded value,double,varies,measure,,,FAOSTAT
```

Recommended fields:

- source variable name;
- project variable name, if different;
- label and precise definition;
- data type;
- unit;
- role in the dataset;
- allowed values or code-list reference;
- missing-value representation;
- source and methodological note.

Avoid definitions that merely repeat the variable name. “Yield: yield value” does not explain the concept, calculation, or unit.

Provenance

Provenance records where data came from and what happened to them.

A small project can use YAML:

```
artifact: data-raw/faostat-maize.csv
provider: Food and Agriculture Organization of the United Nations
dataset: FAOSTAT crop and livestock products
release: "record the release if supplied"
accessed: YYYY-MM-DD
source: "record the URL, endpoint, or delivery method"
license: "record the provider's license or terms"
retrieval: "manual snapshot supplied for the exercise"
checksum_sha256: "record the checksum"
```

For a derived file, also record:

- input artifacts;
- script or workflow step;
- relevant parameters;
- aggregation or conversion rules;
- output artifact;
- known information loss.

Provenance should allow a person to trace a result back to its sources. It is not the same as a data dictionary, which explains variables.

Data quality and fitness for purpose

Data quality is not one universal score. It describes whether data are suitable for an intended use and whether their relevant limitations are understood.

Useful dimensions include:

Dimension	Question
Completeness	Are the expected variables, entities, periods, and values present?
Validity	Do values follow documented types, formats, codes, and rules?
Uniqueness	Does each expected key identify no more than one observation?
Consistency	Do related values, units, classifications, and records agree?
Plausibility	Are values credible given the concept and context?
Timeliness	Are the reference period and release current enough for the intended use?
Accuracy	How closely do values represent the real quantity or state of interest?

Accuracy often cannot be established from the file alone. It may require knowledge of collection methods, sampling, measurement error, estimation, revision, and comparison with independent evidence.

Validation converts justified expectations into repeatable checks. A failed check does not always prove that the source is wrong: the expectation may be wrong, the key may be incomplete, or the data may document a legitimate exception. Preserve the evidence and investigate before correcting or excluding records.

Project organization

A useful starting structure is:

```
maize-yield-project/
  data-raw/           # unchanged source responses
  data-interim/       # outputs between source and analysis-ready data
  data-processed/     # analysis-ready outputs created by code
  metadata/           # dictionaries, code lists, licenses, provenance
  scripts/            # acquisition, validation, integration, preparation
  reports/            # validation, analysis, and communication
```

Guidelines:

- Do not manually edit raw source files.
 - Use code to create derived files.
 - Document the connection between every output and its inputs.
 - Keep credentials outside scripts and version control.
 - Do not commit data merely because they fit in Git.
 - Use `.gitignore` for files that are large, sensitive, restricted, or reproducibly retrieved.
 - Preserve a source snapshot when permitted and necessary for reproducibility.
 - Use checksums or source releases to identify exact inputs.
-

Licensing, privacy, and FAIR principles

Before storing, processing, or sharing data, determine:

- who owns or controls the data;
- which license or terms apply;
- whether attribution is required;
- whether raw or derived data may be redistributed;
- whether the data contain personal, confidential, commercially sensitive, or location-sensitive information;
- who should have access and how access is controlled;
- when data should be archived or deleted.

FAIR data are:

- **Findable**;
- **Accessible** under clearly stated conditions;
- **Interoperable** through shared formats, vocabularies, and identifiers;
- **Reusable** because meaning, provenance, and conditions are documented.

FAIR does not mean that every person must have unrestricted access. Sensitive data can be FAIR when their existence, metadata, access process, and conditions are clear.

From concepts to application

The application turns each concept into an inspectable project artifact or check:

Concept	Application evidence
Grain and key	One-sentence grain statement and duplicate-key check
Metadata	Variable definitions, units, code lists, and flags
Data dictionary	<code>metadata/data-dictionary.csv</code>
Provenance	<code>metadata/provenance.yml</code> and source checksum
Data quality	Structural and semantic validation checks
Project organization	Raw input remains unchanged; derived artifacts have separate roles
Responsible use	License, citation, access, and sharing conditions are recorded

Continue with [Manage the maize-yield data](#). During the exercise, return to this page whenever a task requires a definition or scientific decision.

Check your understanding

1. Write the grain of a country-year-item-element dataset in one sentence.
 2. Why should country codes usually be used instead of country names for matching?
 3. How would you investigate duplicate candidate keys before deciding to remove anything?
 4. What is the difference between provider metadata, a data dictionary, and provenance?
 5. Why might a raw source file be excluded from Git?
 6. Can restricted data satisfy FAIR principles? Explain your answer.
-

Further resources

The following resources expand particular concepts from this page:

Research data and the lifecycle

- [Research Data — The Turing Way](#) introduces research data, the data lifecycle, and data-management planning.
- [Research Data Management — The Turing Way](#) links documentation, storage, sharing, preservation, and FAIR practice to reproducibility.

Data structure and organization

- [Data Organisation in Spreadsheets — The Turing Way](#) explains observations, variables, values, consistent representation, and validation for spreadsheet data.
- [Data Carpentry lessons](#) provide beginner-friendly, domain-based lessons on organizing, cleaning, managing, and analyzing data, including ecology and geospatial curricula.

Metadata and documentation

- [Metadata — UK Data Service](#) explains structured metadata and introduces standards such as Dublin Core, DDI, SDMX, and DataCite.
- [Data documentation: quantitative data — UK Data Service](#) provides practical guidance on variable names, labels, definitions, units, code lists, missing values, and codebooks.
- [Study-level documentation — UK Data Service](#) describes the wider context needed to understand a dataset, including purpose, methods, coverage, processing, quality assurance, access, and licensing.

FAIR principles

- Wilkinson, M. D. et al. (2016), [The FAIR Guiding Principles for scientific data management and stewardship](#), is the foundational publication. Note that FAIR is a set of high-level principles rather than a specific technology or standard.

4.3) Documentation tools for data management

Outline

- Outline
 - Learning objectives
 - Why use plain-text documentation?
 - Human-readable and machine-readable are not opposites
 - Choose a format by information structure
 - Use Markdown when
 - Use YAML when
 - Use CSV when
 - Decision table
 - How the formats work together
 - Prefer one authoritative owner
 - SHA-256 checksums for file identity
 - What is a checksum?
 - What a matching checksum establishes
 - What a checksum does not establish
 - Integrity is not authenticity
 - Why use SHA-256 rather than SHA-1 or MD5?
 - Continue to the reference page
-

Learning objectives

After completing this page, you should be able to:

- choose Markdown, YAML, or CSV according to the structure and intended use of documentation;
- explain how the three formats complement one another in a project;
- explain what a SHA-256 checksum can and cannot establish about a file.

The [reference page](#) that follows this one covers the concrete syntax, a full checksum workflow, validation checks, and a worked application to the maize-yield project.

Why use plain-text documentation?

Markdown, YAML, and CSV are plain-text formats. Plain text is useful in a data science project because it can be:

- opened with many editors and operating systems;
- reviewed without specialized software;
- searched with command-line tools;
- processed by scripts;

- compared line by line with Git;
- included in code review; and
- preserved more easily than an undocumented binary format.

Plain text does not automatically make documentation correct or reproducible. A text file can still be ambiguous, malformed, incomplete, stale, or unsafe to share. The project must define what each file is for and how it is checked.

Human-readable and machine-readable are not opposites

The three formats occupy different positions:

Format	Main strength	Typical reader
Markdown	Narrative explanation and readable structure	People first; rendering tools second
YAML	Nested structured records and configuration	People and programs
CSV	Repeated rectangular records	Programs and spreadsheet/table tools

A useful data-management setup normally uses several formats rather than forcing every kind of information into one file.

Choose a format by information structure

Begin with the structure of the information, not with a preferred file extension.

Use Markdown when

- the order of explanation matters;
- the document contains sections, guidance, decisions, or interpretation;
- readers need examples, tables, lists, links, and code snippets;
- the content is primarily narrative; or
- the document should render as a readable web page or report.

Examples:

- a data-management implementation guide;
 - a README;
 - a validation-report narrative;
 - instructions for updating a snapshot; and
 - a description of limitations and responsible use.
-

Use YAML when

- the information consists of named fields;
- values are nested or grouped;
- one record contains lists or sub-records;
- software must retrieve fields by name; or
- the file configures a tool or records structured provenance.

Examples:

- a provenance record;
 - source metadata;
 - a source register;
 - a pipeline configuration; and
 - a module or report configuration.
-

Use CSV when

- every record has the same fields;
- the information is naturally rectangular;
- one row represents one variable, code, country, source, or validation check;
- table operations such as filtering and joining are useful; or
- learners should inspect the information with R or a spreadsheet.

Examples:

- a data dictionary;
 - a code list;
 - an identifier crosswalk;
 - a validation-results table; and
 - a register with one row per artifact when no nested fields are required.
-

Decision table

Requirement	Markdown	YAML	CSV
Long narrative	Strong	Weak	Poor
Headings and explanatory flow	Strong	Poor	Poor
Named fields	Possible	Strong	Strong
Nested structures	Awkward	Strong	Poor
Repeated uniform records	Possible	Possible	Strong
Tables with many rows	Weak	Weak	Strong
Comments	Native prose	Supported	No standard comment syntax

Requirement	Markdown	YAML	CSV
Easy line-by-line Git review	Strong	Strong	Strong for stable row order
Direct use in R	Via a parser/rendering tool	Via a YAML parser	Via a table reader
Formal schema validation	Tool-dependent	Possible	Possible

How the formats work together

A project can separate responsibilities like this:

```
docs/data-management.md
  Markdown narrative: policy, workflow, decisions, and limitations

metadata/source-metadata.yml
  YAML record: provider dataset, methods, scope, classifications, references

metadata/data-dictionary.csv
  CSV table: one row per project variable

metadata/flag-code-list.csv
  CSV table: one row per provider flag

metadata/provenance.yml
  YAML record: exact artifact, access, retrieval, checksum, and conditions
```

The files should link to one another rather than repeat all content:

```
Markdown guide
  points to source metadata
  points to dictionary and code lists
  explains how provenance is updated

Provenance
  identifies the artifact
  points to source metadata
  records the dictionary relevant to that artifact
```

Prefer one authoritative owner

If the licence is recorded in several files, decide which one owns the exact text and which files reference it. Otherwise, one copy may change while another remains stale.

Duplication is sometimes useful for readability, but duplicated fields require a consistency check or a maintenance rule.

SHA-256 checksums for file identity

What is a checksum?

A cryptographic hash function maps the bytes of a file to a fixed-length digest. SHA-256 produces a 256-bit digest, commonly displayed as 64 hexadecimal characters.

For example:

```
fd2c78cae5a5cf2f82d6b6bdc2b3637ce03b597f74e561099f9666af449605be
```

The same bytes produce the same digest. A changed byte should produce a different digest with extremely high probability.

What a matching checksum establishes

If a newly calculated digest matches a separately recorded expected digest, the file is byte-for-byte identical to the state represented by that expected digest.

This can help answer:

- Is this the fixed teaching snapshot?
 - Did a configuration file change?
 - Was a downloaded file transferred without byte-level alteration?
 - Does a generated artifact match a reviewed release?
 - Did line endings, encoding, whitespace, or final newline change?
-

What a checksum does not establish

A matching checksum does **not** prove that the file is:

- correct;
- complete in a scientific sense;
- well documented;
- valid according to a schema;
- free of bias or measurement error;
- safe to execute;
- licensed for the intended use;
- authentic when the expected checksum came from an untrusted source; or
- unchanged in meaning when external definitions have changed.

Checksums establish file identity, not data quality or fitness for purpose.

Integrity is not authenticity

Suppose an attacker can replace both:

```
configuration.yml
configuration.yml.sha256
```

The new file can match the new checksum. The checksum alone does not identify who approved it.

For stronger authenticity, obtain the expected digest through a trusted channel or combine checksums with controls such as:

- reviewed Git history;
- protected branches;
- signed commits or tags;
- digital signatures;
- access controls; and
- published release manifests.

Why use SHA-256 rather than SHA-1 or MD5?

SHA-256 is part of the SHA-2 family and is appropriate for routine file identity and integrity checks. MD5 and SHA-1 should not be selected for new security-sensitive integrity designs because of known collision weaknesses.

The algorithm must always be recorded with the digest. A hexadecimal string without an algorithm name is ambiguous.

Continue to the reference page

The [tools reference page](#) covers concrete Markdown, YAML, and CSV syntax and guidelines, a step-by-step checksum workflow, validation checks per format, the application to the maize-yield project, common mistakes, and completion checklists. Return here whenever you need to decide which format to use; use the reference page when you need the exact syntax or workflow.

4.4) Manage the maize-yield data

Outline

- Outline
- Learning objectives
- Place in the session
- Scenario and deliverable
- Before you begin
- 1. Preserve and identify the raw input
- 2. Establish the dataset grain
- 3. Create a data dictionary
- 4. Record provenance
- 5. Organize and govern the project artifacts
- 6. Validate structure
 - Required columns
 - Coverage
 - Candidate-key uniqueness
 - Allowed categories
- 7. Validate meaning
 - Missingness
 - Ranges
 - Flags
 - Internal consistency
- 8. Report rather than silently repair
- Troubleshooting
 - Problem 01: Column names differ from the example
 - Problem 02: The candidate key is not unique
 - Problem 03: Numbers were imported as text
 - Problem 04: The checksum differs from another learner's
 - Problem 05: A validation check stops the script
- Completion checklist
- Reflect on the application
- Further resources
 - Practical workflow and checks
 - Organization and documentation
 - Working with tabular data in R

Learning objectives

After completing this exercise, you should be able to:

- inspect a source file without changing it;
- state and test its expected grain and key;
- build a project data dictionary and provenance record;
- organize project artifacts and make justified storage and sharing decisions;
- implement structural and semantic checks in R;

- distinguish a failed validation from a cleaning decision; and
- communicate unresolved data-quality concerns.

Place in the session

This is the **Application** part of the Data Management session:

```
Motivation → Concepts → Application
                        ↑
                    this page
```

Before beginning, review [Why manage data?](#) and [Understand data-management concepts](#). The exercise assumes you can explain grain, candidate keys, metadata, a data dictionary, provenance, organization, and fitness for purpose.

The numbered steps apply those concepts. Do not continue past a failed expectation to reach the end of the instructions — inspect the evidence, explain the failure, and record what must be resolved.

Scenario and deliverable

You have received a fixed FAOSTAT teaching extract of maize production, harvested area, and yield for selected Southern African countries. Before anyone prepares, visualizes, or models these values, the project team must determine what they mean and whether they match the expected structure.

Your task is to create an inspectable data-management package:

```
maize-yield-project/
  data-raw/
    faostat-maize.csv      # supplied input; remains unchanged
  metadata/
    data-dictionary.csv   # meaning of project variables
    provenance.yml        # source and history of the input
  scripts/
    validate-data.R       # repeatable expectations and checks
  reports/
    data-validation.html   # findings, limitations, and status
```

The package is complete when another person can determine:

- what one row and each variable represent;
- where the data came from and whether the exact input changed;
- which structural and semantic expectations were checked;
- which checks passed, warned, failed, or remain unknown;
- which license, citation, access, and sharing conditions apply;
- which decisions are deferred to Data Preparation.

Before you begin

Use the supplied maize-yield project and fixed FAOSTAT teaching extract. Begin at the project root:

```
pwd
git status
ls data-raw metadata scripts reports
```

The exercise assumes that the raw file is named:

```
data-raw/faostat-maize.csv
```

Do not open and resave the raw file in spreadsheet software — this can silently alter dates, encodings, numeric precision, delimiters, and quoting.

Create derived documentation and reports in separate directories; do not replace the source file.

1. Preserve and identify the raw input

Inspect file properties from the terminal:

```
ls -lh data-raw/faostat-maize.csv
file data-raw/faostat-maize.csv
sha256sum data-raw/faostat-maize.csv
```

On systems without `sha256sum`, use an available SHA-256 tool and record which tool was used.

The checksum is a fingerprint of the file's bytes. If the file changes, its checksum should change. A matching checksum does not establish scientific correctness; it establishes that two byte sequences are the same.

Record:

- filename and relative path;
- file size;
- checksum;
- date received or accessed;
- who supplied or retrieved it.

2. Establish the dataset grain

Read the file without modifying it:

```
maize_raw <- readr::read_csv(
  "data-raw/faostat-maize.csv",
  show_col_types = FALSE
)

dplyr::glimpse(maize_raw)
names(maize_raw)
nrow(maize_raw)
```

Use provider documentation and the columns to complete this statement:

One row represents _____.

Propose a composite key. Adapt the names below to the actual extract:

```
candidate_key <- c(
  "Area Code",
  "Year",
  "Item Code",
  "Element Code",
  "Unit"
)
```

Test it:

```
duplicate_keys <- maize_raw |>
  dplyr::count(dplyr::across(dplyr::all_of(candidate_key))) |>
  dplyr::filter(n > 1)

duplicate_keys
```

If duplicates appear, do not remove them immediately. Ask:

- Is the proposed key incomplete?
- Is there another flag, status, method, or version variable?
- Does the provider allow multiple records for the same apparent unit?
- Is this a genuine source anomaly?

Document your conclusion.

3. Create a data dictionary

Create `metadata/data-dictionary.csv` with one row per retained source variable.

Include:

```
variable
label
definition
type
unit
role
allowed_values
missing_values
source
notes
```

For code variables, point to the relevant code list. For **Value**, explain that its unit depends on the associated element/unit variables. For flags, record the provider's definition rather than guessing from the letter.

Review the dictionary using these questions:

- Would a learner understand the variable without opening the source portal?
- Is the difference between label and identifier clear?
- Are units explicit?
- Are special missing codes documented?
- Are quality flags explained?
- Does each definition describe the concept rather than repeat the name?

4. Record provenance

Create `metadata/provenance.yml`:

```
artifact: data-raw/faostat-maize.csv
provider: Food and Agriculture Organization of the United Nations
dataset: "record the exact dataset name"
release: "record if available"
accessed: "YYYY-MM-DD"
source: "record URL, endpoint, or supplied teaching snapshot"
license: "record the applicable terms"
retrieval: "describe the retrieval or delivery process"
checksum_sha256: "paste the checksum"
metadata_reference: "record the source documentation"
notes: "record known limitations or revisions"
```

Do not write secrets, access tokens, passwords, or private URLs into the record.

Commit the provenance record and dictionary when they contain no restricted information. They make future changes to data meaning and source history visible in Git.

5. Organize and govern the project artifacts

Confirm that each artifact has one clear role:

Artifact	Role	Typical Git decision
<code>data-raw/faostat-maize.csv</code>	Unchanged source evidence	Decide from size, license, sensitivity, and reproducibility
<code>metadata/data-dictionary.csv</code>	Meaning of project variables	Commit when it contains no restricted information
<code>metadata/provenance.yml</code>	Source and artifact history	Commit after removing credentials or private locations
<code>scripts/validate-data.R</code>	Repeatable expectations and checks	Commit
<code>reports/data-validation.html</code>	Rendered findings for review	Follow the project's output policy

Check the source license before deciding whether the raw file or derived outputs may be redistributed, and record the required citation. Personal, confidential, or sensitive data should follow the project's access and retention rules instead of the general repository.

Inspect the version-control state:

```
git status --short
git check-ignore -v data-raw/faostat-maize.csv
```

The second command reports the matching ignore rule, if any; no output does not mean committing the file is appropriate.

Document the storage decision in the project README so another contributor never has to infer whether a missing raw file must be downloaded, requested, mounted, or restored from an archive.

6. Validate structure

Create `scripts/validate-data.R`. Start by expressing expectations as code.

Required columns

```
required_columns <- c(
  "Area Code",
  "Area",
  "Year",
  "Item Code",
  "Item",
  "Element Code",
  "Element",
  "Unit",
  "Value"
)

missing_columns <- setdiff(required_columns, names(maize_raw))
stopifnot(length(missing_columns) == 0)
```

Coverage

```
range(maize_raw$Year, na.rm = TRUE)
dplyr::n_distinct(maize_raw$`Area Code`)
dplyr::count(maize_raw, Area, sort = TRUE)
```

Compare the results with the intended countries and years; do not hard-code an expectation until you can explain where it came from.

Candidate-key uniqueness

```
stopifnot(nrow(duplicate_keys) == 0)
```

Use a clear error or report when the expectation fails. A failed check is useful evidence, not an inconvenience to suppress.

Allowed categories

```
dplyr::count(maize_raw, Element, Unit, sort = TRUE)
dplyr::count(maize_raw, Item, sort = TRUE)
```

Compare observed categories with the data dictionary and source code lists.

7. Validate meaning

Structural validity does not establish that values are plausible or correctly interpreted.

Missingness

```
maize_raw |>
  dplyr::summarise(
    dplyr::across(
      dplyr::everything(),
      ~ sum(is.na(.x))
    )
  )
```

Ask whether missingness varies by country, year, element, or flag. A blank value is not automatically zero.

Ranges

```
maize_raw |>
  dplyr::group_by(Element, Unit) |>
  dplyr::summarise(
    minimum = min(Value, na.rm = TRUE),
    maximum = max(Value, na.rm = TRUE),
    .groups = "drop"
  )
```

Plausible ranges depend on element and unit. A single universal threshold is not appropriate for production, area, and yield.

Flags

```
if ("Flag" %in% names(maize_raw)) {
  print(dplyr::count(maize_raw, Flag, sort = TRUE))
}
```

Use provider metadata to interpret each flag. Decide whether it should be retained, summarized, or used to qualify later conclusions.

Internal consistency

If the dataset contains production, harvested area, and yield, investigate whether their definitions imply an approximate relationship. Check units and provider methodology before calculating anything. Differences may reflect rounding, conversions, estimates, or definitions rather than errors.

8. Report rather than silently repair

Render `reports/data-validation.html` or a Markdown equivalent. Include:

1. source and checksum;
2. intended use;
3. stated grain and candidate key;
4. dimensions and coverage;
5. required-column and type checks;
6. duplicate-key results;
7. categories, units, and flags;
8. missingness and range summaries;
9. failed checks and anomalies;
10. limitations and decisions deferred to data preparation.

Classify findings:

- **Pass:** observed data match a justified expectation.
- **Warning:** data can continue through the workflow, but the issue needs attention.
- **Failure:** the input is not the expected dataset or cannot safely proceed.
- **Unknown:** additional provider or subject-matter information is required.

Do not turn every warning into an automatic deletion — preserve the evidence and document the reasoning needed for later preparation.

Troubleshooting

Problem 01: Column names differ from the example

Inspect `names(maize_raw)`, adapt the exercise to the actual schema, and record the real names in the dictionary — do not rename columns inside the raw file.

Problem 02: The candidate key is not unique

Inspect several duplicated key combinations for omitted dimensions or statuses, and consult the metadata before changing the key or removing rows.

Problem 03: Numbers were imported as text

Inspect the problematic values and import specification — likely causes are non-numeric symbols, decimal conventions, or provider-specific missing codes. Document the source representation; conversion belongs in a derived step.

Problem 04: The checksum differs from another learner's

Compare file sizes, source versions, access dates, and retrieval methods — do not assume either file is authoritative until the origins are established.

Problem 05: A validation check stops the script

Read the failed expectation and inspect the relevant records. Change the check only when the expectation was wrong and the revised expectation can be justified.

Completion checklist

- ☐ The raw input remains unchanged.
 - ☐ Its file size and checksum are recorded.
 - ☐ The dataset grain is stated in one sentence.
 - ☐ The candidate key is tested and duplicate results are explained.
 - ☐ Every project variable is documented in a data dictionary.
 - ☐ Source, access, release, license, and retrieval are recorded.
 - ☐ Every artifact has a clear role and documented storage/version-control decision.
 - ☐ Citation, redistribution, sensitivity, access, and retention conditions were considered.
 - ☐ Structural and semantic checks run from a script.
 - ☐ Failed checks and unknowns remain visible.
 - ☐ A validation report summarizes evidence and limitations.
 - ☐ Git status contains only the artifacts intended for version control.
-

Reflect on the application

Answer these questions after completing the checklist:

1. Which concept from the Concepts page changed how you interpreted the raw file?
2. Which candidate-key variables were necessary, and what would go wrong if one were omitted?
3. Which validation result required scientific judgment rather than only a technical check?
4. Which issue, if any, should stop the data from moving to Data Preparation?
5. Which warning can move forward if it remains documented and visible?
6. Which artifacts should be committed to Git, and which should be ignored or stored elsewhere? Justify the decision.
7. What can a future learner reproduce from your package, and what still depends on the external provider?

The next topic, **Data Acquisition & Integration**, asks how to retrieve this and a complementary source reproducibly, align their identifiers and coverage, and audit the resulting integration.

Further resources

Use these resources while implementing or reviewing a data-management workflow:

Practical workflow and checks

- [Research Data Management Checklist — The Turing Way](#) summarizes practical actions for raw-data preservation, planning, and documentation.
- [Data Cleaning — The Turing Way](#) discusses raw-data backups and reproducible cleaning; use it as a bridge to Data Preparation, not a guide for this application.
- [Data management checklist — UK Data Service](#) offers questions for reviewing documentation, storage, access, and sharing.

Organization and documentation

- [Organising data — UK Data Service](#) gives guidance on folder structures and meaningful file names.
- [Study-level documentation — UK Data Service](#) is a checklist for documenting purpose, methods, coverage, and licensing.
- [Data documentation: quantitative data — UK Data Service](#) supports the dictionary task with guidance on variables, units, and missing values.

Working with tabular data in R

- [Starting with data — Data Carpentry](#) introduces importing CSV files and inspecting data frames in R.
- [Data Analysis and Visualisation in R for Ecologists — Data Carpentry](#) provides a broader hands-on sequence for organizing and analyzing tabular data.

4.5) Documentation tools for data management — reference

This page is the detailed reference companion to [Documentation tools for data management](#). Read that page first for the concepts — why plain text, how to choose Markdown, YAML, or CSV, and what a checksum establishes. This page collects the concrete syntax, a full checksum workflow, validation checks, the maize-yield application, common mistakes, and completion checklists.

Outline

- Outline
- Markdown for explanations and guidance
 - What Markdown represents
 - Basic structure
 - Lists
 - Tables
 - Code and file names
 - Links
 - Markdown guidelines for data management
 - Markdown limitations
- YAML for structured records and configuration
 - What YAML represents
 - Mapping
 - Sequence
 - Nested record
 - Literal and folded text
 - Quote ambiguous values
 - Comments
 - YAML for provenance
 - YAML for source metadata
 - YAML guidelines for data management
 - YAML limitations
- CSV for rectangular documentation
 - What CSV represents
 - Simple example
 - Quoting
 - CSV for a data dictionary
 - CSV for a code list
 - CSV for a crosswalk
 - CSV guidelines for data management
 - CSV limitations
- A practical checksum workflow
 - 1. Choose the artifact
 - 2. Calculate SHA-256 on Linux
 - 3. Calculate SHA-256 on macOS
 - 4. Calculate SHA-256 with PowerShell
 - 5. Calculate SHA-256 in R
 - 6. Record the expected digest

- 7. Use a checksum manifest
- 8. Verify in R
- 9. Investigate a mismatch
- Validating documentation and configuration
 - Four different questions
 - Markdown checks
 - YAML checks
 - CSV checks
 - Byte identity versus semantic equivalence
- Application to the maize-yield project
 - Example workflow
 - Appropriate checksum targets
 - Suggested validation responsibilities
- Common mistakes
 - Choosing by file extension rather than structure
 - Treating Markdown as structured configuration
 - Treating YAML as free-form prose
 - Treating CSV as a spreadsheet layout
 - Relying on automatic types
 - Accepting a parse as validation
 - Updating a checksum without reviewing the change
 - Storing the only expected digest with the untrusted artifact
 - Hashing a logical value instead of file bytes
 - Assuming a checksum proves correctness
 - Committing secrets in YAML
- Completion checklist
 - Format choice
 - Markdown
 - YAML
 - CSV
 - Checksums
- Check your understanding
- Further resources
 - File formats
 - Checksums
 - Data-management context

Markdown for explanations and guidance

What Markdown represents

Markdown represents the structure of a document using plain-text conventions. It is suitable for prose, headings, lists, links, quotations, tables, and code examples.

Markdown implementations have dialects and extensions. A project should state which renderer or convention it expects. CommonMark provides a strongly specified core syntax; platforms may add tables, task lists, footnotes, or other features.

Basic structure

```
# Data-management implementation
```

This document explains how the project manages its teaching data.

```
## Raw-data policy
```

- Preserve the tracked teaching snapshot.
- Do not edit raw files manually.
- Ignore large provider downloads.

```
## Validate the snapshot
```

Run:

```
```bash
Rscript scripts/validate-data.R
```
```

Use heading levels hierarchically:

```
# Document title
## Main section
### Subsection
```

Do not choose a heading level only for its visual size. Heading hierarchy communicates document structure to renderers, accessibility tools, and readers.

Lists

Use bulleted lists when order does not matter:

- provider
- dataset
- access date
- licence

Use numbered lists when sequence matters:

1. Acquire the source file.
 2. Record its checksum.
 3. Validate its structure.
 4. Review the results.
-

Tables

Markdown tables work well for small comparison or decision tables:

```
Artifact	Role	Git policy
Teaching snapshot	Fixed input	Track
Bulk download	External working file	Ignore
```

Do not use a large Markdown table as a substitute for a machine-readable CSV. Wide tables become hard to edit, parse, and review.

Code and file names

Use inline code for commands, field names, and repository-relative paths:

```
Run `scripts/validate-data.R` and inspect the `status` column.
```

Use fenced code blocks for examples:

```
```yaml  
provider: FAO
dataset: Crops and Livestock Products
```
```

Specify a language after the opening fence when possible. This improves readability and syntax highlighting.

Links

Prefer descriptive link text:

```
[FAOSTAT Crops and Livestock Products](https://www.fao.org/faostat/en/#data/QCL)
```

Avoid:

```
[click here](https://example.org)
```

For repository files, prefer relative links so that they work in different clones:

```
[Data dictionary](../metadata/data-dictionary.csv)
```

Markdown guidelines for data management

A data-management Markdown document should usually state:

- purpose and intended audience;
 - scope and boundaries;
 - the artifacts and scripts involved;
 - the grain and role of important datasets;
 - storage and version-control decisions;
 - validation and maintenance procedures;
 - licence, sensitivity, access, and sharing considerations;
 - known limitations; and
 - links to structured metadata, dictionaries, provenance, and code.
-

Markdown limitations

Markdown is flexible, which means that software cannot reliably infer every important field from prose. For example, this sentence is readable:

The snapshot was accessed on 3 August 2026.

But a program may have difficulty extracting the date consistently. If a script must use the value, store it in a structured record such as YAML and refer to that record from the narrative.

YAML for structured records and configuration

What YAML represents

YAML represents mappings, sequences, and scalar values using indentation and plain-text syntax.

The core structures are:

- **mapping**: named key–value pairs;
 - **sequence**: an ordered list; and
 - **scalar**: a string, number, Boolean, date-like value, or null value.
-

Mapping

```
provider: Food and Agriculture Organization of the United Nations
database: FAOSTAT
dataset: Crops and Livestock Products
```

Keys should have stable, documented names. Within one mapping, every key should be unique.

Sequence

```
elements:
  - Area harvested
  - Production
  - Yield
```

Nested record

```
coverage:
  countries: 9
  first_year: 1990
  last_year: 2022
```

Indent nested content consistently with spaces. Do not use tab characters for indentation.

Literal and folded text

Use `|` when line breaks must be retained:

```
note: |
  Preserve the source file unchanged.
  Record transformations in derived scripts.
```

Use `>-` for a readable multi-line paragraph that should be loaded as one folded string without a final newline:

```
known_limitations: >-
  Historical observations may be revised, and reporting methods can differ
  among countries.
```

Quote ambiguous values

Some YAML parsers interpret unquoted values as numbers, dates, Booleans, or nulls. Quote values when their exact textual representation matters:

```
accessed: "2026-08-03"
release: "061"
country_code: "008"
answer: "yes"
```

Quoting is especially important for:

- identifiers with leading zeros;
- version numbers;
- date-like strings;
- values such as `yes`, `no`, `on`, `off`, `true`, `false`, `null`, or `~` when they are intended as text; and
- strings containing `:` or `#`.

Parser behavior can differ between YAML versions and libraries. Validate with the same parser used by the project.

Comments

```
release: "061" # MODIS product collection
```

Comments help maintainers, but programs normally discard them after parsing. Do not store information required by software only in a comment.

YAML for provenance

YAML is suitable when one artifact has nested provenance fields:

```
artifact: data-raw/faostat-maize-yield-sample.csv
provider: Food and Agriculture Organization of the United Nations
dataset: Crops and Livestock Products
accessed: "2026-08-03"
retrieval:
  script: scripts/acquire-faostat-data.R
  method: Bulk ZIP download
selection:
  countries: 9
  years: 1990-2022
  item: Maize (corn)
checksum:
  algorithm: SHA-256
  value: fd2c78cae5a5cf2f82d6b6bdc2b3637ce03b597f74e561099f9666af449605be
```

YAML for source metadata

```
provider: Food and Agriculture Organization of the United Nations
database: FAOSTAT
dataset: Crops and Livestock Products
dataset_code: QCL
frequency: Annual
```

```
classifications:
  item: Agricultural commodity
  element: Statistical measure
quality_flags:
  local_code_list: metadata/flag-code-list.csv
```

YAML guidelines for data management

- Use one documented meaning for every key.
 - Prefer descriptive keys such as `checksum_sha256` over `hash`.
 - Keep key naming consistent, for example `snake_case` throughout.
 - State whether a field is required, optional, or conditionally required.
 - Use a list for repeated values rather than a comma-separated string.
 - Keep paths repository-relative when the artifact belongs to the project.
 - Do not place passwords, tokens, private keys, or restricted URLs in tracked YAML.
 - Parse the file in an automated check; visual inspection is insufficient.
 - Consider a schema when several records must follow the same structure.
-

YAML limitations

YAML is sensitive to indentation and type interpretation. It is not ideal for large tables with hundreds of similar rows. Duplicate keys may be accepted, rejected, or silently overwritten depending on the parser. A file can parse successfully while still lacking required fields or containing scientifically invalid values.

CSV for rectangular documentation

What CSV represents

CSV represents a table as delimited text. In the usual form:

- the first row contains column names;
 - each later row represents one record;
 - commas separate fields; and
 - double quotes protect fields containing commas, quotes, or line breaks.
-

Simple example

```
flag,description,source
A,Official figure,FAOSTAT
E,Estimated value,FAOSTAT
```

Quoting

A comma inside a field requires quotes:

```
variable,definition,notes
value,Reported numeric quantity,"Interpret with element, unit, and flag."
```

A literal double quote inside a quoted field is represented by two double quotes:

```
term,definition
raw,"The project's ""raw"" artifact role"
```

CSV for a data dictionary

```
variable,label,definition,type,unit,role,allowed_values,missing_values
area,Area,Reporting geographic area,character,,label,Project countries,blank not expected
year,Year,Calendar year,integer,year,time,1990-2022,blank not expected
value,Value,Reported numeric quantity,double,varies,measure,non-negative,blank means missing
```

One row represents one variable. The header defines the documentation schema.

CSV for a code list

```
code,label,definition
A,Official figure,Value reported as an official figure
E,Estimated value,Value estimated by the provider
```

One row represents one allowed code.

CSV for a crosswalk

```
project_country_id,faostat_label,other_source_code,note
ZAF,South Africa,ZAF,Reviewed mapping
ZMB,Zambia,ZMB,Reviewed mapping
```

One row represents one mapping. State the expected key and valid direction of the mapping before using it in a join.

CSV guidelines for data management

- Use a single header row with unique, stable names.
 - Use UTF-8 encoding unless the project explicitly requires another encoding.
 - Define the delimiter, decimal mark, quote character, and missing-value representation.
 - Use one record type and one grain per table.
 - Do not use formatting, color, comments, or merged cells to communicate meaning.
 - Do not place titles or explanatory paragraphs above the header.
 - Quote fields according to the writer/parser rather than inserting quotes manually.
 - Preserve identifiers such as 008 as text when leading zeros matter.
 - Use ISO-style dates such as 2026-08-03 when a date field is required.
 - Keep row ordering stable when it has no semantic meaning but helps Git review.
 - Validate required columns, types, keys, allowed values, and missingness.
 - Store dataset-level explanation in Markdown or YAML rather than forcing it into every row.
-

CSV limitations

CSV does not carry a standard data schema. It normally does not preserve:

- data types;
- units;
- key constraints;
- categorical definitions;
- missing-value meaning;
- relationships to other tables; or
- dataset-level provenance.

These must be documented and validated separately. Spreadsheet software may also convert identifiers, dates, or large numbers automatically. Inspect the file as text and read it with explicit types when those conversions matter.

CSV conventions vary. RFC 4180 documents a widely used common format, but real files may use semicolons, tabs, different line endings, or locale-specific decimal marks. Record the actual convention rather than assuming it.

A practical checksum workflow

1. Choose the artifact

Record a repository-relative path:

```
metadata/provenance.yml
```

Be clear whether the checksum applies to:

- the original provider response;
- a normalized teaching snapshot;

- a configuration file;
- a documentation file; or
- a generated release artifact.

Each artifact has a different role and may require a different expected digest.

2. Calculate SHA-256 on Linux

```
sha256sum metadata/provenance.yml
```

Typical output:

```
<64-character-digest> metadata/provenance.yml
```

3. Calculate SHA-256 on macOS

```
shasum -a 256 metadata/provenance.yml
```

4. Calculate SHA-256 with PowerShell

```
Get-FileHash metadata/provenance.yml -Algorithm SHA256
```

5. Calculate SHA-256 in R

With the `digest` package:

```
digest::digest(  
  "metadata/provenance.yml",  
  algo = "sha256",  
  serialize = FALSE,  
  file = TRUE  
)
```

The `serialize = FALSE` and `file = TRUE` arguments ensure that the file bytes, rather than an R serialization of a character value, are hashed.

6. Record the expected digest

An artifact provenance record can contain:

```
artifact: metadata/source-metadata.yml
checksum_algorithm: SHA-256
checksum_sha256: "<64-character-digest>"
```

Avoid recording a file's checksum only inside that same file when the goal is to verify the complete file. Adding the digest changes the file and therefore changes its digest. Store the expected value in:

- a separate provenance record;
 - a checksum manifest;
 - a signed release record; or
 - validation code tied to a reviewed release.
-

7. Use a checksum manifest

Create a manifest for several stable files:

```
sha256sum \
  metadata/source-metadata.yml \
  metadata/data-dictionary.csv \
  metadata/flag-code-list.csv \
  > metadata/checksums.sha256
```

Verify it on Linux:

```
sha256sum --check metadata/checksums.sha256
```

On macOS:

```
shasum -a 256 --check metadata/checksums.sha256
```

Review the manifest diff before committing it. Do not regenerate expected digests automatically after a failed verification: first determine why the files changed and whether the change is authorized.

8. Verify in R

```
expected <- "<recorded-sha256>"

observed <- digest::digest(
  "metadata/source-metadata.yml",
  algo = "sha256",
  serialize = FALSE,
```

```

    file = TRUE
)

if (!identical(observed, expected)) {
  stop("Source metadata do not match the reviewed state.")
}

```

9. Investigate a mismatch

Do not immediately replace the expected checksum. Check:

```

git status --short
git diff -- metadata/provenance.yml
git log --oneline -- metadata/provenance.yml

```

Ask:

- Was the file intentionally edited?
- Did a formatter reorder fields or change whitespace?
- Did spreadsheet software change encoding, quoting, dates, or line endings?
- Was the artifact regenerated from a new source release?
- Is the expected checksum for a different artifact or version?
- Has an unexpected or unauthorized change occurred?

Only approve and record the new digest after reviewing the change.

Validating documentation and configuration

A checksum should be one layer in a broader validation strategy.

Four different questions

| Check | Question answered |
|----------------------|--|
| Parse check | Is the file syntactically readable in its format? |
| Schema/content check | Are required fields, types, keys, and allowed values present? |
| Checksum check | Is the file byte-identical to the reviewed state? |
| Human review | Is the meaning correct, current, justified, and fit for purpose? |

All four can be necessary.

Markdown checks

Possible checks include:

- the file is valid UTF-8;
- heading hierarchy is sensible;
- internal and external links resolve;
- fenced code blocks are closed;
- required sections exist;
- commands and examples have been tested; and
- a Markdown linter or renderer accepts the project dialect.

A Markdown checksum detects any byte change, including harmless line wrapping. Use it only when exact byte identity matters, for example for a published release document. Git review and content checks are usually more useful during normal editing.

YAML checks

At minimum, parse the file with the project parser.

In R:

```
record <- yaml::read_yaml("metadata/provenance.yml")
```

Then check required keys:

```
required <- c(
  "artifact", "provider", "dataset", "accessed",
  "retrieval", "checksum_sha256", "license"
)

missing <- setdiff(required, names(record))

if (length(missing) > 0) {
  stop("Missing provenance field(s): ", paste(missing, collapse = ", "))
}
```

Also validate:

- unique keys;
- expected data types;
- allowed values;
- date and identifier formats;
- referenced paths;
- URL or identifier structure where appropriate; and
- consistency with related records.

CSV checks

Read with an explicit table parser:

```
dictionary <- readr::read_csv(  
  "metadata/data-dictionary.csv",  
  show_col_types = FALSE  
)
```

Check the schema and key:

```
required_columns <- c(  
  "variable", "definition", "type", "unit", "role"  
)  
  
stopifnot(all(required_columns %in% names(dictionary)))  
stopifnot(!anyDuplicated(dictionary$variable))  
stopifnot(!any(is.na(dictionary$variable)))  
stopifnot(!any(is.na(dictionary$definition)))
```

Also inspect:

- unexpected extra columns;
 - blank headers;
 - encoding and delimiter;
 - inferred versus intended types;
 - duplicated rows or keys;
 - missing required fields;
 - allowed categories;
 - references to code lists; and
 - consistency with the actual data file.
-

Byte identity versus semantic equivalence

These two YAML documents can represent equivalent mappings:

```
provider: FAO  
dataset: FAOSTAT
```

```
dataset: FAOSTAT  
provider: FAO
```

Their raw bytes and SHA-256 digests differ. Similarly, CSV files can differ in line endings or quotation while a parser returns the same table.

Therefore:

- use a checksum when exact file identity matters;
- use parsing and content checks when meaning matters;
- use both when a reviewed release requires exact identity and valid content.

Canonicalizing a file before hashing is possible, but it creates another specification: the project must define the parser, type rules, sorting, encoding, whitespace, and serialization method. For introductory projects, hashing the original bytes and validating content separately is clearer.

Application to the maize-yield project

The project can use the formats as follows:

| Artifact | Format | Reason |
|------------------------------|-------------------|---|
| README.md | Markdown | Project purpose, workflow, commands, and interpretation |
| docs/data-management.md | Markdown | Repository-specific implementation guidance |
| metadata/source-metadata.yml | YAML | Nested provider, methodology, scope, classification, and quality record |
| metadata/provenance.yml | YAML | Structured identity and history of the fixed teaching artifact |
| metadata/data-dictionary.csv | CSV | One row per field in the teaching extract |
| metadata/flag-code-list.csv | CSV | One row per provider quality flag |
| metadata/checksums.sha256 | Checksum manifest | Expected byte identity of selected stable artifacts, if adopted |

Example workflow

```
Read source metadata
  ↓
Understand provider concepts, methods, units, flags, and limitations
  ↓
Inspect the project data dictionary and code lists
  ↓
Confirm the exact teaching artifact through provenance and checksum
  ↓
Parse and validate YAML and CSV structure
  ↓
Review meaning, licence, sensitivity, and fitness for purpose
```

Appropriate checksum targets

Checksums are most useful for:

- a fixed raw teaching snapshot;

- an instructor-approved metadata release;
- a configuration file deployed to a service;
- a checksum manifest distributed with course materials; and
- an exported or archived release artifact.

Checksums are less useful as a replacement for Git review on actively edited Markdown. Every spelling correction changes the digest, but the digest does not explain the change.

Suggested validation responsibilities

| File | Automated checks | Human review |
|-------------------------------|---|--|
| Markdown implementation guide | Required headings, links, render/lint | Accuracy, clarity, current workflow |
| YAML source metadata | Parse, required keys, allowed structure | Provider meaning, methods, references |
| YAML provenance | Parse, required keys, checksum format, referenced artifact | Retrieval history, licence, sharing decision |
| CSV dictionary | Header, unique variable key, non-empty definitions, allowed roles | Field meaning, units, missingness |
| CSV code list | Header, unique code key, referenced codes covered | Provider definitions and qualifications |
| Teaching data snapshot | Checksum, schema, grain, key, ranges | Fitness for purpose and limitations |

Common mistakes

Choosing by file extension rather than structure

A large YAML list of hundreds of identical variable records is harder to work with than CSV. A CSV containing paragraphs and nested lists is harder to understand than Markdown or YAML.

Treating Markdown as structured configuration

A program should not have to search prose for the access date or checksum. Store required fields in a structured record and explain them in Markdown.

Treating YAML as free-form prose

Very long narrative values make structured fields hard to review. Keep concise summaries in YAML and link to Markdown or authoritative documentation.

Treating CSV as a spreadsheet layout

Do not add blank rows, merged headings, colors, formulas, or multiple tables to a CSV. Those features are either not represented or are interpreted inconsistently.

Relying on automatic types

Identifiers can lose leading zeros, and date-like or Boolean-like YAML values can be converted unexpectedly. Define types and quote ambiguous YAML scalars.

Accepting a parse as validation

A YAML file containing `provider: unknown` can parse successfully. A CSV with duplicate variable names can also parse successfully. Syntax is only the first validation layer.

Updating a checksum without reviewing the change

A mismatch is a signal to investigate. Automatically replacing the expected digest removes the control the checksum was meant to provide.

Storing the only expected digest with the untrusted artifact

If both can be replaced together, a matching digest does not establish authenticity. Use a trusted manifest, reviewed Git history, signature, or protected release channel.

Hashing a logical value instead of file bytes

Programming libraries may hash a serialized object by default. When validating a file, confirm that the function reads and hashes the file's bytes.

Assuming a checksum proves correctness

A perfectly preserved file can contain incorrect values or inappropriate methodology. Combine identity checks with metadata, dictionaries, provenance, validation, and scientific review.

Committing secrets in YAML

Configuration syntax makes credentials look like ordinary fields. Keep secrets outside tracked files, supply them through an approved secret mechanism, and rotate any credential exposed in Git history.

Completion checklist

Format choice

- ☐ Narrative guidance is stored in Markdown.
 - ☐ Nested records and configuration are stored in YAML when appropriate.
 - ☐ Repeated rectangular records are stored in CSV.
 - ☐ Each file has one documented purpose and grain.
-

Markdown

- ☐ The heading hierarchy is logical.
 - ☐ Lists, tables, links, and code blocks are used consistently.
 - ☐ Repository links are relative where appropriate.
 - ☐ Commands and examples have been checked.
 - ☐ Structured values required by software are not hidden only in prose.
-

YAML

- ☐ Indentation uses spaces rather than tabs.
 - ☐ Keys are unique, stable, and consistently named.
 - ☐ Ambiguous strings and identifiers are quoted.
 - ☐ Repeated values use sequences rather than comma-separated strings.
 - ☐ The file parses with the project's YAML parser.
 - ☐ Required keys, types, values, and references are validated.
 - ☐ No secret or restricted value is committed.
-

CSV

- ☐ There is exactly one header row with unique field names.
 - ☐ Every row has the same grain and record type.
 - ☐ Encoding, delimiter, quotation, decimal, and missing-value conventions are known.
 - ☐ Candidate keys are tested for uniqueness.
 - ☐ Required fields and allowed values are validated.
 - ☐ Dataset-level narrative and provenance are stored elsewhere.
-

Checksums

- ☐ SHA-256 is named explicitly as the algorithm.
 - ☐ The intended artifact path and version are clear.
 - ☐ The expected digest is stored through a trusted, reviewed mechanism.
 - ☐ Verification hashes file bytes rather than a serialized program object.
 - ☐ A mismatch stops the workflow and triggers investigation.
 - ☐ Checksums are supplemented by syntax, schema, content, and human review.
 - ☐ New digests are recorded only after intentional changes are reviewed.
-

Check your understanding

1. Why is Markdown usually better than YAML for a long implementation guide?
 2. Why is CSV usually better than Markdown for a 200-variable data dictionary?
 3. Give an example of information that belongs in YAML provenance and not only in Markdown prose.
 4. Why should an identifier such as 008 be treated carefully in YAML and spreadsheet software?
 5. What does a matching SHA-256 checksum establish?
 6. Name four important properties that a checksum does not establish.
 7. Why should a failed checksum not be “fixed” immediately by generating a new expected value?
 8. How can two semantically equivalent YAML documents have different checksums?
 9. Why is storing a checksum beside a file insufficient against an attacker who can replace both?
 10. Which checks would you apply to `metadata/data-dictionary.csv` in addition to a checksum?
-

Further resources

File formats

- [CommonMark specification](#) defines a consistent core Markdown syntax.
 - [CommonMark quick reference](#) provides a compact introduction to Markdown elements.
 - [YAML 1.2.2 specification](#) defines YAML’s data model and syntax.
 - [RFC 4180](#) documents a common CSV format and the `text/csv` media type. Real-world CSV files can still follow different conventions, so projects must record what they use.
-

Checksums

- [NIST FIPS 180-4: Secure Hash Standard](#) specifies SHA-256 and related secure hash algorithms.
 - The R [digest](#) package can calculate file digests programmatically.
-

Data-management context

- [The Turing Way: Research Data Management](#) connects documentation, storage, sharing, preservation, and reproducibility.
- [Metadata, data dictionaries, and provenance](#) provides a detailed conceptual comparison and applies it to the maize-yield project.

5.1) Why integrate data?

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Food-systems questions need several sources
 - Acquisition supports integration
 - Integration is more than joining tables
 - Common integration dimensions
 - Identifiers
 - Time
 - Space
 - Units
 - Schema and classification
 - Grain
 - A reproducible integration workflow
 - What can go wrong
 - The external source changes
 - The join uses names
 - The key is incomplete
 - Unmatched records disappear
 - Aggregation changes meaning
 - Credentials leak
 - How this connects to the maize-yield project
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- explain why a managed dataset may still be insufficient for a question;
 - distinguish data management, integration, and preparation;
 - identify identifier, temporal, spatial, unit, and schema mismatches;
 - describe the main stages of a reproducible integration workflow; and
 - explain why every join requires an audit.
-

Place in the session

This is the **Motivation** part of the Data Integration session:

Motivation → Concepts → Application
↑
this page

The preceding Data Management session established what the maize data mean, where they came from, and how their structure is validated. This session asks how to extend that managed dataset with another source without hiding incompatible identifiers, grains, periods, units, or definitions.

[Understand data-integration concepts](#) develops the mental model. [Integrate maize-yield and precipitation data](#) applies it to FAOSTAT and CHIRPS.

Food-systems questions need several sources

Changes in maize yield may be related to many processes:

- harvested area and production;
- rainfall and temperature;
- soils and topography;
- fertilizer and irrigation;
- market access and prices;
- policies, conflict, and infrastructure.

No single source contains every relevant concept at the same geographic and temporal resolution.

```
FAOSTAT -----\
                  \
Annual climate summaries -----> country-year dataset
                  /
Country identifiers -----/
```

Combining sources enriches an analysis but introduces choices: a result can shift with the provider, query, release, crosswalk, time aggregation, spatial boundary, join type, or unit conversion selected.

Acquisition supports integration

Acquisition belongs to the data lifecycle introduced under Data Management. It recurs here when CHIRPS is added, in support of the main integration problem: obtaining an additional source in a form that can be aligned with existing evidence.

Acquisition determines:

- which provider and dataset are used;
- which variables, countries, years, and quality statuses are requested;
- which version or release enters the analysis;
- which observations are excluded by the query;
- whether the response can be retrieved again;
- whether licensing permits storage and redistribution.

An acquisition script should make these choices visible; a manual download can be documented just as reproducibly if every selection and interaction is recorded.

Integration is more than joining tables

A join combines records that share keys. Scientific integration also asks whether those records refer to compatible concepts.

Two tables may both contain **country** and **year** while differing in:

- geographic definitions;
- calendar versus growing years;
- annual values versus partial-year observations;
- official versus modeled estimates;
- current versus historical borders;
- nominal versus real monetary values;
- tonnes per hectare versus kilograms per hectare.

The software may complete the join without warning. Technical success is not evidence of conceptual compatibility.

Common integration dimensions

Identifiers

Different providers use different codes and labels. A documented crosswalk is usually safer than joining by names.

Time

Daily, monthly, seasonal, and annual values require explicit alignment. Aggregating daily rainfall to a calendar year is not necessarily appropriate for a crop growing season.

Space

Points, administrative polygons, and grid cells represent geography differently. Boundaries, coordinate reference systems, extent, and resolution affect results.

Units

Values can be compared only when definitions and units are compatible. Unit conversion must be explicit and tested.

Schema and classification

Column types, categories, quality flags, and commodity classifications may differ or change over time.

Grain

Both sources must have a known observational grain. Joining a country-year table to a country-year-month table will multiply rows unless the relationship is handled deliberately.

A reproducible integration workflow

```
Define the question and intended final grain
  ↓
Identify and evaluate an additional source
  ↓
Record metadata, query, version, license, and access date
  ↓
Acquire or select a preserved source snapshot
  ↓
Validate each input schema and key
  ↓
Align identifiers, periods, space, units, and classifications
  ↓
Join or aggregate with explicit expectations
  ↓
Audit matches, row counts, duplicates, and missingness
  ↓
Save the integrated result and its lineage
```

The workflow should stop visibly when a critical expectation fails. A plausible-looking result is dangerous when the pipeline silently discarded or multiplied observations.

What can go wrong

The external source changes

An endpoint, schema, historical value, classification, or license can change. Record versions, access dates, and checksums, and preserve permitted responses or teaching snapshots.

The join uses names

Congo, Congo, Rep., and Republic of the Congo may refer to the same entity. Similar names may also refer to different entities. Use stable codes and a reviewed crosswalk.

The key is incomplete

A many-to-many join can multiply records unexpectedly. State and test each source key before integration.

Unmatched records disappear

An inner join can silently remove countries or years. Inspect unmatched keys before choosing how to proceed.

Aggregation changes meaning

Annual or country-level summaries hide variation and depend on weighting and coverage rules. Document those rules and preserve source quality information.

Credentials leak

API tokens or passwords embedded in scripts can enter Git history. Store secrets outside the repository and document how users supply them.

How this connects to the maize-yield project

The exercise begins with the managed FAOSTAT teaching data and combines:

1. a prepared FAOSTAT panel containing maize production, harvested area, and yield; and
2. CHIRPS precipitation aggregated over country polygons for October–April seasons.

This concrete extension exposes realistic issues:

- provider labels versus stable project identifiers;
- gridded daily estimates versus annual country statistics;
- country polygons and spatial aggregation;
- growing seasons versus reporting years; and
- inherited metadata, provenance, uncertainty, and limitations.

The output is a derived country-year dataset plus a join audit — an input for association analysis, not evidence that precipitation causes maize-yield differences.

Check your understanding

1. Why does the managed FAOSTAT dataset not answer every food-systems question?
 2. Give an example of a join that is technically successful but scientifically invalid.
 3. What should be checked before and after every join?
 4. Why is a country-code crosswalk itself a dataset that requires documentation?
 5. What is the difference between integration and data preparation?
 6. Why can students use fixed snapshots even though acquisition remains reproducible?
-

Further resources

- [Research Data Management — The Turing Way](#) connects acquisition, documentation, storage, sharing, and preservation across the research-data lifecycle.
 - [HTTP overview — MDN Web Docs](#) explains the request–response model used by web APIs.
 - [Organising data — UK Data Service](#) places meaningful filenames, formats, and folder structures within practical research-data management.
 - Wickham, H., Çetinkaya-Rundel, M., and Grolemund, G., [R for Data Science \(2e\): Joins](#), explains mutating joins, filtering joins, keys, and common relationship problems in `dplyr`.
 - [World Bank Indicators API documentation](#) provides a realistic public API for practising parameterized acquisition and pagination.
-

Continue to Concepts

Continue with [Understand data-integration concepts](#), which covers evaluating a source, aligning grains and identifiers, stating join relationships, and auditing and preserving lineage.

5.2) Understand data-integration concepts

Outline

- Outline
- Learning objectives
- Place in the session
- Data management, acquisition, integration, and preparation
- Begin with a data requirement
- Evaluate a source
- Access methods
 - Downloadable files
 - APIs
 - Databases
 - Remote data services
- Tabular and semi-structured formats
 - CSV and TSV
 - Spreadsheets
 - JSON
 - Parquet
- Spatial and temporal data
 - Vector data
 - Raster data
 - Temporal data
- Grain, keys, and join relationships
- Identifiers and crosswalks
- Alignment across sources
- Join choice and integration audits
- Lineage
- Plan for changing services
- Credentials and responsible access
- Source and acquisition record
- Check your understanding
- Further resources
 - Acquisition and responsible access
 - Integration and joins
 - Spatial and temporal integration
- Continue to Application

Learning objectives

After completing this page, you should be able to:

- translate an information gap into a requirement for an additional source;
- evaluate authority, coverage, documentation, quality, license, and reproducibility;
- compare file, API, database, and remote-service access;

- recognize common tabular, JSON, relational, vector, raster, and temporal representations;
- state source grains, keys, and expected join relationships;
- explain why crosswalks and temporal, spatial, unit, and classification alignment are analytical artifacts;
- choose a join from the intended population and audit its result;
- anticipate service changes and credential risks; and
- create a complete source and acquisition record without confusing it with an integration audit.

Place in the session

This is the **Concepts** part of the Data Integration session:

```
Motivation → Concepts → Application
              ↑
            this page
```

[Why integrate data?](#) establishes why source and alignment decisions affect scientific results. This page gives you the vocabulary and decision model required by [the maize and precipitation integration application](#).

Use the concepts as questions:

- What evidence does the project require?
- Why is this source suitable, and can the request be reconstructed?
- What does one row in each source represent?
- Which identifiers, periods, spaces, units, and classifications must align?
- What row-count and matching behavior should the join produce?
- Can every output variable be traced back to an input and operation?

Data management, acquisition, integration, and preparation

These activities overlap, but emphasize different questions:

| Activity | Central question | Typical output |
|-----------------|--|--|
| Data management | How are data made understandable, trustworthy, organized, and auditable? | Managed source and derived artifacts |
| Acquisition | How was additional evidence obtained and identified? | Preserved response or fixed source snapshot |
| Integration | How were compatible observations connected across sources? | Audited multi-source interim dataset |
| Preparation | How were values adapted for a particular analysis? | Analysis-ready dataset and derived variables |

Data management applies throughout the lifecycle; acquisition retrieves evidence, integration connects sources, and preparation handles choices such as recoding, filtering, and feature construction. A single script may perform more than one activity, but its decisions should stay distinguishable.

Begin with a data requirement

Do not begin by downloading every available file. First write what the project needs, for example:

```
Concept: annual maize yield
Entities: selected Southern African countries
Period: 1990–2022
Frequency: annual
Geographic level: country
Measurement: provider-defined yield and unit
Quality information: retain provider flags
Required metadata: definitions, methods, codes, license, release
```

A precise requirement helps evaluate source suitability and prevents an acquisition script from becoming an undocumented collection of filters. Record whether each condition is essential or preferred.

Evaluate a source

| Dimension | Questions to ask |
|------------------------------|--|
| Authority and origin | Who created or publishes the data? Is this the original provider or a republished copy? Is the collection method documented? |
| Meaning and scope | Does the provider measure the intended concept? Which populations, geographies, commodities, and periods are covered? Are values observed, reported, estimated, imputed, or modeled? |
| Quality and revisions | Are quality flags or uncertainty measures available? Are historical values revised? Is there a release schedule or version identifier? |
| Access and reproducibility | Is a stable file, API, or query interface available? Can the same subset be requested again? Are authentication, rate limits, cost, or availability constraints documented? |
| Legal and ethical conditions | What license and citation apply? May raw and derived data be redistributed? Are there privacy, confidentiality, or location-sensitivity risks? |

The most convenient source is not necessarily the most authoritative or reproducible one.

Access methods

Downloadable files

A published file is a source snapshot. Record its exact URL or delivery method, access date, release, filename, size, and checksum. Files are simple to inspect, preserve, and use offline, but providers may replace them at the same URL and spreadsheet exports may mix data with presentation.

APIs

An application programming interface accepts a request and returns a structured response. A reproducible request records the endpoint, parameters, pagination, API version, response format, authentication, and request date. Always validate the response — a success status does not prove the expected schema or complete result was returned.

Databases

A database query selects rows and columns from related tables: a **primary key** identifies a record in its table, a **foreign key** refers to a record in another table, and results can change after an update because database state matters. Record the database version, connection target, SQL query, and parameters; use read-only access for learning exercises and never embed passwords in the script. Full database design, administration, and optimization are outside this introductory session.

Remote data services

Large climate and Earth-observation platforms often filter, aggregate, or compute near the hosted data. Record the collection identifier and version, spatial/temporal filters, quality masks, projection, aggregation rule, and export date — the exported table is derived data, not an untouched raw observation. Continuous sensor and stream sources carry similar metadata (device, location, calibration, timestamp, frequency) but sit outside this course’s core exercise.

Tabular and semi-structured formats

CSV and TSV

Delimited text is portable but usually does not preserve data types, definitions, units, missing-value rules, or relationships between tables. Record the delimiter, encoding, decimal convention, date representation, and missing codes.

Spreadsheets

Spreadsheets can contain multiple sheets, formulas, merged cells, notes, and several tables. Identify the sheet and cell range used, and avoid treating color or layout as machine-readable data.

JSON

JSON can represent nested records and is common in APIs, for example a **country** object nested inside a **year/indicator** record. Converting nested JSON to a table requires choices about which objects become rows and how nested fields are expanded. Record those choices.

Parquet

Parquet preserves types and supports efficient analytical access for larger derived tables, but it should not replace source metadata or provenance.

Spatial and temporal data

Vector data

Vector data represent points, lines, or polygons. Common concerns include stable geographic identifiers, coordinate reference system, boundary vintage, invalid geometry, and geographic versus projected coordinates.

Raster data

Raster data represent cells in a grid. Record the variable and unit, cell resolution, extent, coordinate reference system, time interval, missing or masked cells, and processing level. Summarizing raster cells to a country requires decisions about boundaries, weighting, and coverage.

Temporal data

Record whether a value refers to an instant or interval, local time or UTC, day, month, season, calendar year, or growing year, and a complete or partial period. Never infer a time zone or reporting period only from the display format.

Grain, keys, and join relationships

State the grain of every input before joining. For example:

```
FAOSTAT table: one row per country-year-item-element-unit
CHIRPS table: one row per country-October-April season
Target table: one row per country-year
```

The input tables must first be expressed at grains compatible with the target. A join relationship describes how many rows on one side can match a key on the other:

| Relationship | Meaning | Expected row behavior |
|--------------|---|---|
| One-to-one | At most one matching row on each side | Keys should not multiply |
| One-to-many | One left row can match several right rows | Left observations can expand |
| Many-to-one | Several left rows can match one right row | Right values can repeat legitimately |
| Many-to-many | Several rows can match on both sides | Often produces multiplication and requires explicit justification |

The relationship is a property of the data at the stated grain, not an argument supplied to software. Test key uniqueness on both sides before joining — an unexpected many-to-many match usually signals an incomplete key or incompatible grain.

Identifiers and crosswalks

Provider labels are designed for display and can differ in spelling, language, punctuation, abbreviation, or historical meaning. Prefer stable source codes when available.

A **crosswalk** maps source-specific identifiers to a reviewed project identifier:

```
project_country_id,project_country_name,faostat_area_label,valid_from,valid_to,note
ZAF,South Africa,South Africa,1990,2022,Reviewed country mapping
```

Treat the crosswalk as data: retain both identifiers and readable labels, test uniqueness in the join direction, record validity periods and boundary changes, and review fuzzy matches manually rather than guessing.

Alignment across sources

Matching identifiers is only one part of integration:

| Dimension | Question to resolve |
|----------------|--|
| Concept | Do the variables measure compatible phenomena? |
| Grain | Do rows represent compatible observational units? |
| Time | Calendar year, growing season, interval, instant, or partial period? |
| Space | Which boundary vintage, resolution, CRS, and aggregation rule apply? |
| Unit | Are definitions and dimensions compatible before conversion? |
| Classification | Do commodity, land-use, or status categories correspond? |
| Quality | Which flags, uncertainty measures, or source statuses must remain visible? |

Alignment can require aggregation, conversion, or mapping; each operation can discard information or change interpretation, so record the rule and retain what's needed to audit it.

Join choice and integration audits

Choose a join from the intended study population:

- `left_join()` retains every key from the designated primary source;
- `inner_join()` retains only keys observed in both sources;
- `full_join()` retains the union and is useful for diagnosing overlap;
- `anti_join()` reports keys present on only one side.

No join is universally safest. An inner join may silently remove countries or years; a left join may introduce missing complementary values; a full join may include records outside the target population.

Before joining, record the expected relationship, unmatched keys, and row-count effect. Afterwards, audit row counts, unmatched keys in both directions, duplicate output keys, missingness, and coverage, and confirm the output still has the intended grain. Do not suppress an unexpected relationship warning merely to obtain output.

Lineage

Lineage connects each output variable to its origin and transformations. A compact lineage table can record:

| Output variable | Source artifact | Source field | Operation | Unit |
|------------------------------|------------------|-----------------------------|---|------|
| yield_tonnes_per_ha | FAO STAT extract | value where element = Yield | Reshape and convert from kg/ha | t/ha |
| growing_season_precipitation | CHIRPS snapshot | Daily spatial averages | Sum October–April and join by country and ending year | mm |

Also record input checksums, acquisition and integration scripts, crosswalk version, conversion rules, output checksum, and known information loss — provenance identifies the inputs, lineage explains how they became the integrated artifact.

Plan for changing services

External sources can change their endpoint and authentication, schema and variable names, classifications and geographic codes, historical values, rate limits and availability, and license and access rules.

Make a workflow more resilient: request a documented version, record the access date and complete query, preserve the raw response when permitted, validate schema and coverage immediately, and provide a fixed teaching snapshot with refresh instructions. Do not automatically fall back to a different provider — that changes the evidence and requires a scientific decision.

Credentials and responsible access

Never commit API keys, tokens, passwords, or certificates. Read secrets from environment variables or an ignored local config file, request only the permissions required, and keep them out of logs and error reports. Revoke and rotate any exposed credential — a deleted secret may remain in Git history, so prevention is safer than cleanup.

Source and acquisition record

For each source, record:

```
source_id: faostat_maize
provider: "provider name"
dataset: "exact dataset or product"
version: "release or product version"
accessed: "YYYY-MM-DD"
access_method: "file, API, database, or remote service"
location: "URL, endpoint, database, or collection ID"
parameters: "countries, years, variables, filters"
license: "license or terms"
raw_artifact: "project path"
checksum_sha256: "checksum when applicable"
script: "acquisition script"
notes: "limitations and fallback snapshot"
```

The record should allow another person to understand exactly what was requested, even if the service later changes.

Check your understanding

1. Why should a data requirement be written before selecting a source?
 2. What must be recorded to reproduce an API request?
 3. Which information can CSV fail to preserve?
 4. How should credentials be supplied to a script?
 5. What distinguishes a one-to-many join from an accidental many-to-many join?
 6. Why should unmatched keys be inspected from both directions?
-

Further resources

Acquisition and responsible access

- [Research Data Management — The Turing Way](#) connects acquisition, documentation, storage, sharing, and preservation.
- [HTTP overview — MDN Web Docs](#) explains the request–response model underlying web APIs.
- [World Bank Indicators API documentation](#) documents a public indicator service suitable for practising reproducible requests.

Integration and joins

- Wickham, H., Çetinkaya-Rundel, M., and Grolemund, G., [R for Data Science \(2e\): Joins](#), covers keys, mutating joins, filtering joins, and relationship diagnostics.
- [Mutating joins — dplyr reference](#) documents join arguments, unmatched rows, and relationship checks.
- [Data Wrangling with dplyr — Data Carpentry](#) provides a practical introduction to relational operations in R.

Spatial and temporal integration

- [Geocomputation with R](#) introduces vector, raster, coordinate-reference, and spatial aggregation concepts.
 - [CF Conventions](#) document metadata conventions widely used for climate and forecast data in NetCDF.
-

Continue to Application

Continue with [Integrate maize-yield and precipitation data](#). The application turns managed FAOSTAT and CHIRPS inputs, grain and key expectations, a reviewed crosswalk, explicit joins, audit tables, and lineage into inspectable project artifacts.

5.3) Integrate maize-yield and precipitation data

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverable
 - 1. Identify the information gap
 - Why CHIRPS is a reasonable choice
 - 2. Understand and validate each source
 - FAOSTAT maize data
 - CHIRPS precipitation data
 - 3. Define the integration contract
 - 4. Align identifiers, space, and time
 - Country identifiers
 - Spatial support
 - Temporal support
 - 5. Integrate and audit the sources
 - 6. Document and interpret the result
 - Extending the integration
 - Troubleshooting
 - The maize panel is missing
 - The CHIRPS checksum fails
 - A country label is unmatched
 - Rows multiply
 - Years appear shifted
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- explain why precipitation is added to the managed maize dataset;
 - state the grain and candidate key of both sources and the result;
 - distinguish source acquisition from the central integration decisions;
 - explain the country, spatial, and temporal alignment;
 - state a join relationship and predict its effect on row count;
 - audit keys, coverage, duplicates, matches, and missing values; and
 - trace integrated variables to their sources and transformations.
-

Place in the session

This is the **Application** part of the Data Integration session:

```
Motivation → Concepts → Application
                        ↑
                    this page
```

The preceding Data Management topic made the fixed FAOSTAT input understandable and auditable. This application applies the same practices again while extending the project with CHIRPS precipitation.

Review [Why integrate data?](#) and [Understand data-integration concepts](#) before beginning.

Scenario and deliverable

FAOSTAT describes annual maize yield, production, and harvested area, but it does not describe environmental conditions associated with each country-year. The project adds CHIRPS precipitation as a plausible environmental variable.

The tracked inputs are:

```
data/input/faostat-maize-yield-sample.csv
data/input/chirps-growing-season-precipitation.csv
```

The workflow produces:

```
data/derived/maize-yield-panel.csv
data/derived/maize-yield-with-precipitation.csv
results/tables/data-integration-audit.csv
```

The result is an analysis input, not evidence that precipitation causes differences in maize yield.

1. Identify the information gap

Begin with the analytical need:

We want to explore whether wetter or drier growing seasons coincide with differences in reported country-level maize yield.

Record:

- the concept needed: growing-season precipitation;
- the existing analytical unit: country-year;
- desired coverage: nine project countries, 1990–2022;
- expected unit: millimetres;
- acceptable spatial and temporal approximations; and
- limitations that would make a source unsuitable.

CHIRPS provides long-running daily gridded precipitation estimates. Its suitability still depends on how those estimates are summarized and aligned.

Why CHIRPS is a reasonable choice

Applying the evaluation questions from the concepts page:

- **Authority:** CHIRPS is produced by the Climate Hazards Center (UC Santa Barbara) with USGS, a documented, non-commercial climate data provider.
- **Meaning and scope:** it estimates rainfall, not soil moisture or crop water stress — a proxy for growing-season moisture, not a direct yield driver.
- **Quality:** it blends satellite infrared estimates with sparse station observations, so accuracy varies with local station density.
- **Access:** available as a stable gridded product through ClimateSERV and direct download, supporting repeated, versioned requests.
- **License:** public domain with a citation requirement.

A station-based rainfall network or a reanalysis product such as ERA5 could serve the same role, with different coverage, resolution, and access trade-offs. The choice of source is a recorded scientific decision, not a default.

Acquisition supports this step, but is not the central learning objective. The normal student workflow uses fixed, checksummed snapshots and works offline. Maintainers deliberately refresh them using the acquisition scripts.

2. Understand and validate each source

FAOSTAT maize data

The tracked FAOSTAT input has this grain:

One row represents one maize element for one reporting area, calendar year, and unit.

scripts/prepare-maize-data.R validates element-unit combinations, reshapes the values, converts yield from kg/ha to t/ha, and creates one row per country and year.

Before trusting the input, inspect it directly:

```
maize_raw <- readr::read_csv("data-raw/faostat-maize-yield-sample.csv")
dplyr::count(maize_raw, area, element, unit)
```

This surfaces which country, element, and unit combinations are actually present before any reshaping happens — a mismatch here explains a downstream validation failure more directly than reading the validation report alone.

CHIRPS precipitation data

The tracked CHIRPS input has this grain:

One row represents one project country and one October–April season.

Its candidate key is:

```
project_country_id + year
```

Validate that it contains required columns, nine known project identifiers, all years from 1990 through 2022, unique keys, non-negative precipitation, and 212 or 213 daily observations per season.

The source descriptions, dictionaries, provenance, and human-readable dataset pages should be reviewed before integration. The integration inherits the limitations of all inputs.

A parallel check confirms the expected season coverage:

```
chirps_raw <- readr::read_csv("data-raw/chirps-growing-season-precipitation.csv")
dplyr::count(chirps_raw, project_country_id) |> dplyr::filter(n != 33)
```

An empty result confirms that every country has all 33 seasons (1990 through 2022).

3. Define the integration contract

The target grain is:

One row represents one project country and year, containing maize measures and precipitation for the October–April season ending in that year.

The target key is project_country_id + year.

| Property | Expectation |
|--------------------------------|-------------------------------------|
| Left table | Prepared FAOSTAT maize panel |
| Right table | CHIRPS growing-season precipitation |
| Join key | Project country identifier and year |
| Relationship | One-to-one |
| Join type | Left join |
| Expected left rows | 297 |
| Expected output rows | 297 |
| Expected unmatched maize keys | 0 |
| Expected duplicate output keys | 0 |

This contract turns assumptions about the join into testable conditions. A join function cannot determine whether the contract is scientifically valid.

4. Align identifiers, space, and time

Country identifiers

The FAOSTAT input contains provider labels; CHIRPS uses stable project identifiers derived from Natural Earth ADM0_A3 values. The reviewed mapping is stored in:

```
metadata/project-country-crosswalk.csv
```

It maps provider labels to project identifiers and records validity periods and naming notes. A crosswalk is a governed dataset, not an invisible string correction in code.

Spatial support

CHIRPS begins as daily gridded precipitation. ClimateSERV calculates the spatial average across each polygon in `metadata/project-country-boundaries.geojson`. These generalized Natural Earth country polygons are not precise maize-growing areas.

Temporal support

The acquisition workflow sums daily spatial averages from 1 October through 30 April. A season is assigned to the year in which it ends:

```
year = 2022
season_start_date = 2021-10-01
season_end_date = 2022-04-30
```

The result, `growing_season_precipitation_mm`, is a seasonal sum of country-area daily averages. It is not a station measurement or a maize-area-weighted estimate.

5. Integrate and audit the sources

From the example-project root, run:

```
Rscript scripts/prepare-maize-data.R
Rscript scripts/integrate-data.R
```

The integration script:

1. verifies the tracked CHIRPS checksum;
2. validates its schema, keys, values, and coverage;
3. maps FAOSTAT labels through the country crosswalk;
4. checks uniqueness in both sources;
5. performs a declared one-to-one left join; and
6. refuses to write output if row count or key uniqueness changes.

A simplified join is:

```
integrated <- maize_with_id |>
  dplyr::left_join(
    precipitation,
    by = c("project_country_id", "year"),
    relationship = "one-to-one"
  )
```

Inspect results/tables/data-integration-audit.csv. It records source and output rows, duplicate output keys, unmatched keys, and missing precipitation. During development, use anti-joins to inspect unmatched keys directly. Never treat an empty or non-empty anti-join as self-explanatory.

A passing audit looks like:

| Check | Left rows | Right rows | Output rows | Unmatched left | Unmatched right | Duplicate keys |
|-----------------------|-----------|------------|-------------|----------------|-----------------|----------------|
| maize + precipitation | 297 | 297 | 297 | 0 | 0 | 0 |

If unmatched left keys appear, the most likely cause is a country missing from the crosswalk or absent from a CHIRPS season. If the output row count exceeds 297, the relationship is not truly one-to-one, and the script should stop rather than silently multiply rows.

6. Document and interpret the result

The new artifact needs its own documentation:

- human-readable description: docs/data/maize-yield-with-precipitation.md;
- column definitions: metadata/maize-yield-with-precipitation-data-dictionary.csv;
- transformation lineage in metadata/provenance.yml;
- source descriptions in metadata/source-metadata.yml; and
- executable transformation in scripts/integrate-data.R.

For example, the data dictionary should describe `growing_season_precipitation_mm` with its unit, definition, and season boundary, not only its column name:

```
variable,label,definition,type,unit
growing_season_precipitation_mm,Growing-season precipitation,Summed October-April daily spatial-average
```

A reader who sees only the derived table cannot recover the season boundary or the aggregation method from the column name alone; the dictionary and provenance record carry that meaning forward.

The table can describe associations between national maize statistics and country-area precipitation. It cannot establish causation. Rainfall timing, irrigation, heat, soils, varieties, pests, inputs, management, reporting differences, and subnational production patterns are omitted. Both insufficient and excessive rainfall can reduce yield.

Extending the integration

The same workflow generalizes to other environmental variables. Adding a second source — for example mean growing-season temperature — repeats the same sequence: define the requirement, evaluate the source, define a new integration contract with its own expected relationship, align identifiers and time, and add a second audit row. It does not require a new integration pattern.

A more demanding extension would use subnational or maize-growing-area weighted precipitation rather than a country-polygon average. That would change the meaning of `growing_season_precipitation_mm` from a coarse country-wide proxy to an estimate more directly tied to where maize is actually grown, at the cost of a more complex spatial aggregation step and a new source to evaluate and document.

Troubleshooting

The maize panel is missing

Run `scripts/prepare-maize-data.R` before integration.

The CHIRPS checksum fails

Do not bypass the check. Compare the file with its record in `metadata/provenance.yml`.

A country label is unmatched

Review `metadata/project-country-crosswalk.csv`. Do not apply an ad hoc string replacement.

Rows multiply

Stop and inspect both keys and the declared relationship. Do not remove duplicates until their meaning is understood.

Years appear shifted

Review the October–April definition. The join uses the year in which the season ends.

Completion checklist

- ☐ The information gap and reason for adding precipitation are stated.
 - ☐ Both source grains and candidate keys are tested.
 - ☐ Country identifiers use the reviewed crosswalk.
 - ☐ Spatial and temporal aggregation decisions are explicit.
 - ☐ The expected join relationship and row count are stated.
 - ☐ Duplicate, unmatched, and missing records are audited.
 - ☐ The integrated artifact has a dictionary, lineage, and readable documentation.
 - ☐ Scientific limitations are reported without claiming causation.
 - ☐ Fixed source snapshots remain unchanged.
-

Reflect on the application

1. Why is precipitation a reasonable addition to the maize data?
 2. What does one CHIRPS row represent after aggregation?
 3. Why is the project identifier preferable to country-name matching?
 4. Which alignment decision most affects scientific meaning?
 5. What would change if rainfall were weighted to maize-growing areas?
 6. Does a complete one-to-one join establish conceptual compatibility?
 7. Which data-management practices are repeated after integration?
 8. Which additional source would you consider, and what new mismatch would it introduce?
-

Further resources

- [R for Data Science \(2e\): Joins](#)
- [Mutating joins — dplyr reference](#)
- [CHIRPS overview](#)
- [ClimateSERV API documentation](#)
- [Geocomputation with R](#)
- [The Turing Way: Research Data Management](#)

6.1) Why prepare data?

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Managed and integrated data are not automatically analysis-ready
 - Preparation decisions are analytical decisions
 - Preparation happens around integration
 - Preparation should create new artifacts
 - Common preparation decisions
 - What can go wrong
 - How this connects to the maize-yield project
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- explain why managed and integrated data may still be unsuitable for analysis;
 - describe how preparation choices affect the population, variables, and meaning represented;
 - distinguish validation, cleaning, integration, and preparation;
 - explain why preparation may occur before and after integration;
 - identify common risks of undocumented or destructive transformations; and
 - describe the role of an analysis contract, audit, dictionary, and lineage record.
-

Place in the session

This is the **Motivation** part of the Data Preparation session:

```
Motivation → Concepts → Application
      ↑
  this page
```

The preceding sessions made the source artifacts understandable, auditable, and connected the FAOSTAT maize statistics with CHIRPS growing-season precipitation. This session asks how that managed, integrated data should be represented for a defined analysis.

[Understand data-preparation concepts](#) explains the relevant distinctions and transformation patterns. [Prepare the maize country-year data](#) applies them in the example project.

Managed and integrated data are not automatically analysis-ready

A dataset can be well documented, validated, and integrated while still having an inconvenient structure for analysis.

The fixed FAOSTAT input uses a long representation:

| area | year | element | unit | value |
|--------|------|----------------|-------|---------|
| Zambia | 2022 | Yield | kg/ha | 2340 |
| Zambia | 2022 | Production | t | 3621000 |
| Zambia | 2022 | Area harvested | ha | 1547000 |

This representation preserves the provider's element and unit fields. A country-year analysis generally needs a separate column for each retained measure:

| country | year | yield_tonnes_per_hectare | production_tonnes | harvested_area_hectares |
|---------|------|--------------------------|-------------------|-------------------------|
| Zambia | 2022 | 2.34 | 3621000 | 1547000 |

Neither shape is universally superior; the right representation depends on what one observation should mean and which comparisons follow.

Preparation decisions are analytical decisions

Data preparation is sometimes treated as routine housekeeping, but its choices affect every later result: filtering changes the represented population, aggregation changes the grain, reshaping changes what rows and columns mean, unit conversion changes numeric values, recoding can merge or separate categories, and missing-value or outlier treatment changes the distribution.

A script can execute these operations correctly while implementing an inappropriate scientific decision. Preparation therefore begins with an **analysis contract** that states the intended population, grain, key, variables, units, periods, and missing-value policy.

Preparation happens around integration

The teaching sequence presents Data Integration before Data Preparation, but a real workflow is not a single linear pass through topic labels. Source-specific preparation (reshaping FAOSTAT into a country-year panel) is often needed before two sources can be joined, and further preparation (selecting variables, deriving transformations) can follow integration:

```
FAOSTAT long input → reshape → integrate with CHIRPS → select/derive → analyze
```

This does not weaken the distinction between topics; it clarifies their different teaching focus:

| Activity | Main question |
|------------------|---|
| Data management | What do the data mean, and how are they governed? |
| Data integration | How are compatible observations connected across sources? |
| Data preparation | How are data represented for a defined analysis? |

Workflow order follows dependencies; documentation should make each decision visible even when several activities occur in one script.

Preparation should create new artifacts

Managed input data should not be edited in place. A reproducible pattern applies a documented script to the managed input and writes a derived output with its own checks, dictionary, and lineage. This keeps the original evidence available for comparison, lets every transformation be reviewed in version control, allows the output to be regenerated, and lets errors be traced to a specific step.

A derived dataset is not less important than a source dataset. It requires its own grain, key, variable definitions, limitations, and provenance.

Common preparation decisions

- **Select and filter** — retain only the variables and observations the analysis requires, but state the inclusion rule and report how coverage changes.
- **Parse and recode** — convert text to numbers, dates, or categories; report failed parses and unmapped values rather than silently turning them missing.
- **Reshape** — change between long and wide forms only once the input grain and candidate key are known; unexpected duplicate cells usually mean an incomplete key.
- **Convert units** — record the source unit, target unit, formula, and valid domain; never change a unit label without changing the values.
- **Address missingness** — distinguish unavailable measurements, structural absence, invalid values, and join-induced missingness; do not replace a missing value merely because a later function rejects it.
- **Derive variables** — document formulas, inputs, units, valid domains, and interpretation, and retain the untransformed variable for review.

What can go wrong

- **The source is edited manually** — a spreadsheet correction destroys the boundary between source evidence and derived decisions; preserve the input and express corrections in code.
- **Filtering silently changes the population** — removing incomplete countries or years can change the research question; report inclusion criteria and coverage before and after.
- **Reshaping multiplies or loses observations** — a wide transformation requires at most one value per target cell; test the input key rather than choosing an aggregation function to suppress a warning.

- **Missing values become zero** — zero is an observed value with meaning; missing is the absence of a usable value; they are not interchangeable.
 - **Outliers are deleted because they look unusual** — an extreme value may be an error, a legitimate event, or part of the phenomenon; investigate it using definitions, flags, and source evidence.
 - **A transformation leaks test information** — imputation values, scaling parameters, or selected features learned from the complete dataset can reveal the test period to a predictive model; such operations must be estimated from training data only.
 - **The output has no documentation** — a clear script does not replace a dictionary or dataset description; users need to understand the output without reconstructing every expression.
-

How this connects to the maize-yield project

The example project performs source-specific preparation in `scripts/prepare-maize-data.R`: it validates the fixed FAOSTAT input, maps provider elements to project measure names, reshapes the long values into a country-year panel, converts yield to tonnes per hectare, derives log yield for positive values, and writes `data/derived/maize-yield-panel.csv`. The integration script then maps project identifiers and adds CHIRPS precipitation, producing `data/derived/maize-yield-with-precipitation.csv` — the starting point for visualization, descriptive analysis, and modeling.

This session makes the contracts and evidence behind those transformations explicit, and identifies future improvements such as a dedicated preparation audit and documentation for the intermediate maize panel.

Check your understanding

1. Why can a validated dataset still require preparation?
 2. Give an example of a preparation decision that changes the represented population.
 3. Why must the input key be known before reshaping from long to wide?
 4. What is the difference between a missing value and zero?
 5. Why should an unusual value not be removed automatically?
 6. Why can preparation occur both before and after integration?
 7. Which preparation operations can cause leakage in predictive modeling?
 8. What evidence should accompany an analysis-ready dataset?
-

Further resources

- Wickham, H., Çetinkaya-Rundel, M., and Grolemund, G., [R for Data Science \(2e\): Data transform](#) introduces filtering, mutation, grouping, and summarization with `dplyr`.
 - [The Turing Way: Data cleaning](#) discusses reproducible cleaning, preservation of raw data, and documented decisions.
 - [tidymodels: Data preprocessing](#) introduces preprocessing estimated within a modeling workflow and is useful for understanding leakage.
-

Continue to Concepts

Continue with [Understand data-preparation concepts](#). It develops the analysis contract, tidy-data model, transformation vocabulary, missingness and anomaly decisions, preparation audits, lineage, and leakage boundary.

6.2) Understand data-preparation concepts

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Preparation and related activities
 - Begin with an analysis contract
 - Observations, variables, and tidy structure
 - Select and filter
 - Rename, parse, and recode
 - Reshape data
 - Convert units
 - Missing values and structural absence
 - Duplicates and repeated observations
 - Outliers and errors
 - Derived variables and transformations
 - Data types and valid domains
 - Preparation audits
 - Lineage and documentation
 - Preparation and data leakage
 - Idempotence and deterministic outputs
 - Check your understanding
 - Further resources
 - Continue to Application
-

Learning objectives

After completing this page, you should be able to:

- distinguish validation, cleaning, integration, and preparation;
 - specify a target population, grain, candidate key, variables, units, and missing-value policy;
 - select, filter, parse, recode, reshape, convert, and derive values explicitly;
 - distinguish missing values, duplicates, outliers, and errors;
 - design checks that compare preparation inputs and outputs; and
 - identify preprocessing operations that must be estimated from training data to prevent leakage.
-

Place in the session

This is the **Concepts** part of the Data Preparation session:

```
Motivation → Concepts → Application
              ↑
            this page
```

[Why prepare data?](#) explains why managed and integrated evidence may not yet have the representation required for analysis. This page provides the vocabulary and decision model used in [the maize preparation application](#).

Use these concepts as questions: What should one output row represent? Which transformations change values or meaning, and what could be lost? Can every output variable be traced to its inputs and formula?

Preparation and related activities

These activities interact, but they answer different questions:

| Activity | Central question |
|------------------------|---|
| Validation | Do data meet documented expectations? |
| Cleaning | Is a value demonstrably erroneous, and how should it be corrected? |
| Integration | How are compatible observations connected across sources? |
| Preparation | How should data be represented for the intended analysis? |
| Modeling preprocessing | Which transformations must be learned without seeing test outcomes? |

The same operation can play different roles: converting a provider's text year to an integer is source normalization, centering a predictor using the training mean is modeling preprocessing, and replacing a documented invalid code is cleaning — the code alone does not establish the purpose. Validation should generally precede transformation; a failed expectation should not be hidden by a later filter, coercion, or aggregation.

Begin with an analysis contract

An **analysis contract** describes the intended output before code is written:

| Component | Question |
|----------------------------|---|
| Purpose, population, grain | Which task will use the output, which entities/periods does it represent, and what does one row mean? |
| Candidate key, variables | Which variables identify a row uniquely, and which source/derived variables are required? |
| Units, time | Which unit/scale and date, period, or reporting-year convention applies? |
| Missingness, ordering | Which missing states are possible, and is row order meaningful? |
| Output | Where is the artifact written and which process recreates it? |

For the maize project:

One row represents one selected project country and year, containing maize yield, production, harvested area, and precipitation for the October–April season ending in that year.

The candidate key is:

```
project_country_id + year
```

A contract is not permanent: version a new preparation rather than silently changing an existing output's meaning.

Observations, variables, and tidy structure

A common tidy-data convention is:

- each variable has its own column;
- each observation has its own row; and
- each type of observational unit has its own table.

This makes many operations predictable, but does not mean every source must be stored in tidy form. For FAOSTAT, a provider-oriented long table preserves element and unit as values:

| area | year | element | unit | value |
|--------|------|------------|-------|---------|
| Zambia | 2022 | Yield | kg/ha | 2340 |
| Zambia | 2022 | Production | t | 3621000 |

For country-year analysis, yield and production can become separate variables:

| country | year | yield_kg_per_hectare | production_tonnes |
|---------|------|----------------------|-------------------|
| Zambia | 2022 | 2340 | 3621000 |

The important question is not whether a table looks tidy, but whether its grain, key, variables, and relationships are explicit and appropriate.

Select and filter

Selection chooses variables; **filtering** chooses observations. Before filtering, state the inclusion rule; after filtering, record its effect on row count, entities and periods, missingness, and the intended population.

Example:

```
maize <- raw |>
  dplyr::filter(item == "Maize (corn)")
```

This is justified when the analysis contract concerns maize. A filter such as `filter(!is.na(yield))` requires more care, since it changes the population to complete observations and can introduce selection bias. Avoid choosing rows by position; choose them by explicit identifiers, dates, statuses, or values.

Rename, parse, and recode

Renaming can improve consistency and readability, but must retain a mapping to the source names. A good name indicates meaning and, for measures, often the unit:

```
Value → yield_tonnes_per_hectare
```

This mapping is valid only for rows already identified as the FAOSTAT Yield element with unit kg/ha and after converting the values.

Parsing converts a representation into a data type — text to integer, double, date, or categorical value. Check failed parses explicitly: a parser that returns missing values may have encountered an invalid source value, an unexpected locale, or an incomplete format.

Recoding maps values or classifications:

```
measure <- dplyr::case_when(  
  element == "Yield" & unit == "kg/ha" ~ "yield_kg_per_hectare",  
  element == "Production" & unit == "t" ~ "production_tonnes",  
  element == "Area harvested" & unit == "ha" ~ "harvested_area_hectares",  
  TRUE ~ NA_character_  
)
```

Every input value should map to an expected output or be reported unresolved; a silent “other” fallback can conceal source changes.

Reshape data

A **wide** transformation turns category values into columns. Before widening, test that the intended identifiers plus the column-name variable identify at most one value:

```
country + year + measure
```

Then:

```
panel <- tidy |>  
  tidyr::pivot_wider(  
    names_from = measure,  
    values_from = value  
  )
```

If `pivot_wider()` produces list columns or requests an aggregation, stop and inspect the key. Choosing the first value or mean merely to complete the operation can destroy information.

A **long** transformation turns several columns into a name-value pair, useful when the same operation or visualization should apply to several measures. Record which columns were gathered, the names and types of the new variables, and whether the transformation is reversible. Reshaping should change representation, not silently change grain through aggregation.

Convert units

A unit conversion requires the source and target definitions and units, a formula, the valid domain, dimensional consistency, and a check on representative values. For maize yield:

```
yield_tonnes_per_hectare = yield_kg_per_hectare / 1000
```

The denominator remains hectares; only kilograms convert to tonnes. A recommended check:

```
stopifnot(  
  all.equal(  
    yield_tonnes_per_hectare * 1000,  
    yield_kg_per_hectare  
  )  
)
```

Do not mix unit conversion with a conceptual change: converting daily precipitation to a growing-season total also requires temporal aggregation, and must document the period and summary rule.

Missing values and structural absence

A missing value can represent different states — not reported, not applicable, invalid after parsing, not observed after a join, or intentionally withheld — and each deserves its own representation rather than being converted to zero or silently dropped.

Imputation creates estimated values and requires a method, assumptions, an indicator of which values were imputed, and — when used for prediction — training-only estimation. Compute missingness summaries before and after preparation by important groups such as country and year.

Duplicates and repeated observations

A **duplicate** is not defined merely by two identical-looking rows — it depends on the expected grain and candidate key. Repeated records may be legitimate when they differ by item, measurement method, source status or revision, unit, spatial support, time period, or quality flag.

When a candidate key is duplicated, preserve the records, inspect the omitted variables, consult source documentation, and justify any aggregation or correction rather than calling `distinct()` without understanding the key.

Outliers and errors

An **outlier** is unusual relative to a distribution or model; an **error** is a value known to be incorrect under justified evidence — not synonyms. Investigate using provider flags, definitions, neighboring periods, and domain knowledge, then retain and report, correct through a documented rule, exclude while preserving the source, or mark unresolved. Deleting a point because it changes a result is not a defensible preparation rule.

Derived variables and transformations

A derived variable should have a precise name and definition, its input variables and source artifacts, a formula or algorithm, a unit, a valid domain, missingness behavior, and an interpretation and limitation.

The example project derives:

```
log_yield = log(yield_tonnes_per_hectare)
```

The logarithm is defined only for positive yield; a safe transformation returns missing for zero, negative, or missing inputs and records that rule. It also changes interpretation — a one-unit change on the log scale is not a one-tonne-per-hectare change — so the untransformed yield should be retained for plots and interpretation on the original scale.

Other common derived variables include rates, proportions, differences, rolling or grouped summaries, categorical bins, and interaction terms. Each introduces assumptions and should be created as late as necessary for its purpose.

Data types and valid domains

A storage type does not fully specify a variable:

| Variable | Storage type | Domain |
|--------------------|--------------|--|
| Project country ID | Character | Nine reviewed codes |
| Year | Integer | 1990–2022 |
| Yield | Double | Non-negative, t/ha |
| Season start/end | Date | 1 October–30 April spanning the reporting year |

Validate domains after parsing and transformation: a numeric value can still have an invalid unit or range, and a character variable can still contain an unknown code. Dates need explicit formats and time semantics — the string “2022” can mean a calendar year, reporting year, or season-ending year, and its storage type alone does not resolve that meaning.

Preparation audits

A preparation audit compares inputs, decisions, and outputs, for example:

| Check | Example expectation |
|-----------------|---|
| Input identity | Expected path or checksum, unchanged after the run |
| Input key | No duplicate area-year-element-unit keys |
| Mappings | Every element-unit combination is recognized |
| Output key | Country-year is unique; row count matches expectation |
| Unit conversion | Converted values reproduce source values |

| Check | Example expectation |
|------------------------|---|
| Missingness and domain | Changes match documented rules (e.g. log yield missing only when yield is non-positive) |

Classify checks consistently as **pass** (expectation satisfied), **warning** (review required but evidence remains usable), **failure** (a critical contract is violated and output should not be written), or **unknown** (code alone cannot decide). An audit should be machine-readable and summarized for people, and should not rewrite the input to make its own checks pass.

Lineage and documentation

The prepared dataset needs several complementary records: a **data dictionary** defining every output variable, type, unit, role, allowed values, and missingness; **human-readable documentation** explaining what the dataset represents, how it was prepared, and its limitations; and **provenance and lineage** — for example, a compact table mapping each output variable to its source field and operation.

The script is executable evidence, but it is not a substitute for these records.

Preparation and data leakage

A transformation causes **data leakage** when it uses information unavailable at prediction time, or lets test data influence model training — for example, imputing or scaling with full-dataset statistics, or selecting variables using test outcomes. Deterministic, source-defined operations (unit conversion, explicit label mapping, parsing a documented date format) are usually safe before the split; when uncertain, separate preparation into those deterministic transformations and model-estimated preprocessing fit on training data only. This boundary becomes central in the Predictive Modeling session.

Idempotence and deterministic outputs

A deterministic preparation produces the same output from the same inputs, code, parameters, and environment. An **idempotent** workflow can be rerun safely without accumulating duplicated rows or repeatedly transforming its own output. Guidelines: read managed inputs rather than a previous derived output unless the dependency is explicit; overwrite generated outputs only after checks pass; avoid current-date, random, or row-order dependence unless controlled; sort output by explicit keys; and fail visibly on unresolved schemas or mappings.

Checksums can establish byte identity, but deterministic meaning also depends on package versions, locale, numeric representation, and output ordering.

Check your understanding

1. How do validation and preparation differ?
 2. What information belongs in an analysis contract?
 3. What should you inspect when a wide transformation produces multiple values per cell?
 4. Why is missing not equivalent to zero?
 5. When is a repeated row not a duplicate?
 6. What evidence is needed before calling an outlier an error?
 7. Which preprocessing operations must be fit using training data only?
 8. What makes a preparation workflow deterministic and safe to rerun?
-

Further resources

- [R for Data Science \(2e\): Data transform](#) and [Data tidying](#)
 - [The Turing Way: Data cleaning](#)
 - [tidymodels: Preprocess your data with recipes](#) and [Feature Engineering and Selection](#)
-

Continue to Application

Continue with [Prepare the maize country-year data](#). The application states the preparation contract, inspects the managed FAOSTAT input, reshapes its elements, converts and derives variables, validates the country-year panel, integrates CHIRPS, and documents the resulting artifacts.

6.3) Prepare the maize country-year data

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverables
 - Before you begin
 1. State the preparation contract
 2. Inspect the managed input
 3. Validate mappings before reshaping
 4. Create the country-year maize panel
 5. Verify unit conversion and derived variables
 - Yield conversion
 - Log yield
 - 6. Integrate the prepared panel with CHIRPS
 - 7. Audit the prepared artifacts
 - 8. Document preparation and lineage
 - Preparation before predictive modeling
 - Troubleshooting
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- state the target population, grain, key, variables, units, and missing-value policy;
 - validate element-unit mappings and source-key uniqueness before reshaping;
 - reshape the long FAOSTAT values into a country-year panel;
 - verify a unit conversion and a log transformation; and
 - identify the documentation required for intermediate and final derived data.
-

Place in the session

This is the **Application** part of the Data Preparation session:

```
Motivation → Concepts → Application
                        ↑
                        this page
```

Before beginning, review [Why prepare data?](#) and [Understand data-preparation concepts](#).

The preceding sessions established managed FAOSTAT and CHIRPS inputs and an audited integration workflow. This exercise focuses on the transformations that create the country-year representation required by later visualization, description, and modeling.

Do not continue through a failed key, mapping, or coverage expectation only to obtain the expected output filename.

Scenario and deliverables

The managed FAOSTAT teaching input stores three maize elements in long form (Yield in kg/ha, Production in t, Area harvested in ha). The project needs one row per country and year with separate columns for those measures, plus stable project identifiers and growing-season precipitation from the preceding integration topic.

The workflow creates:

```
data/derived/maize-yield-panel.csv
data/derived/maize-yield-with-precipitation.csv
results/tables/data-integration-audit.csv
```

The maize panel is an intermediate derived artifact; the integrated table is the current input to exploration and modeling.

For this exercise, the preparation evidence should establish the input and output grain and keys, element-unit mappings, row-count and coverage expectations, unit conversion and derived-variable domain, missingness behavior, unchanged managed inputs, and lineage from source fields to output variables.

Before you begin

Work from the standalone maize-yield-project repository. Confirm that the working tree is in the expected state:

```
pwd
git status --short
```

Restore and verify the recorded environment:

```
Rscript scripts/setup.R
```

Confirm that the fixed inputs exist:

```
ls -lh \
  data/input/faostat-maize-yield-sample.csv \
  data/input/chirps-growing-season-precipitation.csv
```

The student workflow uses these tracked snapshots and requires no network connection; do not run the acquisition scripts for this exercise.

Relevant files include:

| Role | File |
|-----------------------------|---|
| Managed FAOSTAT input | data/input/faostat-maize-yield-sample.csv |
| FAOSTAT dictionary | metadata/faostat-maize-yield-data-dictionary.csv |
| FAOSTAT flag definitions | metadata/faostat-flag-code-list.csv |
| Provenance | metadata/provenance.yml |
| Preparation script | scripts/prepare-maize-data.R |
| Prepared maize panel | data/derived/maize-yield-panel.csv |
| Integration script | scripts/integrate-data.R |
| Final integrated dictionary | metadata/maize-yield-with-precipitation-data-dictionary.csv |

The data/derived/ directory contains generated outputs and is ignored by Git. Recreate its contents through scripts rather than editing them.

1. State the preparation contract

Complete the following before running the script:

```
Purpose:
Input artifact:
Input grain:
Input candidate key:
Output artifact:
Output population:
Output grain:
Output candidate key:
Retained variables:
Source and target units:
Derived variables:
Missing-value policy:
Expected row count:
```

For this project:

| Component | Contract |
|------------------|--|
| Purpose | Country-year exploration and modeling of maize yield |
| Input | Fixed FAOSTAT maize teaching sample |
| Input grain | One area-item-element-year-unit observation |
| Input key | area + item + element + year + unit |
| Output | Prepared maize country-year panel |
| Population | Nine selected countries, 1990–2022 |
| Output grain | One country and year |
| Output key | country + year |
| Expected rows | 9 countries $\times$ 33 years = 297 |
| Yield unit | Convert kg/ha to t/ha |
| Derived variable | Natural log of positive yield |

| | |
|-------------|--|
| Component | Contract |
| Missingness | Preserve missing measures; never replace with zero |

Predict which source columns will remain and which will become output column names and units.

2. Inspect the managed input

Read without modifying:

```
library(dplyr)
library(readr)

raw <- read_csv(
  "data/input/faostat-maize-yield-sample.csv",
  show_col_types = FALSE
)

glimpse(raw)
nrow(raw)
names(raw)
```

Inspect coverage and classifications:

```
raw |>
  count(area, name = "rows")

raw |>
  distinct(item)

raw |>
  distinct(element, unit) |>
  arrange(element, unit)

range(raw$year)
```

Confirm nine areas, one item (Maize (corn)), years 1990–2022, three documented element-unit combinations, and 891 rows before reshaping. Read the data dictionary and flag code list: a variable name alone is insufficient to interpret value, since its meaning and unit depend on element and unit.

3. Validate mappings before reshaping

The preparation script expects:

```
expected_element_units <- c(
  "Area harvested|ha",
  "Production|t",
  "Yield|kg/ha"
)
```

Compare observed pairs:

```
observed_element_units <- raw |>
  distinct(element, unit) |>
  transmute(pair = paste(element, unit, sep = "|")) |>
  pull(pair)

setequal(observed_element_units, expected_element_units)
```

Test the source candidate key:

```
candidate_key <- c("area", "item", "element", "year", "unit")

duplicate_keys <- raw |>
  count(across(all_of(candidate_key))) |>
  filter(n > 1)

duplicate_keys
```

Expected result: no duplicate candidate keys.

The mapping from provider elements to project measures is:

| Source element | Source unit | Project measure |
|----------------|-------------|-------------------------|
| Yield | kg/ha | yield_kg_per_hectare |
| Production | t | production_tonnes |
| Area harvested | ha | harvested_area_hectares |

Every observed pair must map exactly once; do not use a catch-all label.

4. Create the country-year maize panel

Run the existing project script:

```
Rscript scripts/prepare-maize-data.R
```

The script first creates a normalized long representation:

```
tidy <- raw |>
  filter(item == "Maize (corn)") |>
  transmute(
```

```

country = area,
year = as.integer(year),
measure = case_when(
  element == "Yield" & unit == "kg/ha" ~ "yield_kg_per_hectare",
  element == "Area harvested" & unit == "ha" ~ "harvested_area_hectares",
  element == "Production" & unit == "t" ~ "production_tonnes",
  TRUE ~ NA_character_
),
value = as.numeric(value)
)

```

It then reshapes:

```

panel <- tidy |>
  tidyr::pivot_wider(
    names_from = measure,
    values_from = value
  ) |>
  arrange(country, year)

```

Inspect the output:

```

panel <- read_csv(
  "data/derived/maize-yield-panel.csv",
  show_col_types = FALSE
)

glimpse(panel)
nrow(panel)

```

Test the output key:

```

panel |>
  count(country, year) |>
  filter(n != 1)

```

Expected result: 297 rows and no duplicate country-year key.

If widening produces list columns or multiple values per cell, return to the input key. Do not choose an arbitrary aggregation function to force a scalar output.

5. Verify unit conversion and derived variables

Yield conversion

The project converts:

```

yield_tonnes_per_hectare = yield_kg_per_hectare / 1000

```

Verify the conversion against the source values:

```
source_yield <- raw |>
  filter(element == "Yield", unit == "kg/ha") |>
  transmute(
    country = area,
    year,
    source_yield_kg_per_hectare = value
  )

conversion_check <- panel |>
  left_join(source_yield, by = c("country", "year")) |>
  mutate(
    reconstructed_kg_per_hectare =
      yield_tonnes_per_hectare * 1000,
    difference =
      reconstructed_kg_per_hectare -
      source_yield_kg_per_hectare
  )

max(abs(conversion_check$difference), na.rm = TRUE)
```

Expected result: zero or negligible floating-point difference.

Log yield

The project derives:

```
log_yield = log(yield_tonnes_per_hectare)
```

Check its domain and behavior:

```
panel |>
  summarise(
    non_positive_yield =
      sum(yield_tonnes_per_hectare <= 0, na.rm = TRUE),
    missing_log_for_positive =
      sum(
        is.na(log_yield) &
        yield_tonnes_per_hectare > 0,
        na.rm = TRUE
      ),
    finite_log_values =
      all(is.finite(log_yield[!is.na(log_yield)]))
  )
```

The log is undefined for non-positive values. The original yield remains in the panel so results can be interpreted on the physical scale.

6. Integrate the prepared panel with CHIRPS

Preparation and integration have a dependency:

```
prepared maize panel
      +
CHIRPS country-season snapshot
      +
project country crosswalk
      ↓
integrated country-year table
```

Run:

```
Rscript scripts/integrate-data.R
```

The integration script:

- maps country to project_country_id;
- validates both country-year keys;
- left-joins precipitation on project identifier and year;
- preserves 297 maize rows;
- records unmatched keys and missing precipitation; and
- writes an integration audit.

Inspect:

```
integrated <- read_csv(
  "data/derived/maize-yield-with-precipitation.csv",
  show_col_types = FALSE
)

glimpse(integrated)
nrow(integrated)

integrated |>
  count(project_country_id, year) |>
  filter(n != 1)
```

The final table has one row per project country and year. The CHIRPS season ending in a year is aligned to the FAOSTAT observation for that year.

7. Audit the prepared artifacts

A dedicated preparation audit is not yet stored by the example project. For this exercise, assemble these checks in code or a table:

| Check | Expectation |
|----------------------------------|-------------------------------------|
| Managed input rows | 891 |
| Managed input key duplicates | 0 |
| Recognized element-unit pairs | Exactly 3 expected pairs |
| Prepared panel rows | 297 |
| Prepared output key duplicates | 0 |
| Country coverage | 9 |
| Year coverage | 1990–2022 |
| Unit-conversion discrepancy | 0 within numeric tolerance |
| Missing log for positive yield | 0 |
| Non-finite retained log values | 0 |
| Managed input checksum | Matches provenance before and after |
| Integrated output rows | 297 |
| Integrated output key duplicates | 0 |

A possible machine-readable structure is:

```
check,expectation,observed,status
input-rows,891,891,pass
input-key-duplicates,0,0,pass
prepared-rows,297,297,pass
...
```

Do not hard-code a passing status independently of the observed result. A critical failure should prevent the output from being treated as ready.

This audit is a recommended future improvement to the example project. Until it is implemented there, the existing script checks and integration audit are the executable evidence.

8. Document preparation and lineage

The final integrated artifact already has:

- a human-readable page: docs/data/maize-yield-with-precipitation.md;
- a data dictionary: metadata/maize-yield-with-precipitation-data-dictionary.csv;
- lineage in metadata/provenance.yml; and
- an integration report and audit.

The intermediate maize-yield-panel.csv is recorded in provenance but currently lacks its own data dictionary and dataset page. A complete implementation should consider adding:

```
metadata/maize-yield-panel-data-dictionary.csv
docs/data/maize-yield-panel.md
results/tables/data-preparation-audit.csv
docs/data-preparation.md
```

Avoid duplicating authoritative facts: Markdown explains purpose and limitations, CSV defines output variables, YAML records artifact history, scripts execute transformations, and audit tables record observed checks.

A lineage table for the maize panel should include:

| Output variable | Source field | Operation |
|--------------------------|----------------------------|---------------------------------|
| country | area | Rename retained label |
| year | year | Parse as integer |
| yield_tonnes_per_hectare | Yield value in kg/ha | Divide by 1000 |
| production_tonnes | Production value in t | Reshape |
| harvested_area_hectares | Area harvested value in ha | Reshape |
| log_yield | Prepared yield | Natural log for positive values |

Preparation before predictive modeling

The current transformations are based on fixed source definitions: element-unit mapping, deterministic reshaping, unit conversion, country crosswalk, temporal alignment, and a log formula with a fixed mathematical definition.

Later modeling steps may introduce transformations learned from observed data — mean/median imputation, centering and scaling, feature selection, or data-dependent bins — that must be estimated using training data only and applied unchanged to the test period. Do not add such operations to a general preparation script that runs on the complete dataset.

Troubleshooting

| Symptom | Response |
|---|---|
| The input file is missing | Run git status and confirm the tracked teaching input was checked out; do not run a provider download automatically. |
| Required columns are missing | Compare the file with its dictionary and provenance — a changed schema may mean the wrong artifact is present. |
| An element-unit combination is unexpected | Do not map by element alone; inspect the pair and provider metadata before updating the contract. |
| The source key is duplicated | Do not call distinct() automatically; determine whether the key omits a classification, unit, or revision dimension. |
| Widening creates multiple values | The proposed target cell is not unique; return to the input grain and mapping. |
| The panel has fewer than 297 rows | Inspect missing country-year-element combinations; do not create rows or fill values until the absence is understood. |
| The log contains infinite values | Inspect zero or negative yield; the safe transformation should not retain infinite values. |
| A rerun changes managed inputs | Stop — preparation must write only derived artifacts. Restore the input through version control and inspect the script paths. |

Completion checklist

- ☐ The purpose, population, grain, key, variables, units, and missingness policy are stated.
 - ☐ The managed FAOSTAT input remains unchanged.
 - ☐ Required columns and element-unit combinations are validated.
 - ☐ The source candidate key is unique.
 - ☐ The long-to-wide transformation preserves the intended country-year grain.
 - ☐ Yield conversion is verified against source values.
 - ☐ Log yield is finite and missing only under the documented rule.
 - ☐ The prepared panel has 297 unique country-year rows.
 - ☐ Integration preserves 297 unique project-country-year rows.
 - ☐ Missingness and coverage changes are explained.
 - ☐ Output variables can be traced to inputs and formulas.
 - ☐ Preparation checks are recorded or identified as a current implementation gap.
 - ☐ Data-dependent modeling preprocessing is deferred until after the train/test split.
-

Reflect on the application

1. Why is the long FAOSTAT input useful even though the analysis uses a wide panel?
 2. Which variables define a unique value before widening?
 3. Why is division by 1000 a unit conversion rather than an arbitrary scaling?
 4. Why retain yield on the original physical scale after creating log yield?
 5. Which preparation steps must occur before CHIRPS integration, and which could occur after?
 6. Which future preprocessing operations could leak test information?
-

Further resources

- [R for Data Science \(2e\): Data transform](#)
- [pivot_wider](#) — [tidyr reference](#)
- [Mutate joins](#) — [dplyr reference](#)

7.1) Why visualize data?

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Prepared data do not reveal patterns by themselves
 - A graphic is an analytical choice
 - Exploration and communication have different purposes
 - Visualization supports quality review
 - What can go wrong
 - The chart form does not match the question
 - A truncated scale exaggerates a small difference
 - Aggregation hides the relevant variation
 - Colour carries meaning that some readers cannot access
 - Overplotting creates false absence
 - A visual association is described as causal
 - How this connects to the maize-yield project
 - Initial activity
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- explain why visual representations complement tables and numerical summaries;
 - describe a chart as a sequence of analytical and communication choices;
 - distinguish exploratory from explanatory visualization;
 - identify ways in which scales, aggregation, filtering, and design can mislead;
 - explain how visualization supports data-quality investigation without proving an error; and
 - formulate visual questions for the maize-yield and precipitation data.
-

Place in the session

This is the **Motivation** part of the Data Visualization session:

```
Motivation → Concepts → Application
      ↑
    this page
```

Data Preparation created documented country-year representations of maize yield and growing-season precipitation. This session asks how to map those observations to visual properties so that patterns can be inspected and communicated without hiding the data's meaning or limitations.

[Understand data-visualization concepts](#) develops the question-to-graphic workflow. [Visualize maize yield and precipitation](#) applies it in the example project.

Prepared data do not reveal patterns by themselves

A prepared table preserves exact values, documented variables, units, observation grain, and candidate keys, but a reader cannot easily recognize 297 country-year observations by scanning a CSV file.

Visual representations can make several structures easier to inspect:

- the shape and range of a distribution;
- differences between countries;
- changes and interruptions over time;
- unusual observations;
- missing periods; and
- possible relationships between yield and precipitation.

Visualization does not replace the table: a chart selects and encodes parts of it for a particular question, and useful visualization deliberately reduces complexity rather than displaying every variable and limitation.

A graphic is an analytical choice

A chart is not a neutral photograph of a dataset. Before any mark appears, an analyst has already decided:

- which observations and variables to include;
- whether marks show observations or summaries;
- which variables control position, colour, size, shape, or facets;
- where axes begin and end and whether scales are transformed; and
- which labels, annotations, dimensions, and resolution the audience sees.

These choices determine which comparisons are easy and which variation becomes invisible. For example, showing one mean yield per country answers a different question from showing all annual yields, and free facet scales that ease within-country reading also prevent direct magnitude comparison.

The relevant question is therefore not only, “Is the code correct?” It is also, “Does this visual representation support the intended comparison honestly?”

Exploration and communication have different purposes

Exploratory visualization supports investigation, usually for the analyst or project team. Many temporary graphics may be created to inspect distributions, missingness, group differences, or alternative scales, generating questions and testing interpretations.

Explanatory visualization supports communication: it selects a defensible finding for a defined audience and generally needs a clear title, readable labels, visible units, source information, and a statement of limitations.

The two purposes form a workflow:

```
question
↓
several exploratory views
↓
inspection and revision
↓
one focused communication graphic
↓
caption and qualified interpretation
```

An explanatory figure should remain traceable to prepared data and reproducible code. Manual adjustments that cannot be recreated weaken that traceability.

Visualization supports quality review

Plots can reveal observations that deserve investigation. A time series may show a jump, a histogram an unexpected boundary, or a scatterplot repeated values.

These patterns are signals, not verdicts. An unusual value may represent:

- a source or parsing error;
- a change in definition or reporting practice;
- a real drought, policy change, or agricultural event;
- an artifact of aggregation; or
- legitimate variation.

Return to metadata, flags, provenance, and preparation decisions before correcting anything: a plot can motivate a validation question but cannot prove that an observation is wrong.

What can go wrong

The chart form does not match the question

A chart form should not be chosen merely because it is familiar; begin with the comparison and variable types.

A truncated scale exaggerates a small difference

Bars encode magnitude through length and rely on a meaningful zero baseline. If an axis is restricted, make the choice visible and justify it.

Aggregation hides the relevant variation

A country average can conceal changes across years, and a national value can conceal within-country differences. State what one mark represents and how it was calculated.

Colour carries meaning that some readers cannot access

Avoid palettes with poor contrast or categories distinguished only by red and green; combine colour with position, labels, line type, or shape where appropriate.

Overplotting creates false absence

Many observations at the same location can appear as one point. Transparency, smaller marks, jitter, bins, or facets can reveal density, but each changes the view.

A visual association is described as causal

Yield and precipitation may vary together because of rainfall, country differences, trends, omitted variables, or coincidence. A scatterplot establishes neither causal direction nor a controlled comparison.

How this connects to the maize-yield project

The example project provides two useful analytical artifacts:

- `data/derived/maize-yield-panel.csv`, with country-year maize measures; and
- `data/derived/maize-yield-with-precipitation.csv`, which adds CHIRPS October-April country-area precipitation.

The existing exploration script creates a faceted maize-yield time series; this session expands it into a coherent visual investigation:

1. inspect the maize-yield distribution and cross-country trends;
2. inspect growing-season precipitation and its relationship to yield;
3. compare alternative encodings and scales; and
4. refine one chart for communication.

Every output must retain the documented units and country-year grain. The figures can motivate later descriptive or modeled analysis, but must not claim that precipitation alone explains changes in maize yield.

Initial activity

Before reading the Concepts page, write one visual question for each structure:

| Structure | Your question |
|--------------|-----------------------------------|
| Distribution | How is ... distributed? |
| Comparison | How does ... differ between ...? |
| Change | How has ... changed over ...? |
| Relationship | How do ... and ... vary together? |

For each question, state what one plotted mark should represent, compare answers with another learner, and discuss what different marks or groupings would make easier or harder to see.

Check your understanding

1. Why does a documented table still benefit from visualization?
 2. Which decisions make a chart an analytical choice rather than a neutral image?
 3. How do exploratory and explanatory visualization differ?
 4. Why does an unusual plotted value not prove a data error?
 5. How can aggregation change the story shown by a chart?
 6. Why is a visible yield-precipitation association not necessarily causal?
-

Further resources

- [R for Data Science \(2e\): Data visualization](#) introduces the grammar of graphics using `ggplot2`.
 - Claus O. Wilke, [Fundamentals of Data Visualization](#) discusses visual encodings, scales, colour, uncertainty, and figure design.
 - Kieran Healy, [Data Visualization: A Practical Introduction](#) develops reproducible visualization with applied examples.
 - [The Turing Way: Data visualisation](#) connects visualization with accessible and reproducible research communication.
 - [Data-to-Viz](#) provides a question-oriented overview of common chart families and their limitations.
-

Continue to Concepts

Continue with [Understand data-visualization concepts](#). It explains how questions, variables, marks, mappings, scales, grouping, accessibility, and reproducible export work together.

7.2) Understand data-visualization concepts

Outline

- Outline
- Learning objectives
- Place in the session
- Use a question-to-graphic workflow
- Know the observation grain
- Match questions and variable types
- Use a grammar of graphics
- Choose visual encodings deliberately
- Understand marks and geometric objects
- Treat scales as part of the interpretation
 - Axis limits
 - Transformations
 - Comparable scales
- Make grouping and aggregation explicit
- Address overplotting
- Use facets for repeated comparisons
- Design colour and text accessibly
- Separate exploratory and explanatory graphics
- Export reproducible figures
- Interpret without overclaiming
- Check your understanding
- Further resources
 - Grammar and implementation
 - Design and interpretation
 - Accessibility and reproducibility
- Continue to Application

Learning objectives

After completing this page, you should be able to:

- translate an analytical question into a visual comparison;
 - identify the observation grain and variables represented by plotted marks;
 - distinguish data, mappings, geometric objects, scales, coordinates, facets, and annotations;
 - select appropriate encodings for quantitative and categorical variables;
 - explain how axes, transformations, aggregation, and overplotting affect interpretation;
 - design graphics that remain readable and accessible;
 - distinguish exploratory iteration from explanatory refinement;
 - save a figure reproducibly with documented dimensions and output paths; and
 - write a qualified interpretation that distinguishes pattern, association, and causation.
-

Place in the session

This is the **Concepts** part of the Data Visualization session:

```
Motivation → Concepts → Application
              ↑
            this page
```

[Why visualize data?](#) explains why visual choices matter. This page provides the decision model used in [the maize visualization application](#).

Use one central question throughout:

Which comparison should the viewer make, and which visual choices make that comparison accurate, visible, and interpretable?

Use a question-to-graphic workflow

Avoid starting with, “Which chart should I make?” Begin with:

1. **Purpose:** Are you investigating or communicating?
2. **Question:** Is the focus a distribution, comparison, change, or relationship?
3. **Data:** Which artifact, population, grain, variables, and units are relevant?
4. **Encoding:** Which positions and other visual properties represent variables?
5. **Review:** Could scales, grouping, missingness, or design create a false impression?
6. **Interpretation:** What does the graphic support, and what remains unresolved?

This sequence is not strictly linear: a plot can reveal a new question, a metadata issue, or a need for a different representation. Record consequential changes in code rather than only through an interactive interface.

Know the observation grain

The **grain** states what one row represents. A plotted mark may represent one row, several rows summarized together, or part of a row after reshaping.

For the integrated maize data, one row represents one project country-year, including a growing season ending in that year — so in a scatterplot with yield and precipitation on the axes, one point represents one country-year.

In a plot of average yield by country, one point represents 33 country-year rows summarized by a mean: a different analytical object. Always be able to complete this sentence:

One mark in this graphic represents ...

If the sentence is unclear, return to the data preparation contract and the plotting code.

Match questions and variable types

Different structures suggest different starting graphics:

| Question | Variables | Useful starting graphics |
|---|--|--|
| How are values distributed? | One quantitative variable | Histogram, density plot, empirical distribution, boxplot |
| How do groups compare? | Quantitative value and categorical group | Aligned points, boxplots, violins, facets |
| How does a measure change? | Quantitative value and ordered time | Points and lines |
| How do two measures vary together? | Two quantitative variables | Scatterplot |
| How many observations belong to categories? | One or more categorical variables | Bars or dot plots |
| Where does a measure occur? | Measure and valid spatial geometry | Map, often alongside non-spatial comparisons |

These are starting points, not rules that guarantee validity: a boxplot can hide multimodality, a line can imply continuity between sparse observations, and a map makes area visually prominent even when area is not the quantity of interest.

Use a grammar of graphics

A grammar of graphics treats a plot as explicit components:

| Component | Question | ggplot2 example |
|----------------------------|--|---|
| Data | Which observations enter the graphic? | <code>ggplot(data = maize)</code> |
| Aesthetic mapping | Which variables control visual properties? | <code>aes(x = year, y = yield)</code> |
| Geometry | Which marks represent observations or summaries? | <code>geom_point()</code> , <code>geom_line()</code> |
| Statistical transformation | Is a count, bin, smooth, or summary calculated? | <code>stat_bin()</code> , <code>stat_summary()</code> |
| Scale | How do data values map to positions or colours? | <code>scale_y_log10()</code> |
| Coordinate system | How are scales arranged in the display? | <code>coord_cartesian()</code> |
| Facet | Which groups receive repeated panels? | <code>facet_wrap(vars(country))</code> |
| Labels and theme | How is meaning communicated? | <code>labs()</code> , <code>theme_minimal()</code> |

In `ggplot2`, a property inside `aes()` is mapped to a variable. A property outside `aes()` is fixed:

```
# Map country to colour
geom_line(aes(colour = country))
```

```
# Use one fixed colour for every line  
geom_line(colour = "#29508A")
```

This distinction prevents accidental legends and makes the visual meaning inspectable in code.

Choose visual encodings deliberately

People compare some visual properties more accurately than others. For precise quantitative comparison, prefer:

1. position on a common scale;
2. position on aligned scales;
3. length; then
4. angle, area, volume, or colour intensity.

Use **colour hue** to distinguish a manageable number of categories, and an ordered lightness scale for ordered or continuous values when the palette supports it. Avoid an arbitrary rainbow scale: perceived colour differences do not follow equal numeric differences, so transitions can appear where none exist.

Use **size** carefully. Viewers perceive the area of a mark, so doubling its radius more than doubles its visible area. Let the plotting scale calculate area consistently.

Use **shape** for few categories with enough mark size to recognize it, and encode important distinctions redundantly — through position, label, shape, or line type — rather than colour alone.

Understand marks and geometric objects

Marks carry different implications:

- **points** show individual observations;
- **lines** connect ordered observations and emphasize continuity or change;
- **bars** encode magnitude through length and generally require a meaningful zero baseline;
- **areas** emphasize totals or composition but can make internal comparisons difficult;
- **boxes and violins** summarize distributions and should be supported by sufficient observations;
- **smooths** summarize an estimated pattern and introduce method choices.

A line joining countries is inappropriate because country order is not continuous; a line joining annual observations within one country is meaningful once observations are grouped by country and ordered by year.

A histogram is not a picture of the raw distribution alone. It applies a statistical transformation that assigns values to bins and counts them. Inspect whether conclusions change across reasonable bin widths.

Treat scales as part of the interpretation

Scales determine how numeric and categorical values become visible distances, colours, sizes, or labels.

Axis limits

Restricting a scale can exaggerate differences or remove data before a statistical calculation. `coord_cartesian()` can zoom without dropping observations, but the narrower view still needs justification.

For bars, a non-zero baseline breaks the relation between length and magnitude. For points and lines, zero is not always required, but the selected range must support an honest interpretation.

Transformations

A logarithmic axis can make multiplicative change and wide ranges easier to inspect. Equal visual distances then represent equal ratios rather than equal absolute differences. Name the transformation and keep physical units available for interpretation.

Comparable scales

Fixed scales across facets support magnitude comparison; free scales can reveal within-panel patterns when groups have very different ranges but prevent direct comparison of absolute levels. State which comparison matters.

Make grouping and aggregation explicit

Grouping affects both calculations and visual connections. In a country time series, lines must be grouped by country. In a summary, `group_by(country)` changes one dataset-wide statistic into one statistic per country.

Aggregation requires a contract:

- Which rows enter each group?
- Which statistic is calculated?
- Are observations weighted?
- How are missing values handled?
- What variation disappears?
- What does each output mark represent?

Do not calculate a mean with `na.rm = TRUE` without reporting how many values were missing — a national mean and a mean across countries can imply different weights, and visualization does not remove these statistical choices.

Address overplotting

Overplotting occurs when marks overlap and hide observation frequency. Possible responses include:

- smaller marks;
- transparency with `alpha`;
- jitter for discrete or rounded positions;
- two-dimensional bins or contours;

- faceting by a meaningful group; or
- summarizing after the aggregation rule is stated.

Each response has limitations: transparency depends on display background and export format, jitter adds visual displacement that is not measurement error, bins trade precise locations for counts, and faceting can make cross-panel comparison harder. Choose the response that supports the question.

Use facets for repeated comparisons

Small multiples repeat the same visual structure for groups. Faceting the maize time series by country avoids nine overlapping lines and lets readers inspect patterns consistently.

Review:

- whether all panels use the same variables and transformations;
- whether fixed or free scales match the comparison;
- whether panel order supports interpretation;
- whether empty panels or missing periods are meaningful; and
- whether labels fit at the intended export size.

Facets are often preferable to mapping many categories to similar colours. They use position and repetition, but require enough space.

Design colour and text accessibly

Accessibility is part of analytical communication, not final decoration.

- Use palettes with sufficient contrast and colour-vision-deficiency support.
- Do not rely on red versus green alone.
- Use readable text at the final display size.
- Prefer direct labels when they reduce legend lookup.
- Provide units on quantitative axes.
- Use concise titles that state the subject, not an unsupported conclusion.
- Add captions or nearby prose describing source, grain, and limitations.
- Provide alternative text or an equivalent textual interpretation in the publication context.

Test a saved figure, not only the large interactive preview. A graphic that is readable on a laptop may fail when placed in a report or presentation.

Separate exploratory and explanatory graphics

Exploratory graphics can be numerous and provisional. They help inspect:

- distributions and missingness;
- alternative scales and groupings;
- anomalies and possible data problems;
- sensitivity to bins or aggregation; and
- questions that need descriptive or modeled analysis.

An explanatory graphic should focus on one comparison. Refine it by removing irrelevant elements, emphasizing the intended evidence, adding context, and writing a qualified caption.

Do not choose the most dramatic exploratory view after trying many alternatives without acknowledging the selection: a communication graphic should follow a defensible question, not maximize apparent difference.

Export reproducible figures

A reproducible figure has a traceable input, script, environment, and output path. Save it through code:

```
ggsave(  
  filename = here("figures", "maize-yield-trends.png"),  
  plot = yield_plot,  
  width = 10,  
  height = 7,  
  units = "in",  
  dpi = 300  
)
```

Choose format according to use:

- PNG is suitable for raster display and web use;
- PDF or SVG preserves vector marks and text when supported; and
- dimensions determine layout and readability independently of resolution.

Use stable, descriptive filenames. Do not overwrite a different conceptual figure merely because it is the latest plot. Follow the repository policy for tracked and generated figures.

Interpret without overclaiming

Separate three levels:

1. **Visible pattern:** “Yield values are higher in later years for several countries.”
2. **Descriptive association:** “Higher precipitation observations coincide with different yield values in this sample.”
3. **Causal claim:** “Increasing precipitation causes yield to rise.”

The first describes the graphic. The second requires a defined descriptive analysis. The third requires a causal design and assumptions not supplied by a scatterplot.

For the maize project, country effects, long-term trends, measurement error, spatial aggregation, irrigation, inputs, and temperature can affect a visible rainfall-yield pattern; keep these limitations attached to the visual story.

Check your understanding

1. What should you state before selecting a chart form?
2. What can one point represent in the integrated maize scatterplot?
3. How does a mapped aesthetic differ from a fixed property in `ggplot2`?
4. Why is position generally preferred for precise quantitative comparison?
5. When is a line an inappropriate mark?
6. How can axis limits and log transformations alter interpretation?
7. What information must accompany an aggregated mark?
8. Which approaches can address overplotting, and what does each sacrifice?
9. When are free facet scales useful, and what comparison do they prevent?
10. What makes a saved figure reproducible and accessible?
11. How does a visible association differ from a causal claim?

Further resources

Grammar and implementation

- [R for Data Science \(2e\): Data visualization](#)
- [ggplot2 documentation](#)
- Hadley Wickham, Danielle Navarro, and Thomas Lin Pedersen, [ggplot2: Elegant Graphics for Data Analysis](#)

Design and interpretation

- Claus O. Wilke, [Fundamentals of Data Visualization](#)
- Kieran Healy, [Data Visualization: A Practical Introduction](#)
- [Data-to-Viz](#)

Accessibility and reproducibility

- [The Turing Way: Data visualisation](#)
 - [Viridis colour scales for ggplot2](#)
 - [W3C: Images tutorial](#)
-

Continue to Application

Continue with [Visualize maize yield and precipitation](#). You will formulate plot contracts, inspect the prepared artifacts, generate complementary exploratory views, refine one communication graphic, export it through code, and review the claims it supports.

7.3) Visualize maize yield and precipitation

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverables
 - Before you begin
 - 1. State visual questions and plot contracts
 - 2. Inspect the analytical artifacts
 - 3. Examine the yield distribution
 - 4. Compare yield trends across countries
 - 5. Examine growing-season precipitation
 - 6. Explore yield and precipitation together
 - 7. Refine one communication graphic
 - 8. Export and review the figure artifacts
 - Independent extension
 - Troubleshooting
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- formulate visual questions and state what one plotted mark represents;
 - inspect plotting inputs, grain, variables, units, and missingness;
 - create distribution, time-series, and relationship graphics with `ggplot2`;
 - compare alternative bins, groupings, scales, and facets;
 - identify and respond to overplotting;
 - refine an exploratory chart into an accessible communication graphic;
 - export figures reproducibly; and
 - write a visual interpretation separating pattern from causal claims.
-

Place in the session

This is the **Application** part of the Data Visualization session:

```
Motivation → Concepts → Application
                        ↑
                        this page
```

Before beginning, review [Why visualize data?](#) and [Understand data-visualization concepts](#).

The preceding topic produced a documented maize country-year panel and preparation audit; this exercise treats those as plotting inputs and does not acquire, repair, or edit data.

Scenario and deliverables

The project team wants a small visual story that answers:

How do maize yield and growing-season precipitation vary across the selected countries and years, and which patterns should later be quantified?

The required deliverables are:

```
scripts/visualize-maize-data.R
figures/maize-yield-distribution.png
figures/maize-yield-trends.png
figures/growing-season-precipitation.png
figures/yield-versus-precipitation.png
figures/maize-yield-communication.png
```

The first four figures support exploration; the final figure should focus on one comparison for communication. Also provide a short interpretation that states:

- the visible pattern;
- population and observation grain;
- relevant units and aggregation;
- at least two limitations; and
- one question for the next session.

Do not select a dramatic title before inspecting the evidence. Do not claim that precipitation causes yield differences.

Before you begin

Work from the standalone `maize-yield-project` repository. Confirm the branch and working-tree state:

```
pwd
git status --short --branch
```

Restore the recorded environment and recreate the required data:

```
Rscript scripts/setup.R
Rscript scripts/validate-data.R
Rscript scripts/prepare-maize-data.R
Rscript scripts/integrate-data.R
```

Inspect the documentation before plotting:

| Role | File |
|------------------------------------|---|
| Prepared maize panel documentation | <code>docs/data/maize-yield-panel.md</code> |
| Prepared panel dictionary | <code>metadata/maize-yield-panel-data-dictionary.csv</code> |

| Role | File |
|-------------------------------|---|
| Integrated-data documentation | docs/data/maize-yield-with-precipitation.md |
| Integrated-data dictionary | metadata/maize-yield-with-precipitation-data-dictionary.csv |
| Preparation audit | results/tables/data-preparation-audit.csv |
| Integration audit | results/tables/data-integration-audit.csv |

Both audits must pass before their outputs are treated as ready; the workflow uses fixed snapshots and requires no network access.

Create `scripts/visualize-maize-data.R` with the common setup:

```
# Create reproducible visualizations of maize yield and precipitation.

source("scripts/functions.R")

assert_project_root()
ensure_project_directories()
check_required_packages(c("dplyr", "ggplot2", "here", "readr"))

library(dplyr)
library(ggplot2)
library(here)
library(readr)

panel_file <- here("data", "derived", "maize-yield-panel.csv")
integrated_file <- here(
  "data", "derived", "maize-yield-with-precipitation.csv"
)

required_files <- c(panel_file, integrated_file)
missing_files <- required_files[!file.exists(required_files)]
if (length(missing_files) > 0) {
  stop("Required derived data not found: ", paste(missing_files, collapse = ", "), call. = FALSE)
}

maize <- read_csv(panel_file, show_col_types = FALSE)
integrated <- read_csv(integrated_file, show_col_types = FALSE)

save_figure <- function(name, plot, width = 10, height = 7) {
  ggsave(here("figures", name), plot, width = width, height = height, units = "in", dpi = 300)
}
```

`save_figure()` centralizes the export settings shared by every figure below.

1. State visual questions and plot contracts

Before coding, complete this table:

| Figure | Question | One mark represents | Variables | Scale or grouping decision |
|--------------------|---|---------------------------------|-------------------------------|----------------------------|
| Yield distribution | How are annual yield values distributed? | One bin count | Yield | Bin width |
| Yield trends | How does yield change within countries? | One country-year joined in time | Year, yield, country | Fixed or free facets |
| Precipitation | How does seasonal precipitation vary? | Define this | Define this | Define this |
| Relationship | How do yield and precipitation vary together? | One country-year | Yield, precipitation, country | Transparency and facets |

For every figure, state:

- purpose, audience, and input artifact;
- population, observation grain, and plotted variables and units;
- filtering and missing-value rule;
- statistical transformation, grouping, and scale choices; and
- claim boundary.

The plot contract can be a Markdown table in a project note; it need not become another machine-readable metadata system.

2. Inspect the analytical artifacts

Do not infer meaning only from column names:

```
glimpse(maize)
glimpse(integrated)

nrow(maize)
nrow(integrated)

maize |>
  summarise(
    countries = n_distinct(country),
    first_year = min(year),
    last_year = max(year),
    missing_yield = sum(is.na(yield_tonnes_per_hectare))
  )

integrated |>
  summarise(
    countries = n_distinct(project_country_id),
    first_year = min(year),
    last_year = max(year),
    missing_precipitation =
```

```
sum(is.na(growing_season_precipitation_mm))
)
```

Confirm 297 unique country-year rows in each derived artifact, and review the dictionaries for `yield_tonnes_per_hectare` and `growing_season_precipitation_mm`. The precipitation value is a seasonal total of country-area mean daily CHIRPS estimates, not rainfall measured at maize fields.

3. Examine the yield distribution

Begin with a histogram:

```
yield_distribution <- ggplot(
  maize,
  aes(x = yield_tonnes_per_hectare)
) +
  geom_histogram(
    binwidth = 0.25,
    boundary = 0,
    colour = "white",
    fill = "#29508A",
    na.rm = TRUE
  ) +
  labs(
    title = "Distribution of annual maize yield",
    subtitle = "Nine selected countries, 1990-2022",
    x = "Yield (tonnes per hectare)",
    y = "Country-year observations",
    caption = paste(
      "Source: fixed FAOSTAT teaching sample.",
      "One observation represents one country-year."
    )
  ) +
  theme_minimal(base_size = 11)

yield_distribution
```

Compare at least three reasonable bin widths and record what remains stable: a histogram displays counts after binning and does not preserve the precise position of every value.

Then ask whether pooling countries answers the intended question, create a faceted version, and avoid interpreting a pooled shape as the distribution of a single homogeneous population.

Save the selected exploratory version:

```
save_figure("maize-yield-distribution.png", yield_distribution, width = 9, height = 6)
```

4. Compare yield trends across countries

Use small multiples rather than nine overlapping coloured lines:

```
yield_trends <- ggplot(
  maize,
  aes(x = year, y = yield_tonnes_per_hectare)
) +
  geom_line(colour = "#29508A", linewidth = 0.5, na.rm = TRUE) +
  geom_point(colour = "#29508A", size = 0.8, na.rm = TRUE) +
  facet_wrap(vars(country), ncol = 3) +
  labs(
    title = "Maize yield over time",
    subtitle = "Panels share a common yield scale",
    x = "Year",
    y = "Yield (tonnes per hectare)",
    caption = "Source: fixed FAOSTAT teaching sample."
  ) +
  theme_minimal(base_size = 10) +
  theme(panel.grid.minor = element_blank())

yield_trends
```

Explain why grouping works even though `group = country` is not explicit: each facet contains one country. Create a second version using `scales = "free_y"`. Compare the two:

- Which makes absolute country levels easier to compare?
- Which makes within-country change easier to see?
- Could a reader overlook the scale difference?

Retain fixed scales for the required comparison unless you justify another choice clearly.

```
save_figure("maize-yield-trends.png", yield_trends)
```

5. Examine growing-season precipitation

Design a figure that answers one precipitation question. For example:

```
precipitation_plot <- ggplot(
  integrated,
  aes(x = growing_season_precipitation_mm)
) +
  geom_histogram(
    binwidth = 100,
    boundary = 0,
    colour = "white",
    fill = "#3B8B47",
    na.rm = TRUE
  ) +
```

```

facet_wrap(vars(project_country_name), ncol = 3) +
labs(
  title = "Growing-season precipitation distributions",
  subtitle = "October-April seasons ending in 1990-2022",
  x = "Country-area seasonal precipitation (mm)",
  y = "Seasons",
  caption = paste(
    "Source: CHIRPS v2 via ClimateSERV.",
    "Country-area estimates are not maize-field exposure."
  )
) +
theme_minimal(base_size = 10)

precipitation_plot

```

Inspect sensitivity to bin width and shared versus free scales, and explain why the value should not be described simply as “rainfall received by maize.”

```

save_figure("growing-season-precipitation.png", precipitation_plot)

```

6. Explore yield and precipitation together

Create a scatterplot in which one point represents one country-year:

```

yield_precipitation <- ggplot(
  integrated,
  aes(
    x = growing_season_precipitation_mm,
    y = yield_tonnes_per_hectare
  )
) +
  geom_point(
    colour = "#29508A",
    alpha = 0.55,
    size = 1.3,
    na.rm = TRUE
  ) +
  facet_wrap(vars(project_country_name), ncol = 3) +
  labs(
    title = "Maize yield and growing-season precipitation",
    subtitle = "Each point represents one country-year",
    x = "Country-area seasonal precipitation (mm)",
    y = "Maize yield (tonnes per hectare)",
    caption = paste(
      "Sources: FAOSTAT and CHIRPS v2 via ClimateSERV.",
      "The graphic describes association, not causation."
    )
  ) +
  theme_minimal(base_size = 10) +
  theme(panel.grid.minor = element_blank())

```

```
yield_precipitation
```

Compare this to a pooled plot coloured by country. Discuss:

- whether points overlap;
- whether the pooled view confounds country differences with within-country variation;
- whether a few high-yield countries dominate attention;
- what transparency makes visible; and
- which patterns require numerical description or modeling.

Do not add a fitted line merely as decoration: a smooth or regression line introduces method, functional-form, grouping, and uncertainty choices that become central in later sessions.

```
save_figure("yield-versus-precipitation.png", yield_precipitation)
```

7. Refine one communication graphic

Choose one exploratory result and state one intended comparison. The communication figure may build on the yield trends but must be reviewed as a separate artifact.

Use this refinement sequence:

1. Write a one-sentence message supported by visible evidence.
2. Identify the audience and final display size.
3. Remove encodings and decoration unrelated to the comparison.
4. Use fixed scales when comparing magnitudes across panels.
5. Add title, subtitle, units, source, grain, and limitation.
6. Check contrast, text size, panel labels, and grayscale readability.
7. Ask another learner to state the message without your explanation.
8. Revise the graphic if their interpretation differs from the intended one.

Do not use a title such as “Rainfall increases maize yield”; a defensible title describes the variables and scope, while the accompanying prose states a visible pattern and its uncertainty. Revise the title if your evidence supports a more precise, qualified statement.

Save the result separately:

```
communication_plot <- yield_trends +  
  labs(  
    title = "Maize-yield trajectories differ across countries",  
    subtitle = "Annual country-level observations, 1990-2022",  
    caption = paste(  
      "Source: fixed FAOSTAT teaching sample.",  
      "National observations do not show within-country variation."  
    )  
  )  
  
save_figure("maize-yield-communication.png", communication_plot)
```

8. Export and review the figure artifacts

Run the complete script from the project root:

```
Rscript scripts/visualize-maize-data.R
```

Confirm that all expected files exist and are non-empty:

```
ls -lh figures/*.png
```

Open each saved figure at its intended size. Review:

- title, subtitle, labels, units, caption, and source;
- clipping, overlap, contrast, text size, and facet scales;
- represented observations and missing values;
- consistency with the plot contract; and
- whether rerunning the script recreates the same outputs.

Inspect version-control status:

```
git status --short
git diff -- scripts/visualize-maize-data.R
```

Follow the project figure policy: exploratory artifacts need not all be retained, but their code and the selected communication figure should be reviewable, and temporary or manually exported duplicates should not be committed.

Write a short interpretation with this structure:

```
Figure purpose:
Visible pattern:
Population and grain:
Scale or aggregation choices:
What the figure does not establish:
Question for descriptive analysis:
```

Independent extension

Choose one extension and justify every new visual choice:

1. Compare a physical and log-yield scale, and explain how equal distances change meaning.
2. Build an accessible two-country comparison using both colour and line type.
3. Compare country-level rainfall distributions using aligned dot plots instead of histograms.
4. Investigate one unusual observation using the FAOSTAT flag and source documentation, without deleting it.
5. Export a vector PDF of the communication figure and compare its suitability with PNG.

Add a short note on what the extension reveals, which choices it depends on, and why it does not change the claim boundary.

Troubleshooting

| Problem | Fix |
|--|---|
| Derived input missing | Run the validated preparation and integration scripts in order; do not point the plotting script at an undocumented substitute. |
| Line connects different countries | Check grouping — use <code>group = country</code> , map country to an aesthetic, or facet so each panel holds one country. |
| Histogram changes with bin width | Expected near the bin scale; compare reasonable widths rather than treating one as definitive. |
| Points appear missing | Check missing values, axis limits, transformations, and overplotting; count rows supplied to the geometry. |
| Facets look similar despite different values | Check whether <code>scales = "free_y"</code> is active — free scales support shape comparison, not magnitude comparison. |
| Labels or panels clipped in the PNG | Increase dimensions, shorten labels, or revise the layout; higher DPI alone adds no layout space. |
| Script changes the input data | Stop — visualization must read derived data and write figures, not overwrite managed or derived inputs. |

Completion checklist

- ☐ Every figure begins with a stated question and audience.
- ☐ Input artifact, population, grain, variables, and units are documented.
- ☐ One plotted mark or aggregate can be described precisely.
- ☐ Distribution results were checked across reasonable bin choices.
- ☐ Trend grouping connects observations only within countries.
- ☐ Fixed and free facet scales were compared deliberately.
- ☐ Overplotting and missingness were inspected.
- ☐ Titles, axes, units, captions, and sources are readable.
- ☐ Important distinctions do not depend on colour alone.
- ☐ The script recreates figures through project-relative paths.
- ☐ The communication graphic supports one qualified message.
- ☐ Interpretation distinguishes pattern, association, and causation.
- ☐ One question is carried forward to Descriptive Data Analysis.

Reflect on the application

1. Which visual question was easiest to formulate, and which remained ambiguous?
2. What does one mark represent in each of your figures, and what was lost when observations were aggregated?
3. Which histogram features remained stable across bin widths, and how did fixed versus free facet scales change country comparisons?

4. What did transparency or faceting reveal about overlapping observations?
 5. How did you make the communication figure accessible, and which title did you reject because it overstated the evidence?
 6. What can the yield-precipitation scatterplot show and not show, and which visible pattern should be quantified next?
-

Further resources

- [R for Data Science \(2e\): Data visualization](#)
- [ggplot2 reference](#)
- Claus O. Wilke, [Fundamentals of Data Visualization](#)
- Kieran Healy, [Data Visualization: A Practical Introduction](#)
- [The Turing Way: Data visualisation](#)
- [Viridis colour scales](#)
- [W3C: Images tutorial](#)

8.1) Why describe data numerically?

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Visual patterns need numerical descriptions
 - A single average is not a description
 - Descriptions depend on scope
 - Data-generating processes can change
 - Why stationarity matters before modeling
 - What can go wrong
 - How this connects to the maize-yield project
 - Initial activity
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- explain how numerical descriptions complement visualization;
 - identify why sample size, population, grouping, period, and missingness must accompany a statistic;
 - explain why location, dispersion, and distribution shape belong together;
 - distinguish a stable-looking series from a claim that its process is stationary;
 - describe why changing levels, variation, or associations matter for later models; and
 - formulate descriptive questions for the maize-yield and precipitation data.
-

Place in the session

This is the **Motivation** part of the Descriptive Data Analysis session:

Motivation → Concepts → Application
↑
this page

Data Visualization made distributions, country differences, temporal change, and possible yield-precipitation relationships visible. Descriptive Data Analysis now asks how to quantify those patterns without reducing them to a misleading single number.

[Understand descriptive-data-analysis concepts](#) develops the required measures and the idea of stationarity. [Describe maize yield and precipitation](#) applies them in the example project.

Visual patterns need numerical descriptions

A plot can make a pattern visible, but readers may still need precise answers: how many observations support it, what a typical value is, how strongly values vary, how asymmetric the distribution is, how different countries or periods are, and how strongly two variables vary together.

Numerical summaries make such comparisons explicit and reproducible, and can reveal information that is hard to judge visually, such as an exact median, an interquartile range, or the number of missing observations.

```
visualization: reveal structure and exceptions
description:   quantify selected features of that structure
interpretation: connect both to scope, assumptions, and limitations
```

Neither is sufficient by itself. Different distributions can share a mean and standard deviation, while the same data can look different under a changed visual scale. Inspect observations and calculate relevant summaries.

A single average is not a description

Suppose two countries have the same mean maize yield. One may have relatively stable annual values; the other may alternate between very low and very high values. The common mean conceals a difference that matters to farmers, planners, and later models.

A useful description normally combines:

- **coverage:** observation count, time range, and missingness;
- **location:** mean, median, or selected quantiles;
- **dispersion:** standard deviation, interquartile range, or range; and
- **shape and exceptions:** skewness, boundaries, clusters, and unusual values.

The right combination depends on the question and distribution: the mean uses every value but is sensitive to extremes; the median resists extremes but says nothing about variability; standard deviation pairs naturally with the mean but can mislead for skewed data; quantiles are robust but omit detail. The objective is a small complementary set that answers a stated question, not every statistic software makes available.

Descriptions depend on scope

A statistic describes the observations that entered its calculation. Its meaning changes with the target population and observed sample, the unit represented by one row, filters and inclusion rules, groups and time periods, weighting choices, and the treatment of missing values.

An overall mean across all country-years is not the mean for a typical country unless each contributes equally and that weighting matches the question; a mean national yield is not the yield of a typical farm; and a correlation pooled across countries can differ from correlations calculated within individual countries.

Always report an effective sample size, and state how many observations remain after removing missing values. A precisely calculated statistic with unclear scope is not a meaningful result.

Data-generating processes can change

Many introductory examples treat observations as if they came from an unchanging process, but real food-system data can violate that assumption. Yields may change because of technology, seed varieties, irrigation, input access, policy, reporting practices, land use, or climate; precipitation can display cycles, persistent anomalies, or changing variability. A later period may show a different typical yield, wider or narrower variation, a changed distribution shape, a structural break, or a changed yield-precipitation association. These changes are part of the description, not nuisances to hide, and may determine whether historical evidence is relevant for a later period.

Why stationarity matters before modeling

Stationarity concerns whether the probabilistic properties of a process remain stable over time — whether level, variation, distribution, and temporal dependence appear comparable across the period being studied. It matters because later explanatory and predictive models learn relationships from observed data: if the process changes, a historical average may poorly represent a recent year, a relationship may not transfer across periods, a random train-test split can mix past and future and exaggerate performance, and a model may need explicit time, trend, season, or group terms.

Descriptive analysis cannot prove stationarity from a finite dataset, and non-stationarity does not make analysis impossible. The practical goal is to look for evidence of change, document it, and carry it into the modeling strategy — formal tests can help later but should not replace plots, subject knowledge, and explicit period comparisons.

What can go wrong

Common misreadings of a descriptive summary include pooled statistics that hide group differences (report group-specific summaries when groups matter), missing values dropped silently by default functions (report the effective observation count), correlation read as a causal claim (it quantifies linear co-variation only), and a trend mistaken for a stable relationship (two variables can correlate merely because both change over time). A formal stationarity test does not resolve these either — it depends on assumptions, specification, and a chosen null hypothesis, so its result is evidence within that setup, not a universal verdict.

How this connects to the maize-yield project

The example project contains 297 country-year observations for nine countries from 1990 through 2022, combining national maize yield (tonnes per hectare) and CHIRPS October-April country-area precipitation (millimetres).

The visualization session showed distributions, country trajectories, and paired yield-precipitation observations. This session quantifies them: confirming population, grain, coverage, and missingness; summarizing yield and precipitation location and dispersion by country and period (early, recent training, later test); comparing pooled and country-specific associations; and documenting evidence for or against stability over time.

The result is not an explanatory or predictive model, but an evidence base for deciding what such a model must represent and how it should be evaluated.

Initial activity

Return to the maize-yield trend and yield-precipitation figures and propose one statistic and one complementary graphic per question:

| Question | Statistic | Graphic |
|---|-----------|---------|
| What is a typical annual yield? | | |
| How variable is yield? | | |
| Did yield change between periods? | | |
| Do yield and precipitation vary together? | | |

For every proposed statistic, write its population, grouping, period, unit, and effective observation count. Discuss what the statistic would omit.

Check your understanding

1. Why should a numerical summary be inspected with a visualization?
 2. Why is the mean alone an incomplete description?
 3. How can grouping change the meaning of a statistic?
 4. What kinds of change provide evidence against stationarity?
 5. Why can a finite dataset not prove that a process is stationary?
-

Further resources

- [OpenIntro Statistics](#) — distributions, summary measures, and statistical reasoning.
 - [R for Data Science \(2e\): Exploratory data analysis](#) — questions, variation, and covariation in R.
 - [Forecasting: Principles and Practice — Stationarity](#) — an accessible introduction to time-series stationarity.
 - [The Turing Way: Research Data Management](#) — documented, reproducible analysis practice.
 - [NIST/SEMATECH e-Handbook of Statistical Methods](#) — a broad reference for exploratory statistics.
-

Continue to Concepts

Continue with [Understand descriptive-data-analysis concepts](#). It explains observation scope, coverage, location, dispersion, shape, association, temporal dependence, and stationarity.

8.2) Understand descriptive-data-analysis concepts

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Start with a descriptive contract
 - Define population, sample, and observation grain
 - Report coverage and missingness
 - Describe location
 - Arithmetic mean
 - Median and quantiles
 - Describe dispersion
 - Range and interquartile range
 - Variance and standard deviation
 - Describe distribution shape
 - Compare groups and periods deliberately
 - Quantify association
 - Separate pooled and within-group association
 - Account for ordered observations
 - Understand stationarity
 - Diagnose change descriptively
 - Treat formal tests as optional evidence
 - Connect description to modeling decisions
 - Check your understanding
 - Further resources
 - Descriptive analysis and association
 - Time and stationarity
 - Reproducible interpretation
 - Continue to Application
-

Learning objectives

After completing this page, you should be able to:

- define the population, sample, grain, grouping, period, and denominator of a summary;
 - report coverage and missingness alongside descriptive measures;
 - select complementary measures of location, dispersion, and shape;
 - distinguish covariance from correlation and association from causation;
 - explain why pooled and within-group associations can differ;
 - define stationarity and weak stationarity at an introductory level;
 - identify evidence of trends, changing variance, structural change, seasonality, and association drift; and
 - translate descriptive findings into requirements for later models.
-

Place in the session

This is the **Concepts** part of the Descriptive Data Analysis session:

Motivation → Concepts → Application
 ↑
 this page

[Why describe data numerically?](#) explains why precise summaries and temporal stability matter. This page provides the tools used in [the maize descriptive-analysis application](#).

Use one central question throughout:

Which observations does this statistic describe, which feature does it summarize, and what important structure can it conceal?

Start with a descriptive contract

Define a summary before calculating it:

| Contract element | Question |
|-----------------------|--|
| Analytical question | Which feature or comparison should be quantified? |
| Population and sample | About which observations is the statement intended? Which were observed? |
| Grain and key | What does one row represent, and what identifies it? |
| Variable and unit | What is measured and in which unit? |
| Group and period | Which observations are summarized together? |
| Missing-value rule | Which values are unavailable, and how are they handled? |
| Measure | Why is this statistic appropriate? |
| Companion evidence | Which count, alternative measure, or graphic prevents misinterpretation? |
| Claim boundary | What does the result not establish? |

This contract prevents a software function from defining the question by accident. It also makes outputs reviewable and comparable across analysts.

Define population, sample, and observation grain

The **target population** is the set of units or events about which a statement is intended; the **observed sample** is the subset represented in the data. A census-like administrative dataset can still cover only selected variables, periods, or reporting units.

The **observation grain** states what one row represents. In the integrated maize data, one row represents one selected country and one year, with a growing season ending in that year — not an individual farm, field, household, or weather station.

The grain determines valid denominators and weights. An unweighted mean across country-years gives every available country-year the same influence; it does not weight by harvested area, production, population, or country size. Such a mean can be correct for one question and inappropriate for another.

Report coverage and missingness

Begin every descriptive table with coverage: total rows considered, non-missing and missing values, number of groups, first and last observed period, and gaps or duplicated keys.

Let n be the number of non-missing observations used by a statistic. Report it even if the full dataset has a known row count — pairing two variables can reduce the effective n because both values must be present.

Missingness can itself vary by group or time, so a changing mean may partly reflect changing coverage rather than changing values. Describing only complete cases is insufficient when excluded observations are systematic.

Describe location

Measures of location describe a distribution's centre or typical value.

Arithmetic mean

For values $x_1, \dots, x_n$, the sample mean is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The mean uses every value and supports many later methods, but is sensitive to extreme values, skewness, and weighting choices.

Median and quantiles

The median is the middle ordered value, or the average of the two middle values for an even count. It resists extremes but does not use the magnitude of every deviation. The p -quantile divides ordered observations so that approximately a proportion p lies at or below it; the first and third quartiles correspond to 25% and 75% (algorithms can differ slightly for small samples, so use a consistent implementation).

Report mean and median together when skewness or unusual observations may matter — their difference is a diagnostic, not a full measure of shape.

Describe dispersion

Dispersion measures how widely observations vary.

Range and interquartile range

The range is the maximum minus the minimum: easy to interpret, but it depends entirely on two observations and usually grows with sample size. The interquartile range describes the middle half of values, $IQR = Q_{0.75} - Q_{0.25}$, and is resistant to extremes.

Variance and standard deviation

The sample variance is:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

The standard deviation is $s = \sqrt{s^2}$, returning to the variable's unit. Both are sensitive to extreme values; standard deviation is not the average absolute distance from the mean and does not define a universal interval containing a fixed percentage of observations. Pair standard deviation with the mean and IQR with the median — both pairs help when a distribution is asymmetric or heavy-tailed.

Describe distribution shape

Location and dispersion cannot uniquely identify a distribution. Also inspect symmetry or skewness (whether one tail extends further), modality (whether values form one or several concentrations), boundaries (whether measurement or definition imposes limits), tails (whether extreme values occur frequently), and gaps or heaping (whether rounding, thresholds, or missing regions appear).

Numerical skewness and kurtosis exist, but plots and quantiles are often more interpretable for an introductory analysis. An observation beyond a boxplot whisker is not automatically an error — it is selected by a rule for further investigation.

Compare groups and periods deliberately

Group summaries expose structure hidden by pooled statistics. Use the same variables, units, missing-value rule, and measures across groups, and always report group-specific counts.

Period definitions should follow the analytical workflow rather than be chosen after seeing an attractive contrast. The maize project's modeling workflow already distinguishes 1990–2005 (earlier history), 2006–2017 (recent training history), and 2018–2022 (later test period). This split supports a descriptive check of whether the period reserved for later evaluation resembles the preceding training period. Differences can be absolute, percentage-based, or standardized, but every choice needs an interpretable denominator; avoid percentage change when the reference is zero or near zero.

Description alone does not determine whether a difference is substantively important. Compare magnitude with units, variation, domain knowledge, and the consequences of a decision.

Quantify association

For paired observations (x_i, y_i) , sample covariance is:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Its sign gives the direction of linear co-variation; its magnitude depends on both variables' units, making comparisons across scales difficult.

Pearson correlation standardizes covariance:

$$r = \frac{s_{xy}}{s_x s_y}$$

It lies between -1 and 1 when both standard deviations are positive. Values near -1 or 1 indicate strong negative or positive *linear* association; a value near zero indicates weak linear association, not necessarily no relationship.

Correlation can be unstable with small samples and sensitive to outliers. It must be reported with paired n and inspected with a scatterplot. It does not establish a causal effect, and its square is not automatically the proportion of variance explained outside a specified model.

Separate pooled and within-group association

A pooled correlation combines within-country variation with differences between country means. It may answer a different question from correlations calculated separately for each country.

Countries with higher average precipitation may also have higher average yield, producing a positive pooled relationship even if wetter years within each country are not consistently higher-yielding — or the reverse. This is related to aggregation effects and Simpson's paradox.

Compare the pooled scatterplot and correlation against country-faceted plots and country-specific correlations, period-specific correlations, and, if appropriate, deviations from country-specific means. These are descriptive views, not substitutes for a model that explicitly represents country and time.

Account for ordered observations

Country-year observations are ordered in time. Adjacent years can be more similar than distant years, so observations may not be independent.

Autocovariance and **autocorrelation** quantify association between a series and lagged versions of itself — a lag-one autocorrelation compares values one period apart, and its interpretation requires a regular time order and care around gaps.

Temporal dependence matters because the effective information can be less than the row count suggests, and it explains why shuffling observations into random training and test sets can leak temporal information and produce an unrealistic evaluation.

Understand stationarity

Strict stationarity means that the joint probability distribution of a process is unchanged by shifting time. That definition is demanding and cannot be established through a few summaries.

Weak stationarity is more limited. A process X_t is weakly stationary when:

1. its mean $E[X_t]$ is constant over time;
2. its variance $Var(X_t)$ is finite and constant over time; and
3. its covariance $Cov(X_t, X_{t-k})$ depends on lag k , not calendar time t .

Stationarity is a property of a data-generating process, not merely a table: a finite series can provide evidence consistent or inconsistent with it, but cannot prove universal stability. Different variables or groups can have different stability — yield may display a trend while deviations from that trend are more stable, and one country may show a structural change that another does not. State exactly which series, period, and property are under discussion.

Diagnose change descriptively

Look for several kinds of evidence:

| Pattern | Descriptive evidence | Why it matters |
|----------------------|---|--|
| Trend | Period means, medians, and time plots change | Constant mean may be implausible |
| Changing variance | Period SDs, IQRs, ranges, or residual spread differ | Constant variance may be implausible |
| Structural change | Abrupt persistent level or variability shift | One process may not describe all periods |
| Seasonality or cycle | Repeated pattern at meaningful lags | Dependence changes with cycle position |
| Association drift | Correlations or slopes differ by period or group | A learned relationship may not transfer |
| Coverage drift | Missingness or measurement practice changes | Apparent change may reflect observation change |

No single summary establishes stationarity — compare complementary statistics and plots across periods selected for substantive or workflow reasons, and record possible explanations from metadata and domain knowledge.

Treat formal tests as optional evidence

Tests such as the Augmented Dickey-Fuller or KPSS test use different null hypotheses and assumptions; results depend on trend terms, lag choices, sample size, structural breaks, and the variable tested, and failure to reject a null is not proof that it is true.

For this introductory session, formal testing is optional. The core task is to visualize each country series, compare location and dispersion across meaningful periods, inspect whether associations change, and describe consequences for later analysis. If a formal test is added as an extension, report its null hypothesis, specification, limitations, and relationship to the descriptive evidence.

Connect description to modeling decisions

Descriptive findings should produce an explicit handoff:

| Finding | Possible later response |
|--|---|
| Countries differ strongly in level | Represent country effects or stratify |
| Yield changes over time | Include time or trend; avoid assuming a constant mean |
| Variability differs by country or period | Review transformations or group-specific uncertainty |
| Yield-precipitation association differs by country | Consider interactions or partial pooling |
| Later period differs from training history | Use a time-aware split and qualify transferability |
| Observations are temporally dependent | Avoid independence assumptions and random leakage |

These are prompts for model design, not automatic prescriptions — later sessions must define whether the aim is explanatory or predictive and evaluate the relevant assumptions directly.

Check your understanding

1. Which elements belong in a descriptive contract?
2. How do target population, observed sample, and row grain differ?
3. Why must every statistic include an effective observation count?
4. Why do location and dispersion not fully describe distribution shape?
5. How can a pooled correlation differ from within-country correlations?
6. Why does temporal ordering challenge an independence assumption?
7. What three properties define weak stationarity?
8. How can descriptive findings change a later train-test strategy?

Further resources

Descriptive analysis and association

- [OpenIntro Statistics](#)
- [R for Data Science \(2e\): Exploratory data analysis](#)
- [NIST/SEMATECH e-Handbook: Exploratory Data Analysis](#)

Time and stationarity

- [Forecasting: Principles and Practice — Time-series features](#)
- [Forecasting: Principles and Practice — Stationarity and differencing](#)

Reproducible interpretation

- [The Turing Way: Research Data Management](#)
 - [The Turing Way: Data visualisation](#)
-

Continue to Application

Continue with [Describe maize yield and precipitation](#). You will define descriptive contracts, generate coverage and summary tables, compare periods and groups, quantify associations, assess evidence of change, and write a modeling handoff.

8.3) Describe maize yield and precipitation

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverables
 - Before you begin
 - 1. Define descriptive contracts
 - 2. Inspect coverage and denominators
 - 3. Describe maize yield by country
 - 4. Compare meaningful periods
 - 5. Describe precipitation
 - 6. Quantify yield-precipitation association
 - 7. Assess evidence about stability
 - 8. Write the modeling handoff
 - Independent extension
 - Option A: Within-country deviations
 - Option B: Rolling descriptions
 - Option C: Lag-one autocorrelation
 - Option D: Period sensitivity
 - Troubleshooting
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- define the population, grain, groups, periods, missing-value rules, and measures for a descriptive analysis;
 - calculate reproducible coverage, location, dispersion, and quantile summaries with R;
 - compare pooled, country-specific, and period-specific results;
 - calculate covariance and Pearson correlation with paired observation counts;
 - inspect trends, changing variation, and association drift as evidence relevant to stationarity;
 - distinguish descriptive association from causal or predictive claims; and
 - translate descriptive findings into requirements for later modeling and evaluation.
-

Place in the session

This is the **Application** part of the Descriptive Data Analysis session:

```
Motivation → Concepts → Application
                        ↑
                    this page
```

Before beginning, review [Why describe data numerically?](#) and [Understand descriptive-data-analysis concepts](#).

The preceding visualization topic created reproducible figures from prepared and integrated data. This exercise quantifies selected visual patterns. It does not alter the input datasets or fit an explanatory or predictive model.

Scenario and deliverables

The project team must answer:

How do maize yield and growing-season precipitation vary across countries and periods, how stable do their distributions and association appear, and what must later models represent?

Create these artifacts:

```
scripts/describe-maize-data.R
results/tables/descriptive-coverage.csv
results/tables/maize-yield-summary.csv
results/tables/maize-yield-period-summary.csv
results/tables/precipitation-summary.csv
results/tables/yield-precipitation-association.csv
results/tables/stationarity-diagnostic.csv
results/descriptive-modeling-handoff.md
```

The tables provide machine-readable evidence. The Markdown handoff explains what that evidence means, what it does not establish, and how it affects later sessions.

Before you begin

Work from the standalone `maize-yield-project` repository. Check your location, branch, and working tree:

```
pwd
git status --short --branch
```

Restore the environment and recreate the offline inputs:

```
Rscript scripts/setup.R
Rscript scripts/validate-data.R
Rscript scripts/prepare-maize-data.R
Rscript scripts/integrate-data.R
Rscript scripts/visualize-maize-data.R
```


Review these files:

| Role | File |
|--------------------------------------|---|
| Integrated data | data/derived/maize-yield-with-precipitation.csv |
| Data documentation | docs/data/maize-yield-with-precipitation.md |
| Data dictionary | metadata/maize-yield-with-precipitation-data-dictionary.csv |
| Preparation and integration evidence | results/tables/data-preparation-audit.csv,
results/tables/data-integration-audit.csv |
| Visualization evidence | results/tables/data-visualization-manifest.csv
and figures/ |

The expected input has 297 unique country-year rows: nine countries observed annually from 1990 through 2022. Confirm that upstream audits pass.

Start `scripts/describe-maize-data.R` with explicit setup and checks:

```
# Create reproducible descriptive summaries of maize data.

source("scripts/functions.R")
assert_project_root()
ensure_project_directories()
check_required_packages(c("dplyr", "here", "readr", "tidyr"))

library(dplyr)
library(here)
library(readr)
library(tidyr)

input_file <- here(
  "data", "derived", "maize-yield-with-precipitation.csv"
)
maize <- read_csv(input_file, show_col_types = FALSE)

required_columns <- c(
  "project_country_id", "project_country_name", "year",
  "yield_tonnes_per_hectare", "growing_season_precipitation_mm"
)
missing_columns <- setdiff(required_columns, names(maize))

if (length(missing_columns) > 0) {
  stop("Missing column(s): ", paste(missing_columns, collapse = ", "))
}
if (nrow(maize) != 297L ||
  anyDuplicated(maize[c("project_country_id", "year")])) {
  stop("Expected 297 unique project-country-year rows.")
}

maize <- maize |>
  mutate(
    analysis_period = case_when(
      year <= 2005 ~ "1990-2005: earlier history",
      year <= 2017 ~ "2006-2017: recent training",
      TRUE ~ "2018-2022: later test"
    )
  )
```

```
)
)
```

These periods follow the existing modeling workflow. Do not silently change them after seeing the results.

1. Define descriptive contracts

Before coding, complete this table:

| Output | Population and grain | Group or period | Measures | Missing-value rule | Claim boundary |
|-----------------------|--|-------------------------|---|--------------------|---------------------------------------|
| Coverage | Nine project countries; one country-year | Country and full period | Rows, available values, first/last year | Count explicitly | Does not establish representativeness |
| Yield summary | Define this | Define this | Define this | Define this | Define this |
| Period summary | Define this | Define this | Define this | Define this | Define this |
| Precipitation summary | Define this | Define this | Define this | Define this | Define this |
| Association | Paired yield and precipitation | Define this | Covariance, correlation, paired n | Complete pairs | Not causal |
| Stability diagnostic | Define this | Three fixed periods | Define this | Define this | Evidence, not proof |

State units and explain why selected measures complement one another. Do not add measures merely because software makes them easy to calculate.

2. Inspect coverage and denominators

Create one row per country:

```
coverage <- maize |>
  group_by(project_country_id, project_country_name) |>
  summarise(
    total_rows = n(),
    first_year = min(year),
    last_year = max(year),
    distinct_years = n_distinct(year),
    non_missing_yield = sum(!is.na(yield_tonnes_per_hectare)),
    missing_yield = sum(is.na(yield_tonnes_per_hectare)),
    non_missing_precipitation =
      sum(!is.na(growing_season_precipitation_mm)),
    missing_precipitation =
      sum(is.na(growing_season_precipitation_mm)),
```

```

    .groups = "drop"
  )

write_csv(
  coverage,
  here("results", "tables", "descriptive-coverage.csv"),
  na = ""
)

```

Ask whether every country contributes the same years and whether yield and precipitation are available for the same rows. Equal country-year coverage does not imply coverage of farms or within-country conditions.

3. Describe maize yield by country

Create complementary summaries:

```

yield_summary <- maize |>
  group_by(project_country_id, project_country_name) |>
  summarise(
    n = sum(!is.na(yield_tonnes_per_hectare)),
    mean_t_per_ha = mean(yield_tonnes_per_hectare, na.rm = TRUE),
    median_t_per_ha = median(yield_tonnes_per_hectare, na.rm = TRUE),
    sd_t_per_ha = sd(yield_tonnes_per_hectare, na.rm = TRUE),
    q25_t_per_ha = quantile(yield_tonnes_per_hectare, 0.25, na.rm = TRUE),
    q75_t_per_ha = quantile(yield_tonnes_per_hectare, 0.75, na.rm = TRUE),
    iqr_t_per_ha = IQR(yield_tonnes_per_hectare, na.rm = TRUE),
    minimum_t_per_ha = min(yield_tonnes_per_hectare, na.rm = TRUE),
    maximum_t_per_ha = max(yield_tonnes_per_hectare, na.rm = TRUE),
    .groups = "drop"
  )

write_csv(
  yield_summary,
  here("results", "tables", "maize-yield-summary.csv"),
  na = ""
)

```

Compare each row with the corresponding time-series panel: where mean and median differ, where SD and IQR tell different stories, and whether a wide range reflects persistent variation or one unusual year. These are national country-year descriptions, not farm-level summaries.

4. Compare meaningful periods

Calculate the same core measures by country and predefined period:

```

yield_period_summary <- maize |>
  group_by(project_country_id, project_country_name, analysis_period) |>
  summarise(
    first_year = min(year),
    last_year = max(year),
    n = sum(!is.na(yield_tonnes_per_hectare)),
    mean_t_per_ha = mean(yield_tonnes_per_hectare, na.rm = TRUE),
    median_t_per_ha = median(yield_tonnes_per_hectare, na.rm = TRUE),
    sd_t_per_ha = sd(yield_tonnes_per_hectare, na.rm = TRUE),
    iqr_t_per_ha = IQR(yield_tonnes_per_hectare, na.rm = TRUE),
    .groups = "drop"
  )

write_csv(
  yield_period_summary,
  here("results", "tables", "maize-yield-period-summary.csv"),
  na = ""
)

```

Compare early with recent training history, then recent training with the later test period; the test period has only five annual observations per country, so its SD and IQR are sensitive to individual years. Ask whether typical yield shifts in the same direction across countries, whether dispersion changes with location, and whether recent training history resembles the later evaluation period. These comparisons provide evidence about stability; they do not prove stationarity.

5. Describe precipitation

Use the same reporting discipline:

```

precipitation_summary <- maize |>
  group_by(project_country_id, project_country_name, analysis_period) |>
  summarise(
    n = sum(!is.na(growing_season_precipitation_mm)),
    mean_mm = mean(growing_season_precipitation_mm, na.rm = TRUE),
    median_mm = median(growing_season_precipitation_mm, na.rm = TRUE),
    sd_mm = sd(growing_season_precipitation_mm, na.rm = TRUE),
    q25_mm = quantile(growing_season_precipitation_mm, 0.25, na.rm = TRUE),
    q75_mm = quantile(growing_season_precipitation_mm, 0.75, na.rm = TRUE),
    iqr_mm = IQR(growing_season_precipitation_mm, na.rm = TRUE),
    .groups = "drop"
  )

write_csv(
  precipitation_summary,
  here("results", "tables", "precipitation-summary.csv"),
  na = ""
)

```

Precipitation is the October-April total of country-area mean daily CHIRPS estimates, not rainfall measured at maize fields. Do not infer farm exposure or agronomic thresholds from this summary.

6. Quantify yield-precipitation association

Create complete pairs and calculate pooled, country-specific, and period-specific covariance and correlation. A reusable helper keeps the definition consistent:

```
association_summary <- function(data) {
  paired <- data |>
    filter(
      !is.na(yield_tonnes_per_hectare),
      !is.na(growing_season_precipitation_mm)
    )

  tibble(
    n_pairs = nrow(paired),
    covariance = cov(
      paired$growing_season_precipitation_mm,
      paired$yield_tonnes_per_hectare
    ),
    pearson_correlation = cor(
      paired$growing_season_precipitation_mm,
      paired$yield_tonnes_per_hectare
    )
  )
}

pooled_association <- association_summary(maize) |>
  mutate(scope = "pooled", group = "all", .before = 1)

country_association <- maize |>
  group_by(project_country_id, project_country_name) |>
  group_modify(~ association_summary(.x)) |>
  ungroup() |>
  mutate(scope = "within country", .before = 1)

period_association <- maize |>
  group_by(analysis_period) |>
  group_modify(~ association_summary(.x)) |>
  ungroup() |>
  mutate(scope = "within period", .before = 1)

association <- bind_rows(
  pooled_association,
  country_association,
  period_association
)

write_csv(
  association,
  here("results", "tables", "yield-precipitation-association.csv"),
  na = ""
)
```

If either variable has zero variance, correlation is undefined; preserve NA and explain it. Compare pooled results with faceted graphics and within-country results, and discuss country differences, common trends, irrigation, temperature, inputs, spatial aggregation, measurement error, and omitted variables. Do not call correlation an effect.

7. Assess evidence about stability

Create a compact comparison of recent training and later test periods. Select the two rows per country, reshape them, and calculate:

```
stationarity_diagnostic <- yield_period_summary |>
  filter(analysis_period != "1990-2005: earlier history") |>
  select(
    project_country_id, project_country_name, analysis_period,
    n, mean_t_per_ha, sd_t_per_ha
  ) |>
  pivot_wider(
    names_from = analysis_period,
    values_from = c(n, mean_t_per_ha, sd_t_per_ha),
    names_sep = "__"
  ) |>
  mutate(
    mean_change_t_per_ha =
      `mean_t_per_ha__2018-2022: later test` -
      `mean_t_per_ha__2006-2017: recent training`,
    sd_ratio =
      `sd_t_per_ha__2018-2022: later test` /
      `sd_t_per_ha__2006-2017: recent training`
  )

write_csv(
  stationarity_diagnostic,
  here("results", "tables", "stationarity-diagnostic.csv"),
  na = ""
)
```

The mean change retains yield units. The SD ratio can be unstable with a small or near-zero denominator, and the test period contains only five values — do not turn these measures into universal thresholds.

Review them with the full time-series plots, all period summaries, missingness, source documentation, and period-specific associations. For each country, describe evidence as “no clear change visible,” “possible level change,” “possible variance change,” or “insufficient evidence,” rather than “stationary: yes/no.” Formal tests are optional and require a documented null hypothesis, trend and lag specification, and limitations.

8. Write the modeling handoff

Create `results/descriptive-modeling-handoff.md` with:

```
# Descriptive modeling handoff

## Scope and data
## Coverage
## Yield location, dispersion, and shape
## Period stability
## Precipitation
## Yield-precipitation association
## Implications for explanatory modeling
## Implications for predictive modeling
## Limitations and unresolved questions
```

Support statements with named tables or figures. Include a group or period difference, evidence relevant to stationarity, the difference between pooled and within-country association, a reason for a time-aware split, structures a later explanatory model may need, threats to predictive transferability, and limitations of national FAOSTAT and country-area CHIRPS measures.

Do not select a model yet. State requirements and open questions. Rerun the script from a clean R session and inspect every artifact:

```
Rscript scripts/describe-maize-data.R
git status --short
```

Independent extension

Choose one extension and document its question, implementation, result, and limitations.

Option A: Within-country deviations

Subtract each country's mean from yield and precipitation, then correlate the deviations. Compare with the pooled correlation and explain which differences this removes and which temporal confounding remains.

Option B: Rolling descriptions

For one country, calculate a rolling mean and SD using a documented window. Explain incomplete initial windows and how window length changes the pattern.

Option C: Lag-one autocorrelation

For each country, correlate yield at year t with year $t - 1$. Verify consecutive years before pairing. Discuss sample size and why autocorrelation does not identify a mechanism.

Option D: Period sensitivity

Repeat the comparison with one substantively justified alternative split. State the justification before calculation and identify robust and sensitive conclusions.

Troubleshooting

- **A summary returns NaN:** the group may contain no non-missing observations — count available values and preserve missing results rather than replacing them.
 - **Correlation returns NA:** check complete pairs and whether either variable has zero variance.
 - **Results differ from a figure:** check input, filters, grouping, period labels, units, and missing-value rules; confirm figure and table share the same grain.
 - **Pooled and country-specific correlations have different signs:** this is possible and important — inspect country means, faceted plots, trends, and aggregation rather than retaining only the preferred result.
 - **The later-period SD changes greatly:** the period contains only five observations per country; inspect raw values and report the small denominator.
 - **A formal test contradicts descriptive evidence:** review its null hypothesis, deterministic terms, lag selection, sample size, and sensitivity to structural breaks, and report the disagreement.
 - **The script overwrites analytical data:** stop. Descriptive analysis reads derived data and writes results; it must not modify `data/managed/` or `data/derived/`.
-

Completion checklist

- ☐ Population, sample, grain, groups, periods, variables, and units are stated.
 - ☐ Coverage and missingness accompany every analytical scope.
 - ☐ Every statistic reports an effective observation or pair count.
 - ☐ Mean and median are interpreted with SD, IQR, range, and graphics.
 - ☐ Country-specific summaries are compared with pooled results.
 - ☐ Period definitions follow the documented workflow.
 - ☐ Yield and precipitation preserve their measurement contracts.
 - ☐ Covariance and correlation are interpreted as association, not causation.
 - ☐ Pooled, within-country, and period-specific associations are compared.
 - ☐ Stationarity evidence includes level, variation, time plots, and association drift.
 - ☐ No diagnostic is presented as proof of stationarity.
 - ☐ The later test period's small sample size is visible.
 - ☐ One project-relative script recreates all tables.
 - ☐ The handoff distinguishes explanatory and predictive implications.
 - ☐ Managed and derived inputs remain unchanged.
-

Reflect on the application

1. Which measures best represented each country's yield distribution?
 2. Which finding was hidden by a pooled summary?
 3. Where was there evidence of changing level or variation?
 4. What appeared stable, and why is that not proof of stationarity?
 5. How did pooled and within-country associations differ?
 6. Why is a time-aware test period preferable to a random split?
 7. Which finding threatens predictive transferability?
-

Further resources

- [R for Data Science \(2e\): Data transformation](#)
- [dplyr::summarise\(\)](#) reference
- [OpenIntro Statistics](#)
- [NIST/SEMATECH e-Handbook: Exploratory Data Analysis](#)
- [Forecasting: Principles and Practice — Stationarity and differencing](#)
- [The Turing Way: Data visualisation](#)

9.1) Why explanatory modeling requires causal reasoning

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Description is not explanation
 - A regression coefficient does not create causality
 - Causal questions compare potential outcomes
 - Research design comes before estimation
 - The precipitation intervention is difficult to define
 - What can go wrong
 - How this connects to the maize-yield project
 - Initial activity
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- distinguish descriptive association from causal explanation;
 - explain why regression adjustment alone does not identify a causal effect;
 - formulate a causal question with a target population, exposure contrast, outcome, and time horizon;
 - identify ambiguities in a proposed precipitation intervention; and
 - describe what the maize data can contribute even when a causal effect is not identified.
-

Place in the session

This is the **Motivation** part of the Explanatory Modeling session:

```
Motivation → Concepts → Application
      ↑
    this page
```

Descriptive Data Analysis quantified distributions, temporal changes, and yield-precipitation associations, deliberately avoiding causal language. This session asks whether and under which assumptions an observed relationship can support a causal explanation.

[Concepts](#) separates causal questions, identification, and estimation. [Application](#) applies that framework to the example project.

Description is not explanation

A descriptive analysis can show that two variables vary together in the observed data — for example, that maize yield differs between wetter and drier country-years, or that the association differs across countries and periods. These findings do not answer what would happen if precipitation changed while other conditions stayed comparable.

Several distinct processes can produce the same observed association:

- precipitation may affect plant growth and therefore yield;
- temperature and atmospheric conditions may affect both rainfall and yield;
- countries may differ in climate, soils, irrigation, inputs, varieties, and reporting;
- both precipitation and yield may change over time; or
- measurement, aggregation, or selection into the dataset may distort the comparison.

Explanatory modeling must distinguish these possibilities through a causal question, a defensible research design, explicit assumptions, and a matching estimator.

A regression coefficient does not create causality

Linear regression describes how an outcome differs with an explanatory variable while holding included variables constant — a conditional association, not automatically a causal effect. It becomes causal only if additional conditions justify treating the modeled comparison as the relevant counterfactual. Adding country indicators, a time trend, or many available variables does not automatically satisfy those conditions.

An adjustment variable can help, do nothing, or introduce bias, depending on its causal role:

- a **confounder** is a common cause of exposure and outcome and may need adjustment;
- a **mediator** lies on the causal pathway and should not be adjusted for when estimating a total effect;
- a **collider** is caused by two other variables, and conditioning on it can open a biased path;
- a **proxy** partially represents an unmeasured causal factor; and
- a **precision variable** predicts the outcome without removing confounding.

These roles cannot be learned from a regression table alone — they require temporal ordering, substantive knowledge, and an explicit causal model.

Causal questions compare potential outcomes

A causal effect compares outcomes under alternative conditions for the same target units. Let $Y_i(p)$ denote the potential maize yield for unit i under precipitation exposure p . A unit-level contrast might be:

$$Y_i(p + 100) - Y_i(p)$$

Only one of these potential outcomes can be observed for a given country-year — the alternative is counterfactual. Causal inference therefore uses observed outcomes from other units or times to construct a valid comparison under assumptions.

A useful causal question names:

- **target population:** which countries, years, farms, or fields;
- **exposure and comparison:** the contrasted conditions or policies;
- **outcome:** the measure and time at which it is evaluated;
- **estimand:** the population-level causal quantity of interest; and
- **time zero and follow-up:** when exposure and outcome measurement begin.

Without these elements, “the effect of rainfall on yield” is too ambiguous for one model coefficient to answer.

Research design comes before estimation

The order of a causal analysis should be:

```
causal question and target population
      ↓
exposure contrast and causal estimand
      ↓
causal structure and identification assumptions
      ↓
observed-data representation and adjustment strategy
      ↓
estimator, uncertainty, diagnostics, and sensitivity analysis
      ↓
conclusion bounded by the design and assumptions
```

Beginning with a regression formula reverses this logic: it encourages interpreting whichever coefficient software returns rather than asking whether that coefficient estimates the intended effect. A model can fit the data well while the causal design remains invalid; an honest analysis concluding that the data do not identify the effect is more useful than an unsupported causal claim.

The precipitation intervention is difficult to define

The example project records October-April country-area precipitation totals. A proposed contrast of “100 mm more precipitation” leaves several questions unresolved: rain timing and intensity, whether it falls over maize-growing land, whether it reflects natural variation or irrigation, and whether the same contrast is meaningful in every country and baseline climate.

Different versions can produce different yield outcomes. This is a **consistency** problem: one numeric exposure value may not correspond to one well-defined treatment condition.

The exposure is also spatially aggregated: national yield and country-area precipitation do not show whether maize fields actually received the recorded rain. A stronger analysis would need field-level or remote-sensing measurements aligned to the growing season.

What can go wrong

- **Significance mistaken for causality:** a small p-value does not establish exchangeability, correct measurement, or a well-defined intervention.
- **Every available variable treated as a control:** data-driven adjustment can include mediators or colliders while omitting important common causes.
- **Country/year indicators called complete adjustment:** they capture stable country differences and common trends, not every changing confounder.
- **Diagnostics treated as causal diagnostics:** residual plots reveal functional-form problems, not whether unmeasured confounding is absent.
- **Precision mistaken for identification:** a narrow confidence interval can coexist with severe confounding, measurement error, or exposure ambiguity.

How this connects to the maize-yield project

The project provides 297 observations for nine countries from 1990 through 2022, each combining national maize yield with a country-area seasonal precipitation estimate — an instructive but limited causal-analysis case. Learners will:

1. define a provisional precipitation-yield estimand and draw a causal diagram;
2. evaluate consistency, exchangeability, positivity, interference, and measurement;
3. compare unadjusted, country-adjusted, time-adjusted, and combined regressions;
4. inspect residuals, influential observations, and functional-form sensitivity; and
5. decide which interpretation the evidence supports.

The likely defensible result is an adjusted association together with a clear statement that the dataset does not identify a causal precipitation effect — valuable because it makes this boundary and the required next data explicit.

Initial activity

Complete this causal-question template before reading the Concepts page:

| Element | Provisional definition | Remaining ambiguity |
|-------------------------|------------------------|---------------------|
| Target population | | |
| Exposure | | |
| Comparison exposure | | |
| Outcome | | |
| Time zero and follow-up | | |
| Average causal effect | | |

Then list three variables that may cause both precipitation exposure and yield, and state whether the project measures each one adequately.

Check your understanding

1. How does a descriptive association differ from a causal effect?
 2. Why does adding variables to a regression not automatically remove bias?
 3. What elements make a causal question sufficiently precise?
 4. Why is “100 mm more precipitation” not necessarily one treatment?
 5. Why can failure to identify a causal effect be a useful scientific conclusion?
-

Further resources

- Hernán and Robins, [Causal Inference: What If](#) — causal questions, potential outcomes, identification, and estimation.
 - Cunningham, [Causal Inference: The Mixtape](#) — causal designs and assumptions through applied examples.
 - Facure, [Causal Inference for the Brave and True](#) — an accessible applied introduction.
 - [DAGitty](#) — drawing and evaluating causal diagrams.
 - [The Turing Way: Reproducible Research](#) — transparent, reviewable research workflows.
-

Continue to Concepts

Continue with [Understand explanatory-modeling concepts](#). It distinguishes causal estimands, identification assumptions, regression estimators, uncertainty, diagnostics, and sensitivity analysis.

9.2) Understand explanatory-modeling concepts

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Separate question, estimand, identification, and estimation
 - Use potential outcomes to define causal effects
 - Make causal structure explicit
 - Distinguish variable roles
 - Evaluate identification assumptions
 - Consistency
 - Exchangeability
 - Positivity
 - No interference
 - Measurement and selection
 - Understand linear regression
 - Interpret coefficients conditionally
 - Represent countries and time
 - Check functional form and interactions
 - Interpret uncertainty carefully
 - Use diagnostics for their proper purpose
 - Compare specifications as sensitivity evidence
 - Bound the causal conclusion
 - Check your understanding
 - Further resources
 - Continue to Application
-

Learning objectives

After completing this page, you should be able to:

- distinguish a causal question, estimand, identification strategy, estimator, and estimate;
 - define an average causal effect with potential outcomes;
 - use a directed acyclic graph to express causal assumptions;
 - distinguish confounders, mediators, colliders, proxies, and precision variables;
 - explain consistency, exchangeability, positivity, no interference, and measurement requirements;
 - interpret regression coefficients as conditional associations;
 - state the additional conditions required for causal interpretation;
 - evaluate functional form, residuals, influence, temporal dependence, and specification sensitivity; and
 - distinguish statistical uncertainty from identification uncertainty.
-

Place in the session

This is the **Concepts** part of the Explanatory Modeling session:

Motivation → Concepts → Application
↑
this page

[Why explanatory modeling requires causal reasoning](#) motivates the distinction between association and causation. This page provides the framework used in [the maize-yield causal analysis](#).

Use one organizing principle:

A statistical model receives causal meaning from the question, design, and identification assumptions—not from the model family or coefficient name.

Separate question, estimand, identification, and estimation

These stages answer different questions:

| Stage | Question | Maize example |
|-----------------|---|--|
| Causal question | What change and outcome should be understood? | What would happen to yield under a specified precipitation contrast? |
| Estimand | Which causal quantity summarizes the question? | Average difference in potential yield under two exposures |
| Identification | Under which assumptions can observed data express it? | Conditional exchangeability and exposure overlap |
| Estimator | Which procedure calculates the identified quantity? | Regression adjustment under a stated model |
| Estimate | What value resulted in this sample? | Yield difference per 100 mm with uncertainty |

An estimator can be computed when identification fails. It then estimates an observed-data association, not the intended causal estimand.

Use potential outcomes to define causal effects

Let $Y_i(p)$ represent the maize yield unit i would have under precipitation exposure p . For two specified levels, the unit-level effect is:

$$Y_i(p_1) - Y_i(p_0)$$

The average causal effect over a target population is:

$$E[Y(p_1) - Y(p_0)]$$

The same unit cannot reveal both outcomes at the same time. Causal inference uses other observed units or times as comparisons under assumptions.

For continuous precipitation, one might target the contrast between p and $p + 100$ mm or a dose-response function $E[Y(p)]$. A constant linear slope assumes the same marginal difference across the supported exposure range. That may be implausible if drought and excessive rain both reduce yield.

Make causal structure explicit

A **directed acyclic graph** (DAG) represents assumed causal relationships:

- nodes represent variables or concepts;
- arrows represent direct causal influence at the chosen abstraction;
- paths represent possible routes of association; and
- acyclic means the represented time order contains no feedback loop.

A provisional maize DAG could contain:

```

weather system      precipitation      yield

country and time context  precipitation
                        yield
irrigation, inputs, soils and varieties      yield

```

The diagram is not discovered by selecting significant correlations. It is a claim based on domain knowledge, temporality, measurement, and the research question. Record plausible alternatives.

A broad node such as “country and time context” hides many mechanisms. Drawing it does not mean a country indicator and year term measure every relevant cause.

Distinguish variable roles

A **confounder** is a common cause of exposure and outcome. Appropriate adjustment can block a backdoor path when the variable is measured adequately.

A **mediator** occurs after exposure and carries part of its effect. Soil moisture or crop disease may mediate rainfall effects. Adjusting for a mediator can remove part of a total effect.

A **collider** is caused by two variables. Conditioning on it can create an association between its causes and open a biased path.

A **proxy** imperfectly represents another concept. Country and year terms proxy some contextual differences but do not guarantee control of time-varying confounding.

A **precision variable** predicts the outcome without being required to block confounding. It may improve precision but is not what makes an estimate causal.

Roles depend on the causal question. Irrigation could be a baseline confounder, a response to expected rainfall, a mediator, or part of the intervention. State timing and meaning before adjustment.

Evaluate identification assumptions

Consistency

If a unit receives exposure p , its observed outcome must correspond to $Y(p)$. Equal seasonal totals with different timing, intensity, or spatial distribution may not be equivalent treatment versions.

Exchangeability

Conditional exchangeability requires:

$$Y(p) \perp P \mid C$$

after adjustment for a sufficient set of pre-exposure common causes C . This cannot be tested directly. Temperature, irrigation, input use, economic shocks, crop location, and management remain concerns.

Positivity

Relevant exposure contrasts must occur within adjustment groups. If a country never experiences the required precipitation range, the model extrapolates. For a continuous exposure, inspect distributions and ranges by country and period.

No interference

One unit's exposure should not alter another unit's potential outcome under the treatment definition. Shared water, trade, migration, and regional shocks can challenge this assumption.

Measurement and selection

Recorded variables and rows must represent the intended concepts and target population. Country-area precipitation is not crop-specific exposure, and national yield conceals subnational variation.

Understand linear regression

For country i and year t , consider:

$$Y_{it} = \beta_0 + \beta_1 P_{it} + \alpha_i + f(t) + \varepsilon_{it}$$

where Y is national yield, P is seasonal precipitation, α_i represents country indicators, $f(t)$ represents time, and ε is the observed deviation from fitted yield.

Ordinary least squares minimizes squared residuals. Fitted values are modeled conditional means. Residuals are not measurements of causal effects or every omitted cause.

The intercept is expected yield when numeric variables equal zero and factors are at reference levels. Centering year and scaling precipitation make the intercept and slope easier to read without changing fitted values in an otherwise equivalent linear model.

Interpret coefficients conditionally

When precipitation is expressed per 100 mm, β_1 is the modeled difference in expected yield associated with a 100 mm difference, conditional on included terms.

It is causal only if:

- the coefficient corresponds to the intended estimand;
- the causal effect is identified by the adjustment strategy;
- functional form and measurement are adequate;
- the estimator and uncertainty procedure are appropriate; and
- selection and interference do not invalidate comparison.

“Holding country and year constant” describes a model comparison. It does not mean all country characteristics and historical processes have been fixed.

Represent countries and time

Country indicators allow different intercepts and use within-country exposure variation. They account for stable differences represented by country membership, not unmeasured country characteristics that change over time.

A common linear trend assumes equal additive annual change across countries. Country-specific trends relax that assumption but consume more information and can absorb exposure variation. Flexible time terms may improve fit while increasing uncertainty or extrapolation.

Repeated country observations can have correlated residuals. Default ordinary regression intervals may not represent this dependence. With only nine countries, cluster-based inference also needs caution. Report the limitation rather than presenting default intervals as definitive.

Check functional form and interactions

A linear precipitation term assumes a constant slope. Agronomic reasoning suggests possible nonlinearity: rain may help under dry conditions but offer little benefit or cause damage under wet conditions.

Motivated alternatives include:

- a quadratic precipitation term;
- a spline as an advanced extension;
- precipitation-by-country interactions; and
- alternative time representations.

An interaction means the modeled association differs across another variable and must be interpreted jointly. Flexibility can address functional form; it does not solve confounding or exposure ambiguity.

Interpret uncertainty carefully

A confidence interval describes sampling uncertainty under a model and procedure. It does not quantify uncertainty from:

- unmeasured confounding or a wrong causal graph;
- exposure-version ambiguity;
- selection or interference;
- measurement error; or
- selective model choice.

A p-value evaluates compatibility with a specified null model. It is not the probability that a causal hypothesis is true. Report estimate, unit, interval, sample size, model, and identification limitations together.

Use diagnostics for their proper purpose

| Diagnostic | Possible warning | Does not establish |
|------------------------------|----------------------------------|---------------------------|
| Residuals versus fitted | Nonlinearity or changing spread | No unmeasured confounding |
| Residual quantiles | Heavy tails or unusual residuals | Causal direction |
| Scale-location plot | Non-constant residual variance | Correct measurement |
| Leverage and Cook's distance | Influential observations | Whether influence is bias |
| Residuals over time | Trend or temporal dependence | Exchangeability |
| Exposure by group | Poor overlap or extrapolation | Consistency |

Investigate warnings rather than deleting observations automatically. An influential year may be a real drought, policy shift, measurement problem, or model misspecification.

Compare specifications as sensitivity evidence

Use a planned sequence:

1. unadjusted precipitation association;
2. country indicators;
3. a time representation;
4. country and time together; and
5. motivated nonlinear or interaction terms.

Compare direction, magnitude, uncertainty, residual structure, and exposure support. Large changes reveal specification dependence. Small changes show stability across those specifications, not protection against every omitted variable.

Do not search many models and retain only a significant result. Record the sequence, rationale, and all planned results.

Bound the causal conclusion

A useful conclusion has five layers:

1. **Observed data:** population, grain, exposure, outcome, and coverage.
2. **Statistical estimate:** association, unit, interval, and model.
3. **Identification assessment:** assumptions supported, doubtful, or unknown.
4. **Causal claim:** causal language only to the justified extent.
5. **Next evidence:** measurements or designs that would strengthen inference.

For this project, a defensible core conclusion is likely:

The models estimate precipitation-yield associations conditional on selected country and time terms. The national observational data do not adequately establish a well-defined precipitation intervention or control important time-varying common causes, so coefficients are not identified causal effects.

Check your understanding

1. How do an estimand, estimator, and estimate differ?
 2. What does a potential outcome represent?
 3. Why is a DAG an assumption rather than a fitted result?
 4. How do confounders, mediators, and colliders differ?
 5. What do consistency, exchangeability, positivity, and no interference require?
 6. What does a precipitation coefficient represent in a linear model?
 7. Why do country indicators not remove every country-related confounder?
 8. How can temporal dependence affect uncertainty?
 9. What does a quadratic term address, and what remains unresolved?
 10. Why can a narrow confidence interval accompany weak identification?
 11. Which questions can residual diagnostics answer?
 12. What makes specification comparison useful rather than selective?
-

Further resources

- Miguel A. Hernán and James M. Robins, [Causal Inference: What If](#)
 - Scott Cunningham, [Causal Inference: The Mixtape](#)
 - Brady Neal, [Introduction to Causal Inference](#)
 - [DAGitty](#)
 - [R for Data Science \(2e\): Model basics](#)
 - [R 1m documentation](#)
-

Continue to Application

Continue with [Conduct a causal analysis of maize yield](#). You will define an estimand, draw a causal diagram, assess identification, compare regression specifications, inspect diagnostics, and write a bounded conclusion.

9.3) Conduct a causal analysis of maize yield

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverables
 - Before you begin
 - 1. Define the causal question and estimand
 - 2. Draw the causal diagram
 - 3. Assess identification before estimation
 - 4. Inspect exposure support
 - 5. Fit the planned regression sequence
 - 6. Interpret estimates and uncertainty
 - 7. Diagnose models and test specification sensitivity
 - 8. Write a bounded causal conclusion
 - Independent extension
 - Troubleshooting
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- define a target population, exposure contrast, outcome, time horizon, and causal estimand;
 - encode causal assumptions in a directed acyclic graph and assess identification requirements;
 - fit and compare transparent linear-regression specifications in R;
 - inspect residual patterns, influential observations, temporal structure, and exposure support;
 - separate model uncertainty from causal-identification uncertainty; and
 - conclude whether the results support a causal effect or only adjusted associations.
-

Place in the session

This is the **Application** part of the Explanatory Modeling session:

```
Motivation → Concepts → Application
                        ↑
                    this page
```

Before beginning, review [Why explanatory modeling requires causal reasoning](#) and [Understand explanatory-modeling concepts](#).

Descriptive Data Analysis established coverage, summaries, and association evidence. This exercise uses that evidence to design and challenge a causal analysis — it does not overwrite data or treat prediction and causal explanation as the same task.

Scenario and deliverables

The project team asks:

What is the causal effect of growing-season precipitation on national maize yield in the selected Southern African countries?

Turn this broad question into an auditable analysis and decide whether the available data identify the intended effect.

Create:

```
docs/causal-model.md
scripts/explain-maize-yield.R
results/tables/explanatory-exposure-support.csv
results/tables/explanatory-model-estimates.csv
results/tables/explanatory-model-diagnostics.csv
results/tables/explanatory-residual-dependence.csv
results/explanatory-modeling-conclusion.md
reports/explanatory-modeling.qmd
```

The causal-model document records the question, estimand, diagram, variable roles, and identification judgment. The conclusion must distinguish what the models estimate from what can be interpreted causally.

Before you begin

Work from the standalone `maize-yield-project` repository:

```
pwd
git status --short --branch
```

Restore the environment and recreate upstream evidence:

```
Rscript scripts/setup.R
Rscript scripts/validate-data.R
Rscript scripts/prepare-maize-data.R
Rscript scripts/integrate-data.R
Rscript scripts/visualize-maize-data.R
Rscript scripts/describe-maize-data.R
```

Review:

| Evidence | File |
|-----------------------------|---|
| Integrated-data definition | docs/data/maize-yield-with-precipitation.md |
| Variable dictionary | metadata/maize-yield-with-precipitation-data-dictionary.csv |
| Visual relationship | figures/yield-versus-precipitation.png |
| Country/period descriptions | results/tables/maize-yield-period-summary.csv |
| Association by scope | results/tables/yield-precipitation-association.csv |

| Evidence | File |
|--------------------|--|
| Stability evidence | results/tables/stationarity-diagnostic.csv |
| Modeling handoff | results/descriptive-modeling-handoff.md |

The input contains 297 country-years, not randomized rainfall treatments or field-level exposures.

1. Define the causal question and estimand

Create `docs/causal-model.md` and complete:

| Element | Provisional definition |
|-----------------------|---|
| Target population | Country-years for nine selected countries, 1990–2022 |
| Unit | One country-year |
| Exposure | October–April country-area CHIRPS precipitation total |
| Contrast | 100 mm higher versus the observed reference level |
| Outcome | National maize yield (t/ha) for the ending year |
| Time zero / follow-up | Start of growing season / ending-year yield |
| Estimand | Average difference in potential yield under the two exposures |

Then challenge every row: explain why “100 mm higher” does not specify rainfall timing, intensity, location, or mechanism, and decide whether the contrast is a provisional scientific target rather than a well-defined intervention.

Write the potential-outcomes expression:

$$E[Y(P + 100) - Y(P)]$$

State whether a constant shift is meaningful throughout the observed range. Do not change the estimand after inspecting which model is significant.

2. Draw the causal diagram

Begin with concepts, not only available columns. Include at least: precipitation amount, timing, and intensity; seasonal weather; national maize yield; irrigation, soils, and maize-growing area; varieties, fertilizer, labor, and management; pests and disease; policy, markets, and reporting practices; country context and calendar time; and measurement and selection processes.

For every arrow, write one sentence explaining the assumed mechanism and temporal order, and mark whether each node is measured, proxied, or unmeasured. Use a simplified diagram in the report while retaining the full inventory in `docs/causal-model.md`:


```

seasonal weather    precipitation    maize yield

country/time context    precipitation

                        yield

irrigation, soils, inputs and management    yield

```

This is not a finished adjustment strategy — discuss omitted factors rather than using the diagram merely to justify variables already in the table.

3. Assess identification before estimation

Add an identification table:

| Requirement | Evidence | Judgment | Consequence |
|-----------------|---|-----------------|--|
| Consistency | Seasonal total;
timing/spatial versions
hidden | Doubtful | 100 mm contrast is
ambiguous |
| Exchangeability | Country/year observed;
weather and
management causes
omitted | Not established | Coefficient may remain
confounded |
| Positivity | 33 obs. per country;
ranges differ | Must inspect | Some contrasts require
extrapolation |
| No interference | Countries linked by
markets, water, shocks | Uncertain | Treatment
representation may be
incomplete |
| Measurement | National yield,
country-area rainfall | Limited | Exposure-outcome
alignment imperfect |
| Selection | Nine countries, complete
teaching data | Limited target | Do not generalize
automatically |

Do not write “assumption met” merely because it cannot be tested — state what evidence would be needed, and decide before modeling whether the dataset plausibly identifies the estimand. The core expectation is that it does not fully do so; the statistical models remain useful for quantifying how associations change under selected adjustments and for demonstrating the consequences of the design limitations.

4. Inspect exposure support

Create `scripts/explain-maize-yield.R`:

```

# Conduct the project explanatory-modeling analysis.

source("scripts/functions.R")
assert_project_root()
ensure_project_directories()
check_required_packages(c("dplyr", "here", "readr", "tibble"))

library(dplyr)
library(here)
library(readr)
library(tibble)

input_file <- here(
  "data", "derived", "maize-yield-with-precipitation.csv"
)
maize <- read_csv(input_file, show_col_types = FALSE)

required <- c(
  "project_country_id", "project_country_name", "year",
  "yield_tonnes_per_hectare", "growing_season_precipitation_mm"
)
if (length(setdiff(required, names(maize))) > 0 ||
    nrow(maize) != 297L ||
    anyDuplicated(maize[c("project_country_id", "year")])) {
  stop("Expected 297 unique country-year observations and required columns.")
}

maize <- maize |>
  mutate(
    country = factor(project_country_name),
    year_centered = year - 1990,
    precipitation_100mm = growing_season_precipitation_mm / 100
  )

```

Summarize support:

```

exposure_support <- maize |>
  group_by(project_country_id, project_country_name) |>
  summarise(
    n = sum(!is.na(precipitation_100mm)),
    minimum_100mm = min(precipitation_100mm, na.rm = TRUE),
    q25_100mm = quantile(precipitation_100mm, 0.25, na.rm = TRUE),
    median_100mm = median(precipitation_100mm, na.rm = TRUE),
    q75_100mm = quantile(precipitation_100mm, 0.75, na.rm = TRUE),
    maximum_100mm = max(precipitation_100mm, na.rm = TRUE),
    .groups = "drop"
  )

write_csv(
  exposure_support,
  here("results", "tables", "explanatory-exposure-support.csv"),
  na = ""
)

```

Compare country ranges and distributions — a model with country indicators primarily uses within-country variation. Identify where a 100 mm contrast implies extrapolation beyond observed support. Range overlap is a necessary descriptive check, not proof of conditional positivity.

5. Fit the planned regression sequence

Fit specifications selected before viewing results:

```
models <- list(  
  unadjusted = lm(  
    yield_tonnes_per_hectare ~ precipitation_100mm,  
    data = maize  
  ),  
  country = lm(  
    yield_tonnes_per_hectare ~ precipitation_100mm + country,  
    data = maize  
  ),  
  time = lm(  
    yield_tonnes_per_hectare ~ precipitation_100mm + year_centered,  
    data = maize  
  ),  
  country_time = lm(  
    yield_tonnes_per_hectare ~  
      precipitation_100mm + country + year_centered,  
    data = maize  
  ),  
  nonlinear_sensitivity = lm(  
    yield_tonnes_per_hectare ~  
      precipitation_100mm + I(precipitation_100mm^2) +  
      country + year_centered,  
    data = maize  
  )  
)
```

Document the role of every specification:

| Model | Comparison | Purpose | Remaining concern |
|----------------|----------------------|-----------------------------|----------------------------|
| Unadjusted | All country-years | Pooled linear association | Country/time differences |
| Country | Within-country slope | Stable country levels | Time-varying confounding |
| Time | Common trend | Common linear change | Country differences |
| Country + time | Within-country trend | Main adjusted association | Omitted causes, dependence |
| Nonlinear | Curved association | Functional-form sensitivity | Same identification limits |

Country and time terms are not a declared sufficient adjustment set — they are transparent, available proxies used to show how interpretation changes.

6. Interpret estimates and uncertainty

Extract the linear precipitation term from the first four models:

```
extract_precipitation <- function(model, model_name) {
  coefficient_table <- summary(model)$coefficients
  interval <- confint(model, "precipitation_100mm", level = 0.95)

  tibble(
    model = model_name,
    n = nobs(model),
    estimate_t_per_ha_per_100mm =
      coefficient_table["precipitation_100mm", "Estimate"],
    standard_error =
      coefficient_table["precipitation_100mm", "Std. Error"],
    confidence_low = interval[1],
    confidence_high = interval[2],
    p_value = coefficient_table["precipitation_100mm", "Pr(>|t|)"],
    r_squared = summary(model)$r.squared,
    adjusted_r_squared = summary(model)$adj.r.squared
  )
}

model_estimates <- bind_rows(
  lapply(
    names(models)[1:4],
    function(name) extract_precipitation(models[[name]], name)
  )
)

write_csv(
  model_estimates,
  here("results", "tables", "explanatory-model-estimates.csv"),
  na = ""
)
```

Interpret each estimate in tonnes per hectare per 100 mm and compare direction and magnitude across models before discussing p-values.

The intervals use default `lm` assumptions, including an error structure that does not represent within-country temporal dependence. They quantify model-based sampling uncertainty only — not uncertainty from confounding, measurement, intervention ambiguity, or specification choice. Do not interpret the linear term from the quadratic model alone: the modeled difference varies with baseline precipitation, so inspect predicted contrasts across supported values instead.

7. Diagnose models and test specification sensitivity

For every model, calculate transparent checks:

```

model_diagnostics <- bind_rows(lapply(names(models), function(name) {
  model <- models[[name]]
  cooks <- cooks.distance(model)
  residual <- residuals(model)

  tibble(
    model = name,
    n = noobs(model),
    residual_sd = sigma(model),
    maximum_absolute_residual = max(abs(residual)),
    maximum_cooks_distance = max(cooks),
    observations_above_4_over_n = sum(cooks > 4 / noobs(model))
  )
}))

write_csv(
  model_diagnostics,
  here("results", "tables", "explanatory-model-diagnostics.csv"),
  na = ""
)

```

Inspect, do not merely calculate: residuals versus fitted values, the residual distribution and scale-location pattern, high-leverage/Cook's-distance observations, residuals over time within countries, and lag-one residual correlation by country.

Create `explanatory-residual-dependence.csv` with lag-one pair counts and correlations for the main `country_time` model, verifying consecutive years before forming pairs.

Ask whether adjustment substantially changes the precipitation coefficient, whether a linear form misses a visible curve, whether conclusions are driven by a small number of country-years, and which findings are stable across specifications. Diagnostics can reveal statistical misspecification; they cannot establish exchangeability or correct measurement.

8. Write a bounded causal conclusion

Create `results/explanatory-modeling-conclusion.md`:

```

# Explanatory-modeling conclusion

## Causal question and estimand
## Data and measurement
## Causal diagram and adjustment reasoning
## Identification assessment
## Statistical models and estimates
## Diagnostics and specification sensitivity
## Supported interpretation
## Unsupported interpretations
## Evidence needed for stronger causal inference

```

Your conclusion must include:

- the estimated association per 100 mm from every planned model, and how it changes with country and time terms;
- the nonlinear sensitivity result and relevant overlap, influence, residual, and temporal warnings;
- which identification assumptions are doubtful or unassessable, and why default confidence intervals exclude identification uncertainty;
- an explicit statement that correlation and adjusted regression are not automatically causal; and
- concrete data or design improvements.

A defensible conclusion may be:

The models quantify associations between country-area growing-season precipitation and national maize yield conditional on selected country and time terms. Because the exposure is not a well-defined intervention and important time-varying common causes and measurement limitations remain, the available data do not identify the proposed causal effect.

Render `reports/explanatory-modeling.qmd` so the question, diagram, identification table, estimates, and conclusion stay linked to the reproducible artifacts.

Independent extension

Choose one extension and document its causal purpose and limitations.

- **Alternative exposure representation:** compare the seasonal total with dry-day counts or sub-season periods; explain the effect on consistency.
- **Country-specific associations:** add precipitation-by-country interactions and explain the small sample size and increased uncertainty.
- **Negative-control reasoning:** propose an exposure or outcome that should share bias mechanisms without a causal link, and explain what a detected association would reveal.
- **Stronger research design:** sketch a field experiment, irrigation intervention, or subnational panel, stating its assignment mechanism, estimand, and practical limits.

Troubleshooting

- **Coefficient sign changes:** inspect model comparisons, exposure support, and influential observations; report the sensitivity rather than the preferred sign.
 - **Country coefficients dominate the table:** they are nuisance parameters — retain them but present a focused precipitation-estimand table.
 - **Quadratic term is significant:** do not interpret it in isolation; plot fitted values and contrasts at supported precipitation levels.
 - **Residuals are temporally correlated:** default standard errors may be inappropriate; report the evidence rather than claiming independence.
 - **Confounder unavailable:** do not silently replace it with a country indicator — record it as unmeasured and bound the causal claim.
 - **High R^2 :** fit is not identification; the precipitation coefficient can remain causally biased regardless.
 - **Script modifies integrated data:** stop — read the derived artifact and write results only, never overwrite `data/input/` or `data/derived/`.
-

Completion checklist

- ☐ Question, contrast, outcome, timing, and estimand are stated, including the meaning of an additional 100 mm.
 - ☐ The DAG distinguishes confounders, mediators, colliders, and proxies with justified arrows.
 - ☐ Identification assumptions are assessed and exposure support is inspected by country and period.
 - ☐ The regression sequence was specified before inspecting significance; estimates retain units and intervals.
 - ☐ Nonlinear sensitivity uses contrasts, not one coefficient; residuals and influence are inspected.
 - ☐ Statistical and identification uncertainty are kept separate; results are not selected by p-value.
 - ☐ The conclusion states what is and is not causally supported, and code recreates all outputs.
-

Reflect on the application

1. Which part of the provisional precipitation intervention remained most ambiguous?
 2. Which causal arrows were supported by domain knowledge but not measured?
 3. How did country and time terms change the estimate and its interpretation?
 4. Which causal assumptions could not be assessed with residual diagnostics?
 5. Which additional measurement or design would most strengthen the causal analysis?
-

Further resources

- Miguel A. Hernán and James M. Robins, [Causal Inference: What If](#)
- Scott Cunningham, [Causal Inference: The Mixtape](#)
- [DAGitty](#)
- [R lm documentation](#)
- [R diagnostic plots for linear models](#)
- [The Turing Way: Reproducible Research](#)

10.1) Why predictive modeling requires unseen-data evaluation

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Prediction concerns observations not used for fitting
 - Explanation and prediction are different objectives
 - Intended use determines meaningful evaluation
 - Time makes maize-yield prediction difficult
 - Simple baselines are scientifically useful
 - What can go wrong
 - How this connects to the maize-yield project
 - Initial activity
 - Check your understanding
 - Further resources
 - Continue to Concepts
-

Learning objectives

After completing this page, you should be able to:

- distinguish a predictive objective from a descriptive or causal objective;
 - explain why training-data performance does not establish predictive usefulness;
 - connect prediction time and intended use to the information available to a model;
 - justify evaluating later observations instead of a random sample of country-years; and
 - recognize leakage, distribution shift, and an underspecified prediction task.
-

Place in the session

This is the **Motivation** part of the Predictive Modeling session:

```
Motivation → Concepts → Application
      ↑
  this page
```

The Explanatory Modeling session asked what a modeled relationship could mean under explicit causal assumptions. Predictive modeling asks a different question: how accurately can a procedure predict outcomes that were not used to construct it?

[Understand predictive-modeling concepts](#) defines a prediction contract, data splits, leakage, baselines, and evaluation metrics. [Evaluate maize-yield predictions](#) applies those concepts to the example project.

Prediction concerns observations not used for fitting

A fitted model can describe its training observations very closely and still fail on new observations: it may have learned stable structure, or instead noise, unusual cases, or information unavailable when a prediction is needed.

Predictive performance therefore concerns **generalization** — performance on relevant observations outside the data used for fitting and model selection. Learners must define the intended use, protect an evaluation set, fit every learned operation using training data only, generate predictions without seeing the answers, and then quantify the errors.

The useful question is not how closely the fitted line follows its estimation data, but how well the complete procedure predicts observations representing its intended use.

Explanation and prediction are different objectives

Explanatory and predictive models can use the same mathematical family, such as linear regression, while serving different purposes. An explanatory analysis needs a defensible intervention, causal structure, and an estimator that matches the estimand — for example, whether a precipitation contrast has a causal effect on maize yield. A predictive analysis needs a precise prediction task and evaluation on representative unseen data — for example, how accurately later national yields can be predicted from information available at a stated time.

Neither objective is automatically stronger: a variable can improve predictions without causing the outcome, a causal variable can add little predictive accuracy, and predictive accuracy cannot repair an unidentified causal analysis. The objective must be chosen before selecting models and performance measures.

Intended use determines meaningful evaluation

“Predict maize yield” is incomplete. A prediction task must say what outcome and unit are predicted, which countries and years form the target population, when the prediction is made and how far ahead it reaches, which information exists at that time, how data are separated for development and evaluation, and which errors matter for the intended decision.

For example, realized annual precipitation cannot be used for a forecast made before the rainy season because those observations do not yet exist, though it may be valid for a post-season estimate — a different task even with the same outcome and model.

Evaluation must reproduce the intended use as closely as the teaching data allow: if use concerns later years, randomly mixing early and late country-years between training and test sets can make performance appear more reliable than it is.

Time makes maize-yield prediction difficult

The maize panel combines repeated annual observations for a small set of countries, where neighboring years are related and technology, management, markets, climate, and reporting systems can alter both the level and variability of yield over time.

Consequently, the future may not follow the same distribution as the past — a possibility called **distribution shift**. The Descriptive Data Analysis session examined trends, period differences, changing variance, and temporal dependence because each affects later-period prediction.

A temporal split does not eliminate shift; it exposes the model to one realistic form of it by learning from earlier observations and evaluating on later ones. Good performance for 2018–2022 still does not guarantee good performance after 2022, in new countries, or under unprecedented conditions.

Simple baselines are scientifically useful

A complex model is not useful merely because it can produce predictions — it should improve on simple, credible rules under the same evaluation design. For maize yield: does it beat one historical average? Does a common time trend help? Does accounting for persistent country differences help further? Does an added predictor improve held-out errors enough to justify its cost?

Baselines reveal how much performance comes from simple regularities. If a complex model cannot beat them, complexity has not earned its cost; if it does, the size and stability of the improvement still matter.

What can go wrong

Common failures include:

- reporting training fit as predictive performance;
- choosing or repeatedly tuning models against the test period after inspecting its outcomes;
- preprocessing all years before splitting the data, or using variables measured after the stated prediction time;
- allowing duplicate or related observations to cross split boundaries;
- selecting a model without comparing a meaningful baseline, or reporting only one aggregate metric that hides country-specific failures;
- interpreting predictive associations causally; and
- deploying beyond the countries, years, and conditions represented by the evaluation.

These are workflow and design problems, not ones a more sophisticated algorithm automatically solves.

How this connects to the maize-yield project

The example project defines a transparent teaching benchmark:

- predict annual national log maize yield;
- use country identity and calendar year as the available information;
- train on 1990–2017 observations;
- hold out 2018–2022 observations;
- compare a historical mean, a common linear trend, and a country-plus-time model; and
- evaluate row-level errors with mean absolute error (MAE) and root mean squared error (RMSE).

This is not an operational crop forecast: the split contains only five test years per country, the predictors are deliberately limited, and calendar year is a trend representation rather than a mechanism, teaching an honest predictive workflow and the boundaries of its evidence.

Precipitation can support a separate extension only once its availability at prediction time is specified — realized seasonal precipitation supports a nowcasting or retrospective task, not necessarily an early-season forecast.

Initial activity

Write a short prediction contract for one of these uses:

1. forecast annual maize yield before the growing season;
2. update the prediction during the growing season; or
3. estimate yield after the season but before official yield data are published.

State the target, prediction time, horizon, available information, target countries, and one important error. Then answer:

- Would realized growing-season precipitation be available?
- Would a random train/test split represent the use?
- Which simple rule should the model beat?

Check your understanding

1. Why can a regression with excellent training fit predict later years poorly?
2. Why does prediction time determine whether precipitation is a valid predictor?
3. What does a temporal holdout test that a random holdout may not?
4. Why does predictive performance not establish a causal effect?
5. What makes the current maize exercise a teaching benchmark rather than an operational forecast?

Further resources

- [An Introduction to Statistical Learning](#) introduces prediction, resampling, and model assessment.
- [Tidy Modeling with R](#) presents reproducible model development and evaluation in R.
- [The Turing Way: Reproducible Research](#) explains auditable, reusable computational practice.
- [scikit-learn: Common pitfalls and recommended practices](#) gives examples of preprocessing leakage that apply outside Python.
- Shmueli, G. (2010). “To Explain or to Predict?” *Statistical Science*, 25(3), 289–310, clarifies why explanatory and predictive modeling need different evaluation criteria.

Continue to Concepts

Continue with [Understand predictive-modeling concepts](#) to define the task, protect unseen data, and interpret predictive errors.

10.2) Understand predictive-modeling concepts

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Begin with a prediction contract
 - Separate training, validation, and testing
 - Choose a split that represents use
 - Prevent leakage
 - Evaluate the complete workflow
 - Use baselines to define useful performance
 - Understand prediction errors, MAE, and RMSE
 - Evaluate errors beyond one average
 - Recognize overfitting and distribution shift
 - Handle transformed outcomes carefully
 - Define reproducible prediction artifacts
 - Bound interpretation and use
 - Check your understanding
 - Further resources
 - Continue to Application
-

Learning objectives

After completing this page, you should be able to:

- specify a prediction contract with a target, information set, horizon, use, and evaluation design;
 - distinguish training, validation, and test data;
 - select a temporal or grouped split that represents the deployment problem;
 - identify target, preprocessing, feature, and test-set leakage;
 - compare candidate models with transparent baselines;
 - calculate and interpret prediction errors, MAE, and RMSE;
 - inspect performance across observations, countries, and periods;
 - explain overfitting, temporal dependence, and distribution shift;
 - handle log-scale predictions deliberately; and
 - define the artifacts needed to reproduce a predictive evaluation.
-

Place in the session

This is the **Concepts** part of the Predictive Modeling session:

```
Motivation → Concepts → Application
              ↑
            this page
```

Why predictive modeling requires unseen-data evaluation introduced the need for an intended use and representative holdout. This page supplies the concepts used in the maize-yield predictive application.

Use one organizing principle:

Predictive performance is a property of a complete procedure, evaluated on relevant unseen observations—not a property of a fitted equation alone.

Begin with a prediction contract

A **prediction contract** is a documented agreement about what will be predicted, when, from which information, for whom, and how success will be judged. It prevents the task from changing after results are visible.

| Component | Question | Maize teaching task |
|-------------------|-------------------------------------|--|
| Target | What value and scale are predicted? | Annual national log maize yield |
| Unit | What receives one prediction? | One country-year |
| Target population | Where should predictions apply? | Project countries in later observed years |
| Prediction time | When is the prediction issued? | Represented as prediction of held-out later years |
| Horizon | How far ahead is the outcome? | Not operationally specified; a teaching limitation |
| Information set | What is available then? | Country identity and calendar year |
| Evaluation | Which observations remain unseen? | 2018–2022 country-years |
| Loss and metrics | Which errors matter? | Absolute and squared log-scale errors |
| Intended use | Which decision will use it? | Method demonstration, not operational allocation |

The limitation in the horizon is important. Calling this workflow a forecast would imply an operational issue date and information cutoff that the exercise does not fully define. “Held-out prediction benchmark” is more accurate.

The **information set** contains only predictors legitimately known at the prediction time. A value can exist in a historical table but be unavailable when a real prediction would be issued. Realized growing-season precipitation, for example, cannot support a pre-season forecast. It can support a separately defined post-season estimate or nowcast.

Separate training, validation, and testing

The three data roles answer different questions:

| Partition | Primary role | Permitted use |
|------------|--------------------------------------|---|
| Training | Learn parameters and transformations | Fit means, coefficients, imputers, encodings, or other learned operations |
| Validation | Compare candidates and settings | Select models or hyperparameters without using the final test set |
| Test | Estimate final generalization | Evaluate the fixed workflow once |

With a small dataset, validation may use resampling within the training period rather than a permanent third partition; for temporal data, rolling-origin or forward-chaining resampling usually represents later prediction better than ordinary random cross-validation.

The current maize benchmark compares a small set of predefined candidates without extensive tuning, but its 2018–2022 test period must still stay excluded from fitting and candidate invention — repeatedly changing the workflow after viewing test errors turns the test data into validation data and removes the independent final assessment.

Choose a split that represents use

A split is part of the scientific design. Common strategies include:

- **random split** when observations are exchangeable and use resembles a random observation from the same population;
- **group split** when all observations from a person, site, farm, or country must remain together;
- **temporal split** when training on earlier observations and predicting later ones represents use;
- **spatial split** when geographic transfer is the objective; and
- **rolling-origin evaluation** when several successive past-to-future tests are needed.

The maize task predicts later years for countries already present in training, so a temporal split is appropriate; it does **not** test performance for a new country — a leave-country-out design would answer that different question.

Do not select the split because it produces favorable metrics; record its boundary, rationale, and exclusions before inspecting test outcomes.

Prevent leakage

Data leakage occurs when model development uses information unavailable under the prediction contract. It creates unrealistically good results.

Common routes include:

- **target leakage**: a predictor directly or indirectly contains the outcome;
- **future leakage**: later information is used to predict an earlier outcome;
- **preprocessing leakage**: means, scales, imputation values, categories, or selected variables are learned from all data before splitting;
- **group leakage**: related or duplicated observations appear on both sides;

- **test-set leakage:** test outcomes influence features, candidates, or tuning; and
- **publication leakage:** decisions are revised after test performance is known but reported as if they were prespecified.

Prevent leakage by splitting before learned preprocessing, fitting the complete recipe on training data only, preserving group and time boundaries, and logging every interaction with the test set.

Deterministic operations defined independently of observed values — such as a documented unit conversion — can apply to both partitions; operations that estimate something from a distribution must learn it from training only.

Evaluate the complete workflow

A predictive workflow includes more than `lm()` or another model call:

```
prediction contract
  ↓
split definition
  ↓
training-fitted preparation
  ↓
candidate model fitting
  ↓
prediction on unseen rows
  ↓
error calculation and subgroup checks
  ↓
artifacts, limitations, and report
```

Performance belongs to this whole sequence. Changing the split, imputation, feature definition, or prediction scale changes the evaluated procedure even when the model formula remains unchanged.

Use baselines to define useful performance

A **baseline** is a simple prediction rule that an elaborate candidate should improve upon. It provides context that a metric alone cannot.

The maize workflow uses:

- a **historical mean**, which predicts one training-period mean log yield;
- a **common linear trend**, which extrapolates a trend across countries; and
- a **country-plus-time model**, which represents persistent country differences and a common temporal trend.

Other defensible baselines include the last observed value or a per-country training mean — a country mean asks whether identity alone is sufficient, and a last-value rule asks whether the most recent observation is strongly predictive.

Candidates must use the same split, target rows, outcome scale, and metric. A more complex model earns its cost only through material and reasonably stable unseen-data improvement, not lower training error.

Understand prediction errors, MAE, and RMSE

For observation i , define prediction error as:

$$e_i = y_i - \hat{y}_i,$$

where y_i is observed and $\hat{y}_i$ is predicted. Positive errors mean underprediction and negative errors mean overprediction under this convention.

Mean absolute error is:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Root mean squared error is:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Both are non-negative; lower is better on the same test set. MAE weights absolute errors linearly. RMSE squares errors and emphasizes large misses. Neither is universally best. Choose loss to reflect intended use and report complementary evidence.

Signed mean error can reveal systematic over- or underprediction, but positive and negative values can cancel. In this project all core metrics are calculated on `log_yield`, so they are not errors in tonnes per hectare.

Evaluate errors beyond one average

One aggregate metric can hide important failure modes. Inspect:

- row-level observations, predictions, and errors;
- errors by country and year;
- error distributions and extreme misses;
- systematic over- or underprediction;
- performance for high and low target values; and
- whether candidate rankings change across subgroups.

Country-level results contain only five test observations per country. They are diagnostic signals, not precise country performance claims. Always report their denominators. Observed-versus-predicted time plots can expose bias and missed changes that MAE and RMSE compress into one number.

Recognize overfitting and distribution shift

Overfitting occurs when a workflow adapts to training details that do not generalize: flexible models can reduce training error while increasing unseen error, and simpler models can **underfit** by omitting stable predictive structure. Validation evidence, not test-set optimization, should balance these risks.

Repeated country observations may be autocorrelated — adjacent years are often more similar than distant ones — so a random split can place near neighbors from the same country on both sides and overstate later performance.

Distribution shift means inputs, outcomes, or their relationship change between development and use. Trends, agricultural innovation, climate extremes, measurement changes, and new countries can all create shift; compare periods descriptively, but do not redesign the final evaluation silently after inspecting test outcomes.

Stationarity cannot be proved from a short historical panel. The design should anticipate change, and the conclusion should stay bounded to the evaluated countries and years.

Handle transformed outcomes carefully

The project models `log_yield`. This can stabilize variance and represent multiplicative differences additively, but it changes interpretation.

Exponentiating a predicted mean log outcome does not generally equal the mean outcome on the original scale. A naive back-transformation can be biased due to the nonlinear exponential function. For the core exercise, compare candidates consistently on the log scale and state that scope.

If an intended use requires tonnes per hectare, define the desired quantity, justify a back-transformation method, and evaluate predictions on that scale as an extension. Do not combine log-scale and original-scale metrics in one model ranking.

Define reproducible prediction artifacts

An auditable evaluation should preserve:

- the prediction contract and intended exclusions;
- a deterministic split rule or split manifest;
- versions and checksums for input data;
- feature and preprocessing definitions;
- fitted objects or enough code to recreate them;
- one row-level table containing keys, observations, and predictions;
- overall and subgroup metric tables;
- diagnostic figures;
- software environment information; and
- a report stating limitations and intended use.

Artifacts should be generated by scripts rather than edited manually. Stable keys must connect predictions to source rows, and the workflow should fail when required inputs, years, or columns are absent.

Bound interpretation and use

The maize evaluation supports one narrow statement: how three predefined procedures performed for known project countries in 2018–2022 after being fitted on 1990–2017 observations.

It does not establish:

- performance in countries absent from training;
- performance after 2022;
- operational pre-season forecast accuracy;
- robustness to unprecedented conditions;
- a causal effect of year, country, or precipitation; or
- suitability for high-stakes resource allocation.

Model documentation should say where the workflow was evaluated, where it can fail, which evidence supports it, and when it needs reevaluation.

Check your understanding

1. Which elements turn “predict maize yield” into a prediction contract?
 2. Why is 2018–2022 excluded from both fitting and model invention?
 3. Which task would a leave-country-out split evaluate?
 4. Give two leakage routes involving seasonal precipitation.
 5. Why must preprocessing estimates be learned from training data only?
 6. How do MAE and RMSE weight large errors differently?
 7. Why can aggregate performance hide an unsuitable model?
 8. What exactly does the current temporal test support?
 9. Why is exponentiating a mean log prediction not automatically an unbiased mean-yield prediction?
 10. Which artifacts allow another analyst to reproduce the evaluation?
-

Further resources

- [An Introduction to Statistical Learning](#) covers the bias–variance trade-off, resampling, regression, and model assessment.
 - [Tidy Modeling with R](#) explains data splitting, resampling, recipes, tuning, and evaluation using tidy-models.
 - [Forecasting: Principles and Practice](#) provides an open treatment of time-aware evaluation, residuals, accuracy, and forecasting workflows.
 - [scikit-learn: Model selection and evaluation](#) is a useful language-independent reference despite its Python examples.
 - [The Turing Way: Reproducible Research](#) connects reproducibility, documentation, and transparent computational workflows.
 - Kuhn, M. and Johnson, K. (2013). *Applied Predictive Modeling*. Springer, provides a practical treatment of preprocessing, resampling, and evaluation.
-

Continue to Application

Continue with [Evaluate predictive models for maize yield](#) to inspect the contract, run the benchmark workflow, and evaluate its evidence.

10.3) Evaluate predictive models for maize yield

Outline

- Outline
 - Learning objectives
 - Place in the session
 - Scenario and deliverables
 - Before you begin
 - 1. Write the prediction contract
 - 2. Audit the temporal split
 - 3. Inspect the candidate baselines
 - 4. Run the predictive workflow
 - 5. Reconstruct and verify the metrics
 - 6. Diagnose errors by country and year
 - 7. Compare evidence with intended use
 - 8. Write a bounded predictive conclusion
 - Independent extension
 - Troubleshooting
 - Completion checklist
 - Reflect on the application
 - Further resources
-

Learning objectives

After completing this exercise, you should be able to:

- document a predictive task before evaluating models;
 - verify that training and test observations follow the intended temporal split;
 - generate and validate reproducible row-level predictions;
 - independently calculate, compare, and diagnose MAE and RMSE by country and year;
 - recognize the limits of a five-year, known-country test design; and
 - communicate a bounded recommendation for model use.
-

Place in the session

This is the **Application** part of the Predictive Modeling session:

```
Motivation → Concepts → Application
                        ↑
                    this page
```

Before beginning, review [Why predictive modeling requires unseen-data evaluation](#) and [Understand predictive-modeling concepts](#).

The Explanatory Modeling workflow asked whether modeled relationships could support a causal interpretation. This exercise preserves a separate script, objective, and evidence standard: it evaluates later observations and does not interpret predictive coefficients as causal effects.

Scenario and deliverables

The project team wants a transparent benchmark for predicting recently observed national maize yields:

When models are fitted using country-year observations from 1990–2017, how accurately do simple procedures predict log maize yield for the same countries in 2018–2022?

The project already contains a compact implementation. Audit it as a predictive workflow, reproduce and extend its evaluation, and state what the evidence supports.

Use or create these artifacts:

```
docs/predictive-modeling.md
scripts/model-maize-yield.R
results/tables/maize-yield-predictions.csv
results/tables/model-performance.csv
results/tables/predictive-performance-by-country.csv
figures/predictive-observed-versus-predicted.png
results/predictive-modeling-conclusion.md
```

The first two generated tables belong to the starting workflow; add the country-level table, diagnostic figure, and conclusion reproducibly rather than editing outputs by hand.

Before you begin

Work from the standalone `maize-yield-project` repository. Confirm the branch and working tree:

```
git status --short --branch
```

Restore the declared R environment if required:

```
renv::restore()
renv::status()
```

The predictive input is `data/derived/maize-yield-panel.csv`. Do not edit it manually — recreate it through the earlier acquisition, integration, preparation, and validation stages. Review:

```
README.md
docs/data-preparation.md
docs/descriptive-data-analysis.md
docs/explanatory-modeling.md
scripts/functions.R
scripts/model-maize-yield.R
```

Before running the script, predict which candidate will perform best and write one reason that expectation could fail under temporal change.

1. Write the prediction contract

Create `docs/predictive-modeling.md`, or add an equivalent section if it already exists. Record the contract before inspecting test outcomes.

| Component | Core teaching decision |
|-------------------|---|
| Target | Annual national <code>log_yield</code> |
| Prediction unit | One country-year |
| Target population | Project countries represented during training |
| Training period | 1990–2017 |
| Test period | 2018–2022 |
| Information set | Country identity and calendar year |
| Candidates | Historical mean, common trend, country-plus-time model |
| Evidence | Row-level held-out errors, MAE, and RMSE |
| Intended use | Teaching benchmark for later-period prediction |
| Excluded uses | Operational forecasts, new-country transfer, causal inference |

Add an explicit statement about prediction time: the task does not define a real issue date or operational horizon, so describe it as a held-out later-period benchmark rather than a validated pre-season forecast.

Answer before continuing: why is precipitation absent from the core information set, what must be defined before adding it, and does the test assess later years, new countries, or both? Do not revise the contract silently after seeing which model wins.

2. Audit the temporal split

Read the prepared data and inspect its grain and coverage:

```
library(dplyr)
library(readr)

maize <- read_csv(
  "data/derived/maize-yield-panel.csv",
  show_col_types = FALSE
)

maize |>
  summarise(
    rows = n(),
    countries = n_distinct(country),
    first_year = min(year),
    last_year = max(year),
    missing_log_yield = sum(is.na(log_yield))
  )

split_audit <- maize |>
  mutate(partition = if_else(year <= 2017, "training", "test")) |>
  count(partition, country, name = "rows") |>
```

```

  arrange(partition, country)

split_audit

```

Verify these invariants: each row represents one country-year; training ends in 2017 and testing starts in 2018; every test country also exists in training; no country-year appears twice; and no test observation enters a training summary or model call.

```

stopifnot(!anyDuplicated(maize[c("country", "year")]))

training <- maize |> filter(year <= 2017)
testing <- maize |> filter(year >= 2018)

stopifnot(max(training$year) == 2017)
stopifnot(min(testing$year) == 2018)
stopifnot(all(unique(testing$country) %in% unique(training$country)))

```

Write the audit to `results/tables/` if not already recorded. A random country-year split would answer a different question — training on later years while evaluating earlier ones, with nearby observations from one country on both sides. The temporal split also does not test transfer to an unseen country, since the country model requires levels already seen in training.

3. Inspect the candidate baselines

Open `scripts/model-maize-yield.R` and identify every object learned from training data:

```

historical_mean <- mean(training$log_yield, na.rm = TRUE)
trend_model <- lm(log_yield ~ year, data = training)
country_model <- lm(log_yield ~ year + country, data = training)

```

| Candidate | Learned information | Main limitation |
|-------------------|--------------------------------------|-----------------------------------|
| Historical mean | One average log yield | Ignores country and time |
| Common trend | Intercept and common year slope | Ignores country differences |
| Country plus time | Country differences and common slope | Assumes common linear time change |

Confirm that each object is fitted only from `training`. Testing may supply country and year to `predict()`, but `testing$log_yield` must not enter fitting or preprocessing.

Write down your expected ranking: country differences may favor the country model, but different slopes or later-period shifts may make extrapolation fail, and a few large errors can affect RMSE more strongly than MAE.

4. Run the predictive workflow

From the project root, run:

```
Rscript scripts/model-maize-yield.R
```

Alternatively, run the complete pipeline when dependencies are available:

```
Rscript scripts/run-all.R
```

Inspect outputs rather than trusting a successful exit code:

```
ls -lh results/tables/maize-yield-predictions.csv \
    results/tables/model-performance.csv
```

Verify completeness in R:

```
predictions <- read_csv(
  "results/tables/maize-yield-predictions.csv",
  show_col_types = FALSE
)
performance <- read_csv(
  "results/tables/model-performance.csv",
  show_col_types = FALSE
)

stopifnot(nrow(predictions) == nrow(testing))
stopifnot(!anyDuplicated(predictions[c("country", "year")]))
stopifnot(all(predictions$year >= 2018))

performance
```

Check required columns for missing values, and record the lowest-MAE and lowest-RMSE candidates and the improvement over the historical mean — but do not call the winner “accurate” without a decision-relevant threshold.

5. Reconstruct and verify the metrics

Recalculate metrics independently from row-level predictions:

```
candidate_columns <- c(
  "historical_mean",
  "linear_trend",
  "country_model"
)

verified_performance <- lapply(candidate_columns, function(candidate) {
  errors <- predictions$log_yield - predictions[[candidate]]
})
```



```

tibble(
  model = candidate,
  observations = sum(!is.na(errors)),
  mean_error = mean(errors, na.rm = TRUE),
  mae = mean(abs(errors), na.rm = TRUE),
  rmse = sqrt(mean(errors^2, na.rm = TRUE))
)
}) |>
  bind_rows()

verified_performance

```

Compare these values with `model-performance.csv` using a floating-point tolerance. MAE is the average absolute miss; RMSE weights large misses more; both remain on the log-yield scale, and mean error diagnoses systematic under- or overprediction.

If MAE and RMSE rank models differently, locate the largest errors — their disagreement reflects different loss functions, not automatically a software failure.

6. Diagnose errors by country and year

Reshape predictions and calculate country-level diagnostics:

```

library(tidyr)

prediction_long <- predictions |>
  pivot_longer(
    cols = all_of(candidate_columns),
    names_to = "model",
    values_to = "estimate"
  ) |>
  mutate(
    error = log_yield - estimate,
    absolute_error = abs(error),
    squared_error = error^2
  )

performance_by_country <- prediction_long |>
  group_by(model, country) |>
  summarise(
    observations = n(),
    mean_error = mean(error),
    mae = mean(absolute_error),
    rmse = sqrt(mean(squared_error)),
    .groups = "drop"
  )

write_csv(
  performance_by_country,
  "results/tables/predictive-performance-by-country.csv"
)

```

Every country has only five test observations, so treat patterns as diagnostic: ask whether the overall winner fails for one country, whether errors have a consistent direction, and whether one country-year drives RMSE.

Create an observed-versus-predicted plot:

```
library(ggplot2)

prediction_plot <- ggplot(
  prediction_long,
  aes(x = year, group = model, colour = model)
) +
  geom_line(aes(y = estimate), linewidth = 0.7) +
  geom_point(aes(y = estimate), size = 1.5) +
  geom_line(aes(y = log_yield, group = 1), colour = "black") +
  geom_point(aes(y = log_yield), colour = "black") +
  facet_wrap(~ country, scales = "free_y") +
  labs(
    title = "Observed and predicted maize yield in the test period",
    subtitle = "Black: observed log yield; colour: benchmark prediction",
    x = "Year", y = "Log yield", colour = "Model"
  ) +
  theme_minimal()

ggsave(
  "figures/predictive-observed-versus-predicted.png",
  prediction_plot,
  width = 10, height = 7, dpi = 300
)
```

Adapt aesthetics to project conventions while preserving scale, period, and model identity in the labels.

7. Compare evidence with intended use

Return to the contract and assess alignment:

- **Population:** later observations from known countries, not unseen countries or farms.
- **Time:** one five-year period gives limited evidence about later extrapolation and none about unprecedented conditions.
- **Information:** country and year are proxies, not measurements of agricultural mechanisms.
- **Loss:** MAE and RMSE quantify log-scale errors without a policy-specific cost or tolerance.
- **Causality:** useful prediction does not show that country, year, or precipitation causes yield.

The comparison can rank transparent benchmarks for this test; it cannot by itself certify an operational crop forecast or a high-stakes decision system.

8. Write a bounded predictive conclusion

Generate `results/predictive-modeling-conclusion.md` from the analysis script or report step. Include the contract's key fields, MAE and RMSE for every candidate, the improvement over the historical-mean baseline, notable country- or year-specific errors, whether ranking depends on the metric, and conditions requiring reevaluation.

Use this pattern:

Among the predefined procedures, `[candidate]` produced the lowest `[metric]` for `[n]` held-out country-years from 2018–2022, improving on the historical-mean baseline by `[amount]`. Performance differed by `[country/year pattern]`, and the short test period provides limited evidence. The result supports this model only as a reproducible benchmark for later observations from the represented countries. It does not validate an operational forecast, transfer to new countries, or a causal claim.

Replace every placeholder with generated evidence. If complexity does not beat the baseline, report that result directly.

Independent extension

Choose **one** extension, keep the test set protected, update the contract, and compare against the same baseline:

- **Country-mean baseline:** calculate country means from training only, join them to test rows, and compare with the country-plus-time model.
- **Rolling-origin validation:** within 1990–2017, define expanding training windows and check whether rankings stay stable, leaving 2018–2022 untouched for the final assessment.
- **Precipitation-based task:** specify pre-season, in-season, or post-season framing, identify which CHIRPS observations exist at the issue date, and add the feature without leakage.
- **Original-scale yield:** define and justify a back-transformation to a conditional median or mean, and evaluate it without mixing scales in one ranking.

Troubleshooting

- **renv out of sync:** run `renv::status()` and restore from `renv.lock` rather than editing the lockfile to suppress the message.
- **Prepared panel missing:** run the preceding acquisition, validation, integration, and preparation stages rather than copying an undocumented panel.
- **Test country new to the country model:** decide whether the task now concerns new-country transfer and adopt a grouped evaluation, rather than silently coercing or dropping it.
- **Predictions contain missing values:** check predictors, factor levels, formulas, and `predict()` output, and report exclusions explicitly.
- **Reconstructed metrics or rankings disagree:** confirm the target, error convention, log scale, test rows, and RMSE formula, then inspect large residuals instead of one summary.
- **Quarto cannot find a report:** run from the repository root; in a container, ensure changed `scripts/` and `reports/` are mounted, not baked into an older image.

Completion checklist

- ☐ The contract fixes target, unit, information, split, metrics, and use.
 - ☐ Training ends in 2017 and testing begins in 2018, and every test country occurs in training.
 - ☐ Models and learned operations use training data only, and the test set was not used to invent or tune candidates.
 - ☐ Row-level predictions are keyed by country and year.
 - ☐ MAE and RMSE were independently verified.
 - ☐ Performance was inspected overall and by country or year, with a diagnostic figure and reported sample sizes and target scale.
 - ☐ The conclusion compares candidates with a baseline and excludes unsupported operational and causal claims.
 - ☐ Artifacts can be recreated from documented code and environment files, and repository changes were reviewed before committing.
-

Reflect on the application

1. Which workflow decision most protects the credibility of performance?
 2. How would the task change for a country absent from training?
 3. Is the best candidate's gain stable across countries?
 4. Which metric reflects the intended error cost, and what is missing to decide?
 5. Which operations would leak information if learned from the complete panel?
 6. What evidence is needed before operational use, and how does that differ from the causal conclusion?
 7. Which artifacts let another analyst audit the result?
-

Further resources

- [Tidy Modeling with R](#) covers reproducible splitting, fitting, and evaluation in R.
- [Forecasting: Principles and Practice](#) explains time-aware evaluation and forecasting limitations.
- [An Introduction to Statistical Learning](#) introduces regression, resampling, and the bias-variance trade-off.
- [FAO crop and livestock statistics](#) supplies context for the source yield data.
- [CHIRPS](#) documents precipitation data available for a carefully defined extension.

Keep the conclusion narrow: this exercise evaluates transparent prediction procedures on one later period, and its value lies in honest, reproducible evaluation — not in claiming an operational forecasting system.